BasicAnalysis Documentation

Release 2026.10.05

BSC Performance Tools

Oct 29, 2026

CONTENTS:

1 Introduction **3**
- 1.1 Parallel execution environments . . . 3
- 1.2 Analysis workflow . . . 4

2 Getting Started **5**
- 2.1 Requirements . . . 5
- 2.2 Python dependencies . . . 5
- 2.3 Installation . . . 6
- 2.4 Running a first analysis . . . 6
- 2.5 Analysis results . . . 6
- 2.6 Next steps . . . 7

3 Running BasicAnalysis **9**
- 3.1 Standard workflow . . . 9
- 3.2 Metric workflow . . . 9
- 3.3 Scaling model . . . 10
- 3.4 Selecting the MPI+GPU metric model . . . 11
- 3.5 Dimemas simulation . . . 11
- 3.6 Trace-mode detection . . . 12
- 3.7 Trace ordering . . . 12
- 3.8 Comparing programming models . . . 12
- 3.9 Parallel trace analysis . . . 13
- 3.10 Maximum trace size . . . 14
- 3.11 Debug information . . . 14
- 3.12 Version information . . . 14
- 3.13 Command-line help . . . 14

4 Staged Analysis Workflow **15**
- 4.1 Trace analysis . . . 16
- 4.2 Merging raw data . . . 17
- 4.3 Computing metrics from merged data . . . 18
- 4.4 Reanalyzing failed traces . . . 18
- 4.5 Complete example . . . 19
- 4.6 Choosing between the workflows . . . 19

5 Performance Analysis Methodology **21**
- 5.1 Hierarchical efficiency analysis . . . 21
- 5.2 Parallel Runtime Model . . . 24
- 5.3 Runtime-Specific Analysis . . . 27
- 5.4 Host/Device Model . . . 31
- 5.5 I/O analysis . . . 34
- 5.6 Scaling analysis . . . 35
- 5.7 Comparison of programming models . . . 37
- 5.8 Complementary analytical views . . . 38

6 Performance Metrics **39**
6.1 From traces to efficiency metrics . . . 39
6.2 Simple Metrics . . . 41
6.3 Hybrid MPI+X Metrics for CPU-Based Applications . . . 45
6.4 Hybrid MPI+GPU Metrics . . . 52
6.5 GPU Traces Generated with nsys2prv . . . 64
6.6 OpenMP Runtime-Specific Metrics . . . 73
6.7 I/O Metrics . . . 76
6.8 General Metrics . . . 81
6.9 Metric Availability . . . 83

7 Performance Report **85**
7.1 Report Interface . . . 85
7.2 Execution Overview . . . 86
7.3 Parallel Runtime Model . . . 87
7.4 Runtime-Specific Analysis . . . 88
7.5 Host/Device Model . . . 88
7.6 GPU Hardware Counters . . . 90
7.7 I/O Analysis . . . 90
7.8 Scaling . . . 93
7.9 Metric Details . . . 94
7.10 Comparing Complementary Views . . . 95
7.11 Exporting Report Content . . . 95
7.12 Programming Model Comparison . . . 97
7.13 Following the Analysis Workflow . . . 98

8 Output **99**
8.1 Performance Report . . . 99
8.2 Numerical Output . . . 99
8.3 Static Visualizations . . . 101
8.4 Intermediate Analysis Files . . . 101

9 Limitations **103**
9.1 Simulation-Derived Metrics . . . 103
9.2 Number of Parallel Runtimes . . . 103
9.3 Hierarchical Execution Model . . . 103
9.4 GPU Execution-Domain Analysis . . . 104
9.5 GPU Trace Sources: Extrae and nsys2prv . . . 104
9.6 I/O Analysis . . . 106
9.7 Comparison of Programming Models . . . 107

BasicAnalysis automates the extraction of POP performance metrics from Paraver traces. These metrics help diagnose how efficiently a parallel application is running with respect to performance factors such as load balance, communication, and computation scalability, and help identify factors that may limit application scalability.

The tools extracts the performance data required to compute the efficiency metrics and organizes the results into complementary analytical views. The generated reports expose the relationships between the metrics and provide interpretation guidance, possible causes of efficiency losses, and possible next steps for further performance analysis.

CHAPTER

ONE

INTRODUCTION

BasicAnalysis automates the extraction of POP performance metrics from Paraver traces. POP metrics help diagnose how efficiently a parallel application is running with respect to basic performance factors such as load balance and communication, as well as identify which factors may limit application scalability.

The tool automatically extracts from the traces the performance data required to compute the efficiency metrics and organizes the results into complementary views. These views expose the relationships between the metrics and provide different perspectives on the performance factors represented by the model.

The main analytical views provided by BasicAnalysis are:

- **Execution Overview**, which summarizes the analyzed traces, execution configurations, parallel resources, and general performance characteristics.
- **Parallel Runtime Model**, which presents the main application-level efficiency factors and their hierarchical relationships and, for hybrid applications, shows the contribution of the active parallel runtimes to Parallel Efficiency.
- **Runtime-Specific Analysis**, which provides additional metrics for the active parallel runtimes when runtime-specific information is available.
- **Host/Device Model**, which provides complementary Device and Host views for accelerator applications and helps identify where accelerator-related inefficiencies manifest.
- **Scaling**, which shows how efficiency metrics and other performance indicators evolve across the analyzed execution configurations.

The generated interactive and printable reports preserve the relationships between the efficiency metrics and provide metric definitions, interpretation guidance, possible causes of efficiency losses, and possible next steps for further performance analysis.

1.1 Parallel execution environments

The efficiency metrics implemented by BasicAnalysis are based on general performance factors of parallel executions and are therefore not restricted to a specific parallel programming model. Since most of the metrics are derived from useful computation and execution time, BasicAnalysis can be applied to different parallel execution environments represented in Paraver traces.

Examples of parallel programming models and combinations currently handled by BasicAnalysis include:

- MPI
- OpenMP
- MPI+OpenMP
- MPI+CUDA
- MPI+HIP

Depending on the parallel execution environment, BasicAnalysis may provide additional runtime-specific metrics and decompositions. The exact set of metrics available also depends on the information contained in the trace and the selected analysis model.

1.2 Analysis workflow

BasicAnalysis can be executed directly for a set of Paraver traces or through a staged workflow in which trace analysis, raw-data merging, and metric computation are performed independently.

The standard execution workflow is introduced in *Getting Started*, while the staged workflow is described in detail in *Staged Analysis Workflow*.

CHAPTER

TWO

GETTING STARTED

This chapter describes the basic requirements for running **BasicAnalysis** and shows how to perform a first analysis from a set of Paraver traces.

For a complete description of the command-line options and execution modes, see *Running BasicAnalysis*.

2.1 Requirements

BasicAnalysis requires Python 3 and relies on tools from the BSC Performance Tools ecosystem to extract performance information and, when required, simulate idealized executions.

The main external requirements are:

- **Paraver / paramedir**, used to extract performance data from Paraver traces.
- **Dimemas**, used when simulation is required to compute communication efficiency submetrics such as Serialization Efficiency and Transfer Efficiency.

The tools are available from:

- Paraver: https://tools.bsc.es/paraver
- Dimemas: https://tools.bsc.es/dimemas

The corresponding executables must be available through the `PATH` environment variable. A typical configuration is:

```
export PATH=<paraver-install-dir>/bin:$PATH
export PARAVER_HOME=<paraver-install-dir>

export PATH=<dimemas-install-dir>/bin:$PATH
export DIMEMAS_HOME=<dimemas-install-dir>
```

2.2 Python dependencies

BasicAnalysis requires Python 3.

Additional Python packages are used for data processing, plotting, and report generation. The required packages and their installation are described with the BasicAnalysis distribution.

Some output functionality may depend on optional external tools. In particular, generation of the printable PDF performance report requires a supported Chromium-compatible browser.

2.2.1 PDF report export

BasicAnalysis generates an interactive HTML performance report as part of the analysis results.

A printable PDF version of the report can be generated on demand using the **Export** function available in the interactive report. PDF generation is not performed automatically during a BasicAnalysis execution.

The PDF export requires a supported Chromium-compatible browser to be available on the system, such as:

- `chromium`
- `chromium-browser`
- `google-chrome`
- `google-chrome-stable`

This separation allows BasicAnalysis to run on HPC systems where a compatible browser may not be installed. The interactive HTML report and the remaining analysis outputs are generated independently of PDF export.

2.3 Installation

BasicAnalysis does not require a separate installation procedure.

Clone or copy the BasicAnalysis repository to the desired location. The directory containing the BasicAnalysis executable scripts can optionally be added to `PATH`:

```
export PATH=<basicanalysis-dir>:$PATH
```

2.4 Running a first analysis

The standard BasicAnalysis workflow analyzes one or more Paraver traces. The tool accepts both uncompressed `.prv` traces and compressed `.prv.gz` traces and computes the corresponding efficiency metrics and other performance indicators, tables, plots, and reports.

BasicAnalysis can be executed with:

```
modelfactors.py [options] <list-of-traces>
```

The trace list can contain explicit trace filenames, multiple traces, or wildcard expressions. Only valid Paraver traces are retained for analysis.

For example:

```
modelfactors.py application.prv
```

Compressed trace:

```
modelfactors.py application.prv.gz
```

Several execution configurations can be analyzed together:

```
modelfactors.py trace_1.prv trace_2.prv trace_3.prv
```

Wildcards can also be used:

```
modelfactors.py *.prv
```

When several traces are analyzed, BasicAnalysis can compare their performance and evaluate how the efficiency factors evolve across the execution configurations.

2.5 Analysis results

A standard BasicAnalysis execution can produce:

- raw performance data;
- efficiency and model-factor tables;
- CSV files;

- performance and scalability plots; and
- an interactive HTML performance report.

The performance report organizes the analysis into complementary views such as Execution Overview, Parallel Runtime Model, Runtime-Specific Analysis, Host/Device Model when applicable, and Scaling when several configurations are available.

The interactive report can also be exported as a printable PDF when the required browser support is available.

A detailed description of the generated files is provided in *Output*.

2.6 Next steps

After completing a first analysis:

- See *Running BasicAnalysis* for the available execution options and analysis configuration.
- See *Staged Analysis Workflow* when traces should be analyzed independently and merged before computing the final metrics.
- See *Performance Analysis Methodology* to understand the BasicAnalysis performance-analysis methodology.
- See *Performance Report* for guidance on reading the generated performance report.
- See *Performance Metrics* for the definitions of the efficiency metrics.

CHAPTER

THREE

RUNNING BASICANALYSIS

This chapter describes how to run the standard BasicAnalysis workflow and configure the analysis.

For workflows in which trace analysis and metric computation need to be performed independently, see *Staged Analysis Workflow*.

3.1 Standard workflow

The standard BasicAnalysis workflow is executed with `modelfactors.py`:

```
modelfactors.py [options] <list-of-traces>
```

The command accepts one or more Paraver traces in either uncompressed `.prv` format or compressed `.prv.gz` format.

Trace filenames can be specified explicitly:

```
modelfactors.py trace_1.prv trace_2.prv trace_3.prv
```

Wildcard expressions can also be used, for example:

```
modelfactors.py *.prv
```

or for compressed traces:

```
modelfactors.py *.prv.gz
```

BasicAnalysis automatically filters the input list to retain valid Paraver traces, including compressed traces, and excludes traces generated internally by simulation.

By default, traces are ordered according to their parallel configuration before the metrics are computed. When several execution configurations are provided, BasicAnalysis uses them to evaluate performance and scalability trends.

3.2 Metric workflow

BasicAnalysis automatically selects the metric workflow according to the programming models detected in the analyzed traces.

If all traces correspond to simple execution models, BasicAnalysis uses the simple metric workflow. If at least one trace contains hybrid parallelism, BasicAnalysis uses the hybrid metric workflow.

Therefore, users normally do not need to select the metric workflow explicitly.

The metric workflow can be controlled manually with:

```
-m, --metrics {simple,hybrid}
```

The available modes are:

simple
Force the use of the simple metric workflow.

hybrid
Allow BasicAnalysis to use the hybrid metric workflow when required by the detected trace modes. If all analyzed traces correspond to simple execution models, BasicAnalysis automatically uses the simple metric workflow.

The default is `hybrid`.

3.3 Scaling model

The scaling model can be selected with:

```
-s, --scaling {weak,strong,auto}
```

The available modes are:

strong
Use the strong-scaling formulation when computing scalability metrics.

weak
Use the weak-scaling formulation when computing scalability metrics.

auto
Let BasicAnalysis determine the scaling behavior from the analyzed executions.

The default is `auto`.

3.3.1 User-defined workload

When the workload of each execution is known, for example the batch size of an inference or training run, it can be given instead of a scaling model:

```
--workload W1,W2,...
--ratios-workload R1,R2,...
```

--workload
Workload sizes, one per trace. The workload ratio of each trace is its size divided by the size of the reference trace.

--ratios-workload
Workload ratios, one per trace, already relative to the reference trace (the reference trace has ratio 1). If the ratio of the reference trace is not 1, all ratios are divided by it and a warning is printed.

The values are matched, in order, with the `.prv` traces as they are given on the command line, before the traces are filtered by size or sorted; each trace keeps its value after sorting. Shell wildcards expand in alphabetical order (`b1`, `b128`, `b16`, `b2`, ...), so the traces should be listed explicitly in the same order as the values. The trace, workload, and ratio used for each execution are printed in the scaling model summary and shown in the **Scaling** section of the report. With `compute_metrics_from_merged.py`, the values are matched with the trace list of the merged file.

The two options cannot be combined with each other or with `--scaling weak` or `--scaling strong`, and the number of values must be the same as the number of traces. All values must be greater than zero.

For example, four vLLM runs on the same GPU with batch sizes 1, 8, 64, and 256:

```
modelfactors.py --workload 1,8,64,256 \
    vllm_b1.prv vllm_b8.prv vllm_b64.prv vllm_b256.prv
```

In the Host/Device model (MPI+GPU, GPU without MPI, and nsys2prv traces), the Host and the device can have their own workload, when the work of one of them does not grow like the workload of the application:

```
--host-workload W1,W2,...     --host-ratios-workload R1,R2,...
--device-workload W1,W2,...   --device-ratios-workload R1,R2,...
```

Each pair has the same size and ratio forms as `--workload` and `--ratios-workload`, with one value per trace in the same order. They need `--workload` or `--ratios-workload`: the Host or device workload that is not given is that of the application. For example, if the Host does about the same work for every batch size (the same number of steps and kernel launches) while the GPU work grows with the batch:

```
modelfactors.py --workload 1,8,64,256 --host-ratios-workload 1,1,1,1 \
    vllm_b1.prv vllm_b8.prv vllm_b64.prv vllm_b256.prv
```

In execution models with a single domain (MPI, MPI+OpenMP, OpenMP or pthreads, Burst mode), the Host and device workloads are ignored with a warning. The Host and device values are printed and shown with the workload of the application.

With a user-defined workload, the automatic detection of weak or strong scaling is not used, and the report shows it as *Not used*. The reference is the first execution, as in weak and strong scaling (see *Performance Analysis Methodology*).

The scaling model affects the computation and interpretation of scalability metrics. The detected and selected scaling models are reported in the **Scaling** section of the performance report.

See *Performance Analysis Methodology* and *Performance Metrics* for the definition and interpretation of the scalability metrics.

3.4 Selecting the MPI+GPU metric model

For MPI+GPU applications, BasicAnalysis computes both the classic multiplicative model and the TALP-based model.

The model displayed in the standard output can be selected with:

```
-pop-model, --pop_model_to_apply {classic,talp}
```

The available models are:

`classic`
: Display the classic multiplicative model in the standard output.

`talp`
: Display the TALP-based model in the standard output.

The default is `talp`.

This option affects the model presented in the standard output. The interactive Performance Report provides access to both models, allowing users to inspect the complementary information provided by each one.

The classic and TALP-based models, including the Host and Device execution-domain analysis associated with the TALP-based model, are described in *Performance Analysis Methodology* and *Performance Metrics*.

3.5 Dimemas simulation

Some communication-efficiency metrics require information obtained from an idealized execution simulated with Dimemas.

Simulation can be disabled with:

```
-skip-simul, --skip-simulation
```

When this option is used, BasicAnalysis does not run the Dimemas simulation. Metrics that depend on simulated execution information may consequently be reported as unavailable.

Additional simulation options are available for traces containing OpenMP or CUDA activity:

```
-somp, --simulation_openmp
-scuda, --simulation_cuda
```

--simulation_openmp
Enable simulation of OpenMP events.

--simulation_cuda
Enable simulation of CUDA events.

The role of Dimemas, the idealized execution, and the metrics that depend on simulation are described in *Performance Metrics*.

3.6 Trace-mode detection

BasicAnalysis automatically determines the programming and tracing mode of the analyzed traces.

The source used for trace-mode detection can be selected with:

```
-tmd, --trace_mode_detection {pcf,prv}
```

pcf
Detect the trace mode using the Paraver configuration file. This is the default.

prv
Inspect the trace itself to determine the available instrumentation. This mode is useful for customized traces, such as filtered or cut traces, whose contents may no longer correspond exactly to the original `.pcf` description.

The default is `pcf`.

3.7 Trace ordering

By default, BasicAnalysis orders the analyzed traces according to their parallel configuration.

This behavior is controlled with:

```
-ord, --order_traces {yes,not}
```

yes
Order the traces before performing the comparative analysis.

not
Preserve the input trace order.

The default is `yes`.

Trace ordering is particularly relevant when several configurations are analyzed because the first execution is used as the reference for several relative performance and scalability metrics.

Preserving the input order is also useful when the analyzed traces are not a scaling study, for example when comparing executions with the same parallel resources but different problems, or executions that use different programming models (see *Comparing programming models*). In these cases, list the trace to be used as reference first and use `-ord not`.

3.8 Comparing programming models

BasicAnalysis can compare executions that use different programming models, for example a new MPI+CUDA implementation of an application with its legacy MPI or MPI+OpenMP version, or the MPI+CUDA and MPI+HIP versions of the same application.

The comparison is activated automatically when more than one trace is analyzed and the traces do not all use the same programming model. No additional option is required:

```
modelfactors.py app_mpi.prv app_mpi_cuda.prv app_mpi_openmp.prv
```

When all traces use the same programming model, BasicAnalysis performs the usual analysis and its output is not affected.

In the comparison:

- Each trace is described with the efficiency metrics of its own programming model. Metrics that do not apply to a programming model, such as Device metrics for an MPI execution, are left empty.
- Scaling detection and the metrics computed relative to a reference execution, such as Global Efficiency and Computation Scalability, are not computed across programming models.
- Elapsed time and Speedup remain available. Speedup is the runtime ratio relative to the first trace.
- The interactive report contains a **Programming Model Comparison** tab and one tab per programming model with the complete analysis of its traces.

The first trace is the reference for Speedup. By default, traces are ordered according to their parallel configuration (see *Trace ordering*). To use a specific execution as reference, for example the legacy version of the application, list it first and preserve the input order:

```
modelfactors.py -ord not app_legacy_mpi.prv app_new_mpi_cuda.prv
```

The comparison is supported for **Detailed** traces of MPI-based programming models, such as MPI, MPI+OpenMP, MPI+CUDA, and MPI+HIP. If a trace set with different programming models contains a Burst trace or a trace without MPI, BasicAnalysis stops and lists the traces that cannot be compared. These traces should be analyzed separately.

The comparison always uses the hybrid metric workflow, which covers all MPI-based programming models, even when `--metrics simple` is selected.

The comparison is also available in the staged workflow (*Staged Analysis Workflow*). The methodology of the comparison is described in *Comparison of programming models*, and the corresponding report views in *Programming Model Comparison*.

3.9 Parallel trace analysis

Independent traces can be analyzed concurrently to reduce the total analysis time.

The number of concurrent workers is controlled with:

```
--jobs <number|auto>
```

1
: Analyze traces sequentially. This is the default.

<number>
: Analyze up to the specified number of traces concurrently.

auto
: Let BasicAnalysis determine an appropriate number of concurrent workers according to the available resources.

The estimated memory available to each worker can be controlled with:

```
--mem-per-worker-gb <value>
```

This option overrides the automatic memory estimate used when determining parallel trace-processing capacity.

Parallel processing applies to independent trace analyses; it does not change the performance metrics computed for each trace.

3.10 Maximum trace size

The maximum input trace size accepted by BasicAnalysis can be controlled with:

```
-ms, --max_trace_size <MiB>
```

The default maximum size is 1024 MiB.

Traces larger than the configured limit are excluded from the analysis. BasicAnalysis reports the traces that have been excluded. If every input trace exceeds the limit, the analysis terminates without computing metrics.

3.11 Debug information

Additional diagnostic information can be enabled with:

```
-d, --debug
```

This option is useful when validating trace detection, metric computation, or report generation.

3.12 Version information

The installed BasicAnalysis version can be displayed with:

```
-v, --version
```

3.13 Command-line help

The complete set of options supported by the installed BasicAnalysis version can always be obtained with:

```
modelfactors.py --help
```

The command-line help should be considered the authoritative reference for the options available in a particular BasicAnalysis version.

CHAPTER

FOUR

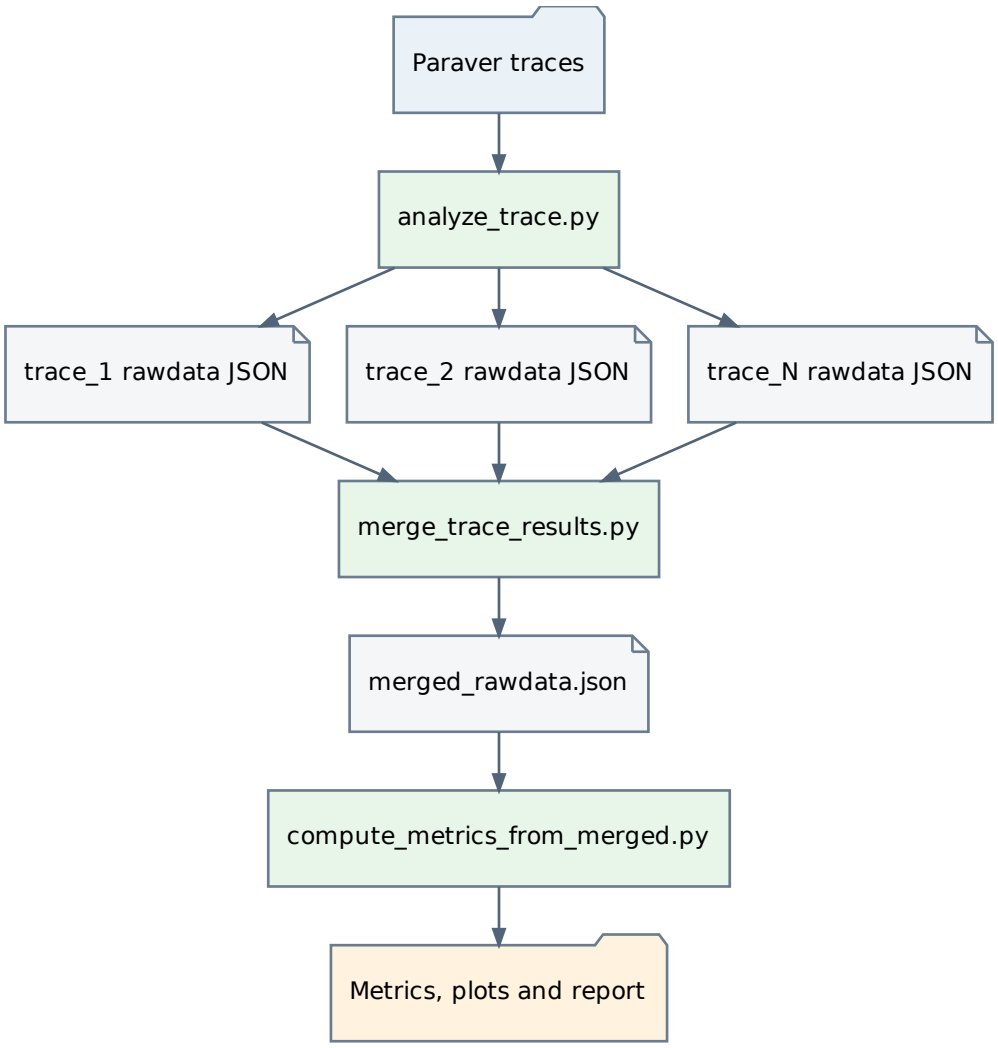
STAGED ANALYSIS WORKFLOW

BasicAnalysis can separate trace processing from metric computation by using a staged analysis workflow.

Instead of analyzing all traces and computing the final metrics in a single `modelfactors.py` execution, the analysis can be divided into three independent stages:

1. **Trace analysis**: analyze each Paraver trace and generate a raw-data JSON file.
2. **Raw-data merge**: combine the per-trace raw-data files into a single merged dataset.
3. **Metric computation**: compute the efficiency metrics and other performance indicators, tables, plots, and performance report.

The workflow can be summarized as:



The staged workflow is particularly useful when analyzing multiple large or computationally expensive traces. Because each trace is analyzed independently, trace processing can be distributed across different jobs or compute nodes. This is especially useful when trace processing or simulation requires significant memory or execution time.

The independent per-trace results also make the analysis more flexible. Once a trace has been successfully analyzed, its raw-data file can be preserved and reused in subsequent stages. If a trace needs to be reanalyzed, only that trace has to be processed again, and the resulting raw-data files can then be merged without repeating the other trace analyses.

Finally, separating trace processing from metric computation allows the final metrics and reports to be regenerated from previously extracted raw data without processing the original traces again.

4.1 Trace analysis

The first stage is performed with `analyze_trace.py`:

```
analyze_trace.py [options] [trace_list ...]
```

The command accepts Paraver traces in both uncompressed `.prv` format and compressed `.prv.gz` format.

For example:

```
analyze_trace.py trace_1.prv trace_2.prv trace_3.prv
```

Compressed traces can be analyzed directly:

```
analyze_trace.py trace_1.prv.gz trace_2.prv.gz
```

Wildcard expressions can also be used:

```
analyze_trace.py *.prv
```

The trace-analysis stage performs the operations required to prepare the raw performance information for each execution. Depending on the selected analysis model, this includes:

- validating the input trace;
- detecting the programming model and trace mode;
- extracting performance data from the trace; and
- running the required Dimemas simulations when applicable.

Each successfully analyzed trace produces an independent raw-data JSON file. These files contain the information required by the later metric-computation stage, so the original trace does not need to be analyzed again when only the final metrics or reports need to be regenerated.

4.1.1 Analysis configuration

The trace-analysis stage supports the analysis options that affect raw-data extraction and simulation, including trace-mode detection, simulation configuration, trace-size limits, and parallel processing of independent traces.

For example:

```
analyze_trace.py --jobs auto *.prv
```

can analyze independent traces concurrently.

The main analysis options are described in *Running BasicAnalysis*. The complete set of options supported by the installed version can be obtained with:

```
analyze_trace.py --help
```

4.2 Merging raw data

After the required traces have been analyzed, their raw-data JSON files can be combined with `merge_trace_results.py`:

```
merge_trace_results.py --output merged_rawdata.json <rawdata-files>
```

For example:

```
merge_trace_results.py --output merged_rawdata.json *.rawdata.json
```

By default, the merge stage orders traces according to the number of processes. This behavior can be controlled with the `-ord` option.

The merge operation combines the independent trace-analysis results into a single dataset that can be used for comparative and scalability analysis.

The merged file preserves the information required for metric computation, including the analyzed trace set, trace metadata, raw performance data, and parallel-configuration information.

Raw-data files do not need to have been generated in the same `analyze_trace.py` execution. This makes it possible to analyze traces independently or to replace the raw data corresponding to a trace that needed to be reanalyzed.

The available command-line options can be displayed with:

```
merge_trace_results.py --help
```

4.3 Computing metrics from merged data

The final stage computes the performance metrics and generates the analysis results from the merged raw-data file:

```
compute_metrics_from_merged.py --merged-input merged_rawdata.json [options]
```

For example:

```
compute_metrics_from_merged.py --merged-input merged_rawdata.json
```

This stage does not analyze the original Paraver traces. It consumes the previously extracted raw data and computes the final efficiency metrics, tables, plots, and interactive performance report.

This separation is useful when the metric configuration or report generation needs to be repeated without rerunning the more expensive trace-analysis stage.

4.3.1 Metric configuration

Options that affect final metric computation can be specified at this stage. These include the metric workflow, scaling model, and the performance model used for MPI+GPU applications.

For example:

```
compute_metrics_from_merged.py \
    --merged-input merged_rawdata.json \
    --scaling strong
```

or:

```
compute_metrics_from_merged.py \
    --merged-input merged_rawdata.json \
    -pop-model classic
```

The complete set of options can be displayed with:

```
compute_metrics_from_merged.py --help
```

4.4 Reanalyzing failed traces

One of the main advantages of the staged workflow is that an unsuccessful trace analysis does not require all traces to be processed again.

For example, suppose three traces are analyzed:

```
analyze_trace.py trace_1.prv trace_2.prv trace_3.prv
```

and the analysis succeeds for `trace_1` and `trace_3` but fails for `trace_2`.

After resolving the problem affecting `trace_2`, only that trace needs to be analyzed again:

```
analyze_trace.py trace_2.prv
```

The available raw-data files can then be merged:

```
merge_trace_results.py --output merged_rawdata.json *.rawdata.json
```

and the final metrics regenerated:

```
compute_metrics_from_merged.py --merged-input merged_rawdata.json
```

The successful analyses of `trace_1` and `trace_3` therefore do not need to be repeated.

4.5 Complete example

A complete staged analysis can be performed as follows.

First, analyze the traces:

```
analyze_trace.py trace_1.prv trace_2.prv trace_3.prv
```

Second, merge the generated raw-data files:

```
merge_trace_results.py --output merged_rawdata.json *.rawdata.json
```

Finally, compute the performance metrics and generate the reports:

```
compute_metrics_from_merged.py --merged-input merged_rawdata.json
```

The resulting analysis follows the same performance-analysis methodology as the standard BasicAnalysis workflow. The difference is that raw-data extraction and final metric computation are performed as independent stages.

4.6 Choosing between the workflows

The standard workflow with `modelfactors.py` is convenient for direct analysis when all traces can be processed in a single execution.

The staged workflow is preferable when:

- a large number of traces must be analyzed;
- individual trace analyses are expensive;
- traces are processed at different times or on different systems;
- failed traces need to be reanalyzed independently; or
- metrics and reports need to be regenerated without repeating trace processing.

Both workflows ultimately use the same BasicAnalysis metric models and performance-analysis methodology.

CHAPTER

FIVE

PERFORMANCE ANALYSIS METHODOLOGY

BasicAnalysis organizes performance analysis through a hierarchy of efficiency metrics and a set of complementary analytical views. The objective is not only to report individual efficiency values, but also to help users understand which performance factors contribute to the observed efficiency losses and which aspects of the execution should be investigated further.

The analysis can progress from application-level behavior toward more specific performance factors, parallel runtimes, and execution components. Depending on the analysis objective, users can combine several analytical views to understand the overall performance behavior or focus on the view most relevant to a particular performance aspect.

The methodology is organized around the following perspectives:

- **Hierarchical Efficiency Analysis**, which decomposes high-level efficiency metrics into progressively more specific contributing factors.
- **Parallel Runtime Model**, which attributes parallel-efficiency behavior to the active parallel runtimes.
- **Runtime-Specific Analysis**, which provides additional metrics describing the behavior of individual parallel runtimes.
- **Host/Device Model**, which provides a complementary Host/Device analysis for accelerator applications.
- **I/O Analysis**, which provides complementary metrics characterizing the contribution and distribution of File I/O activity.
- **Scaling Analysis**, which examines how performance and efficiency factors evolve across execution configurations.
- **Comparison of Programming Models**, which compares executions that use different programming models.

The following sections describe how these perspectives are related and how they can be used according to the objective of the performance assessment.

5.1 Hierarchical efficiency analysis

As BasicAnalysis automates the computation of POP performance metrics, it follows the POP hierarchical metric model, in which high-level efficiencies are decomposed into more specific contributing factors. This hierarchy allows the analysis to progress from an overall efficiency loss toward increasingly specific factors that can explain it.

At the application level, the hierarchy starts with Global Efficiency, which is decomposed into Parallel Efficiency and Computation Scalability:

$$Global\ Efficiency = Parallel\ Efficiency \times Computation\ Scalability$$

These two factors represent the two main sources of inefficiency in parallel applications. Parallel Efficiency captures the overheads introduced by parallel execution, while Computation Scalability captures how the computation scales as the computational resources increase.

5.1.1 Parallel Efficiency

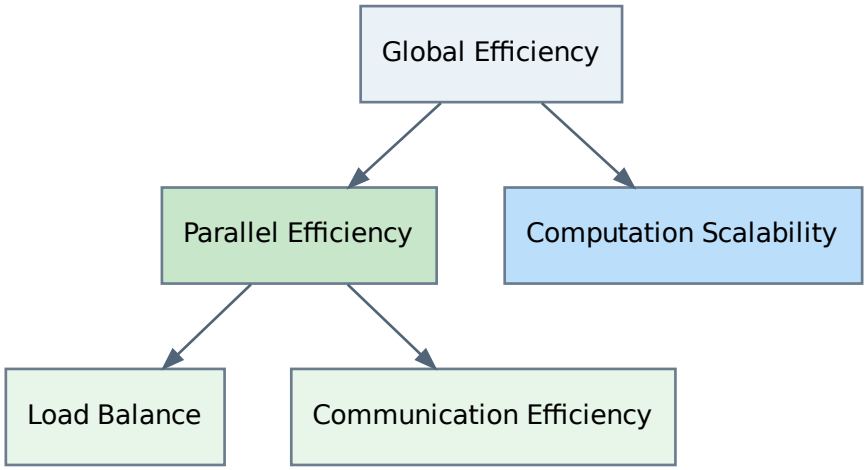
Parallelization requires dividing the application workload among the available parallel units and defining the interactions required among those units to complete the computation.

Parallel Efficiency characterizes the efficiency of this parallelization. Its losses can arise from two fundamental aspects: an uneven distribution of the computational workload among the parallel units and the overhead associated with the communication and coordination required among them.

Therefore, Parallel Efficiency is decomposed as:

$$Parallel\ Efficiency = Load\ Balance \times Communication\ Efficiency$$

This results in the following application-level hierarchy:



Load Balance characterizes how evenly the computational workload is distributed among the parallel units, while Communication Efficiency characterizes the efficiency loss associated with the communication and coordination required among them.

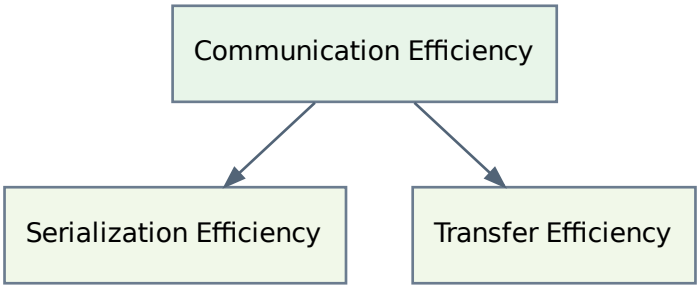
At this level of the hierarchy, communication is a general parallel-execution concept and is not restricted to message passing or to a particular programming model. Its specific interpretation depends on the parallel execution model being analyzed.

Communication Efficiency can be further decomposed to distinguish losses associated with serialization from those associated with data transfer:

$$Communication\ Efficiency = Serialization\ Efficiency \times Transfer\ Efficiency$$

Serialization Efficiency characterizes losses associated with dependencies and synchronization among parallel units, while Transfer Efficiency characterizes the additional cost associated with transferring the information required for their interaction.

The Communication Efficiency branch can therefore be represented as:



The way these communication factors are obtained depends on the parallel execution model. BasicAnalysis directly derives the MPI factors using idealized communication information generated through Dimemas. For hybrid executions, additional runtime contributions can be derived from the relationships between the hybrid- and runtime-level metrics, as described in the Runtime-Specific Analysis section.

5.1.2 Computation Scalability

The other main factor contributing to Global Efficiency is Computation Scalability. While Parallel Efficiency captures the overheads introduced by parallel execution, Computation Scalability characterizes how the computation scales as the computational resources increase.

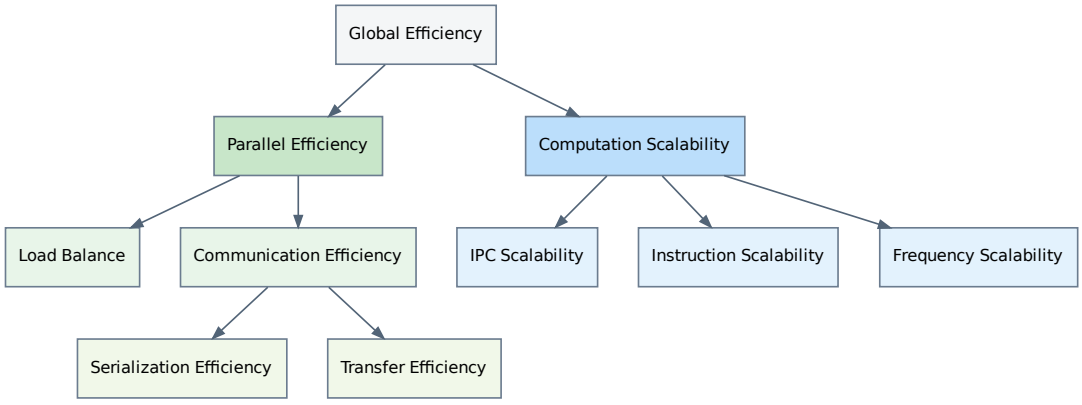
Unlike Parallel Efficiency, Computation Scalability is evaluated relative to a reference execution. It captures changes in the computational behavior as the application scales, independently of the overheads attributed to parallel execution.

Computation Scalability is decomposed as:

$$Computation\ Scalability = IPC\ Scalability \times Instruction\ Scalability \times Frequency\ Scalability$$

The scaling of the computation can therefore be decomposed into three components: Instructions, IPC, and Frequency. These represent the three main factors that determine the duration of the computation: the amount of work performed (Instructions), the rate at which that work is executed (IPC), and the operating speed of the computational resource (Frequency).

Together with the Parallel Efficiency decomposition introduced above, these metrics complete the application-level efficiency hierarchy:



The hierarchy provides a systematic way to navigate an efficiency analysis. When a parent metric shows a significant loss, its child metrics can be inspected to determine which performance factor contributes to that loss.

For example, a low Parallel Efficiency can lead to the inspection of Load Balance and Communication Efficiency. If Communication Efficiency shows the larger loss, Serialization Efficiency and Transfer Efficiency can then be examined to distinguish whether the degradation is primarily associated with dependencies and synchronization or with data-transfer costs.

Similarly, Computation Scalability can be investigated through its IPC, Instruction, and Frequency Scalability components to determine which factor contributes to the observed degradation.

The analysis can continue through the hierarchy until the available metrics provide the level of detail required for the performance assessment.

This application-level hierarchy provides the basis for the additional analysis performed by BasicAnalysis. When multiple parallel runtimes participate in the execution, the Parallel Efficiency branch can be extended to distinguish the contribution of each runtime while preserving the same underlying performance factors. This extension is introduced by the Parallel Runtime Model in the following section.

The complete definitions and formulations of these metrics are provided in *Performance Metrics*.

5.2 Parallel Runtime Model

The application-level hierarchy introduced in the previous section can also be applied to applications that combine multiple parallel runtimes. In this case, however, Parallel Efficiency and its contributing factors characterize the combined behavior of the parallel runtimes participating in the execution.

For a hybrid application, this application-level Parallel Efficiency is referred to as **Hybrid Parallel Efficiency**. Similarly, Hybrid Load Balance and Hybrid Communication Efficiency characterize workload distribution and communication and coordination for the hybrid execution as a whole.

This decomposition identifies which performance factor limits the hybrid execution, but it does not reveal how the different parallel runtimes contribute to that factor. To provide this additional level of analysis, BasicAnalysis uses the **Parallel Runtime Model**.

The Parallel Runtime Model introduces a runtime dimension into the Parallel Efficiency hierarchy. For a hierarchical hybrid execution, it distinguishes an **outer parallel runtime** and an **inner parallel runtime**. For example, MPI is the outer runtime and OpenMP the inner runtime in MPI+OpenMP, while MPI is the outer runtime and the accelerator runtime the inner runtime in MPI+CUDA and MPI+HIP.

5.2.1 Parallel-runtime performance factors

The Parallel Runtime Model preserves the three performance factors introduced for parallel execution—Parallel Efficiency, Load Balance, and Communication Efficiency—and evaluates them at different runtime levels.

For compactness, the following notation is used in the equations in this section:

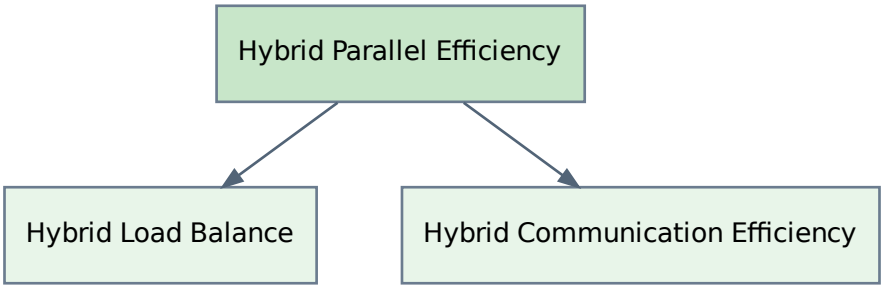
- **PE**: Parallel Efficiency
- **LB**: Load Balance
- **CommE**: Communication Efficiency

The model distinguishes three levels: the **hybrid level**, the **outer-runtime level**, and the **inner-runtime level**.

At the hybrid level, the metrics characterize the complete hybrid parallelization, considering all participating parallel runtimes. The performance-factor decomposition remains:

$$Hybrid_PE = Hybrid_LB \times Hybrid_CommE$$

Here, $Hybrid_LB$ characterizes workload distribution across the complete hybrid execution, while $Hybrid_CommE$ characterizes the communication and coordination overhead associated with all participating parallel runtimes.

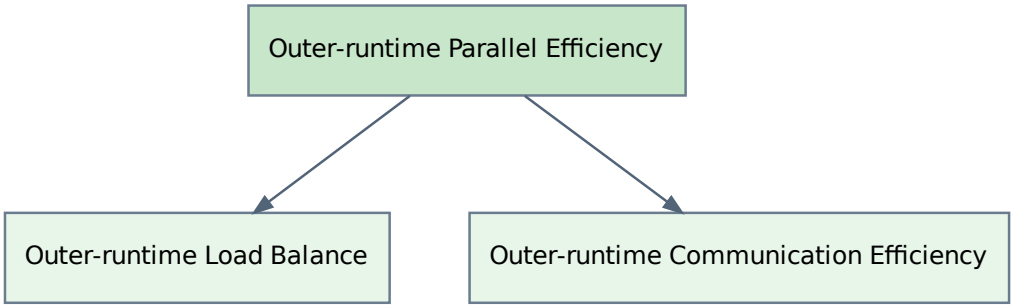


To isolate the contribution of the outer runtime, the execution is analyzed from its perspective. Activity associated with the outer runtime is treated as parallel-runtime overhead, while execution outside that runtime is considered computation at this level.

The same performance-factor decomposition is preserved:

$$Outer_PE = Outer_LB \times Outer_CommE$$

For MPI+OpenMP, MPI is the outer runtime. From the MPI perspective, time inside MPI represents MPI-runtime activity, while execution outside MPI, including OpenMP activity, is considered computation. The resulting MPI metrics therefore characterize the performance factors associated with the outer parallelization.



Once the hybrid and outer-runtime factors are known, the contribution of the inner runtime can be derived using the multiplicative structure of the model. For Parallel Efficiency:

$$Hybrid_PE = Outer_PE \times Inner_PE$$

The same relationship applies to Load Balance and Communication Efficiency:

$$Hybrid_LB = Outer_LB \times Inner_LB$$

$$Hybrid_CommE = Outer_CommE \times Inner_CommE$$

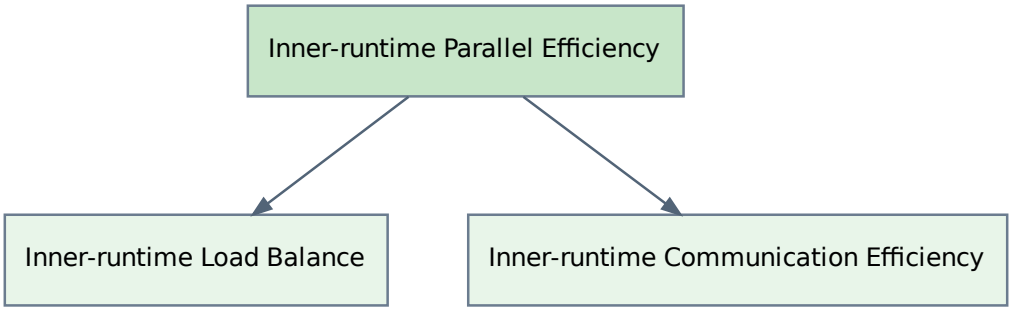
Therefore, the inner-runtime performance factors are obtained as:

$$Inner_PE = \frac{Hybrid_PE}{Outer_PE}$$

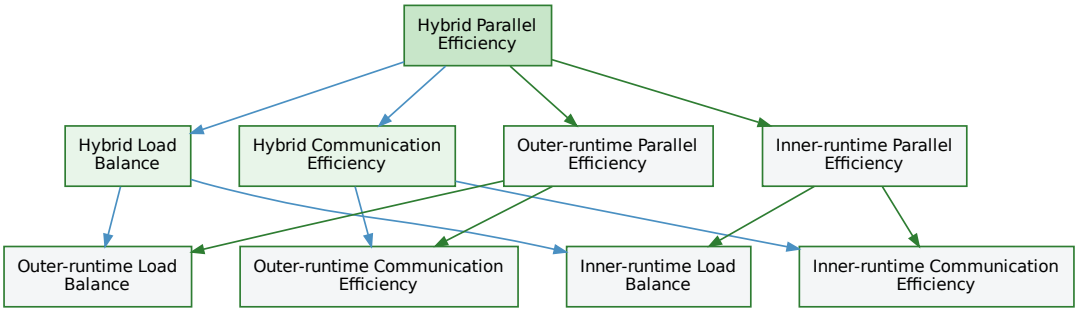
$$Inner_LB = \frac{Hybrid_LB}{Outer_LB}$$

$$Inner_CommE = \frac{Hybrid_CommE}{Outer_CommE}$$

The inner-runtime metrics therefore represent the contribution required to relate the outer-runtime factors to those observed for the complete hybrid execution.



The resulting generic metric hierarchy for a hybrid parallel application can be represented as follows:



The hierarchy can be followed either by performance factor, to examine Load Balance or Communication Efficiency across runtime levels, or by runtime, to examine the factors contributing to the parallel efficiency of the outer or inner runtime.

For an MPI+OpenMP execution, the outer and inner runtimes correspond to MPI and OpenMP, respectively. The generic runtime decomposition described above can therefore be expressed directly in terms of these two runtimes.

For Parallel Efficiency, the relationship becomes:

$$Hybrid_PE = MPI_PE \times OpenMP_PE$$

and the OpenMP contribution is therefore obtained as:

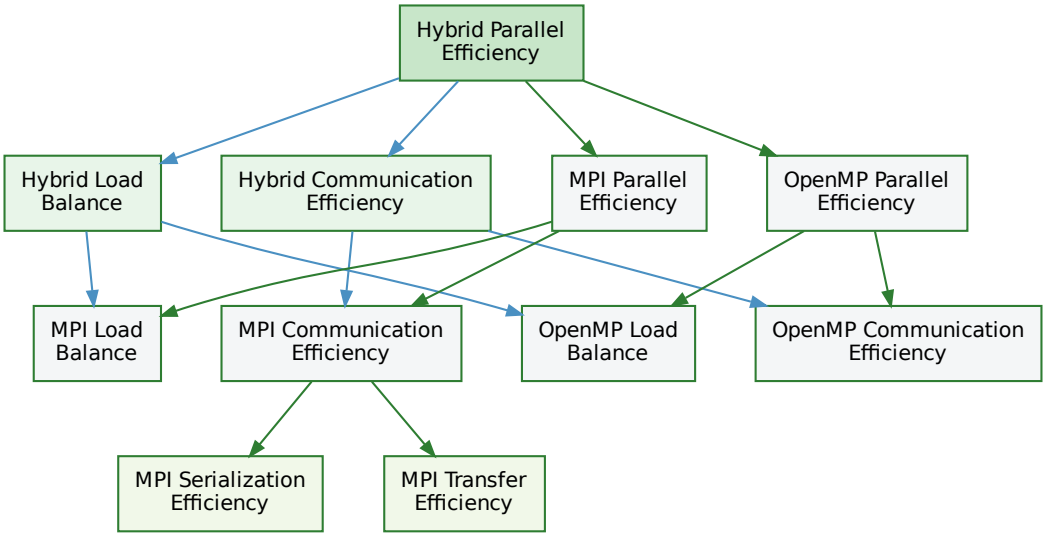
$$OpenMP_PE = \frac{Hybrid_PE}{MPI_PE}$$

The same decomposition applies to the other two performance factors. Hybrid Load Balance is determined by the MPI and OpenMP Load Balance contributions, while Hybrid Communication Efficiency is determined by the corresponding MPI and OpenMP Communication Efficiency contributions:

$$Hybrid_LB = MPI_LB \times OpenMP_LB$$

$$Hybrid_CommE = MPI_CommE \times OpenMP_CommE$$

The resulting Parallel Runtime Model for an MPI+OpenMP application can therefore be represented as:



When MPI is the outer runtime, MPI Communication Efficiency can be further decomposed into MPI Serialization Efficiency and MPI Transfer Efficiency. These factors provide a deeper characterization of the communication losses associated with the MPI runtime.

5.2.2 Interpreting derived runtime contributions

Unlike the hybrid and outer-runtime factors, inner-runtime contributions are not independent efficiency measurements. They are derived from the multiplicative relationship between the hybrid and outer-runtime levels.

Because an inner-runtime contribution is obtained as the ratio between the corresponding hybrid and outer-runtime factors, its value may exceed 100% when the hybrid factor is greater than the outer-runtime factor. Such a value should not be interpreted as an independently measured physical efficiency above its ideal value, but as the relative contribution of the inner runtime within the multiplicative model.

Inner-runtime factors should therefore be interpreted together with the hybrid and outer-runtime factors from which they are derived.

5.3 Runtime-Specific Analysis

The Parallel Runtime Model identifies how the active parallel runtimes contribute to the main parallel-performance factors. Once a runtime is identified as contributing to an efficiency loss, however, additional information may be required to understand the behavior behind that contribution.

BasicAnalysis therefore complements the Parallel Runtime Model with **Runtime-Specific Analysis**. Depending on the programming model, this analysis can either extend the decomposition of the common performance factors or introduce additional metrics that characterize behavior specific to a particular runtime.

These two perspectives are complementary. The Parallel Runtime Model preserves common performance factors across runtime levels, allowing their contributions to the hybrid execution to be distinguished. Runtime-specific metrics provide a more specialized view of the execution and can expose sources of inefficiency that are meaningful for a particular programming model.

The runtime-specific information currently available depends on the parallel programming model and on the information that can be obtained from the analyzed traces.

5.3.1 Extending the Parallel Runtime Model

The Parallel Runtime Model introduced in the previous section attributes Parallel Efficiency, Load Balance, and Communication Efficiency to the parallel runtimes participating in a hybrid execution. When additional information is available, the same multiplicative principle can be extended to the factors explaining Communication Efficiency.

Communication Efficiency can be decomposed into Serialization Efficiency and Transfer Efficiency:

$$CommE = Serialization \times Transfer$$

Serialization Efficiency characterizes losses associated with dependencies and synchronization among parallel units, while Transfer Efficiency characterizes the additional overhead associated with the mechanisms required to perform their interaction.

For an MPI+OpenMP execution, BasicAnalysis extends this decomposition to the hybrid and runtime levels. This makes it possible not only to identify whether Communication Efficiency is limiting the hybrid execution, but also to distinguish how MPI and OpenMP contribute to its Serialization and Transfer components.

Hybrid and MPI communication factors

The MPI and hybrid communication factors are characterized from different execution perspectives.

At the MPI level, BasicAnalysis uses an idealized MPI execution to separate the effects associated with serialization from those associated with data transfer. OpenMP activity is not modeled as an independent source of parallel-runtime overhead at this level.

At the hybrid level, both MPI and OpenMP participate in the execution model. Idealized execution information is used to separate structural limitations from the additional overheads introduced by the mechanisms used to coordinate and communicate among the parallel units.

This provides Serialization and Transfer factors at both the hybrid and MPI levels. The remaining contribution can then be attributed to the OpenMP runtime using the same multiplicative principle introduced by the Parallel Runtime Model.

Deriving the OpenMP contribution

Following the same multiplicative approach used for the other parallel-runtime factors, Hybrid Serialization Efficiency is expressed in terms of the MPI and OpenMP contributions:

$$Hybrid_Serialization = MPI_Serialization \times OpenMP_Serialization$$

Similarly, Hybrid Transfer Efficiency is expressed as:

$$Hybrid_Transfer = MPI_Transfer \times OpenMP_Transfer$$

The OpenMP contributions can therefore be derived from the corresponding hybrid and MPI factors:

$$OpenMP_Serialization = \frac{Hybrid_Serialization}{MPI_Serialization}$$

$$OpenMP_Transfer = \frac{Hybrid_Transfer}{MPI_Transfer}$$

These relationships extend the runtime attribution introduced for Load Balance and Communication Efficiency to the two factors that explain Communication Efficiency.

Interpreting the OpenMP communication factors

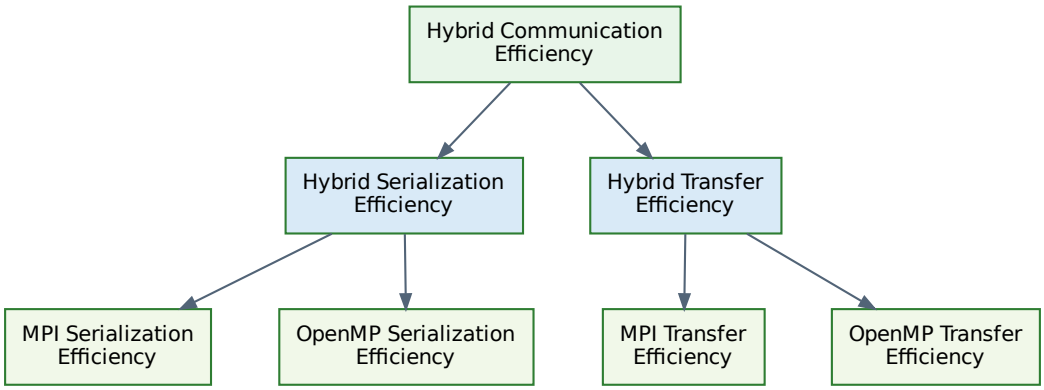
At the OpenMP level, Serialization and Transfer Efficiency should be interpreted according to the mechanisms through which threads interact rather than in terms of message passing.

OpenMP Serialization Efficiency characterizes structural limitations that restrict concurrent execution, such as dependencies or limited available parallelism.

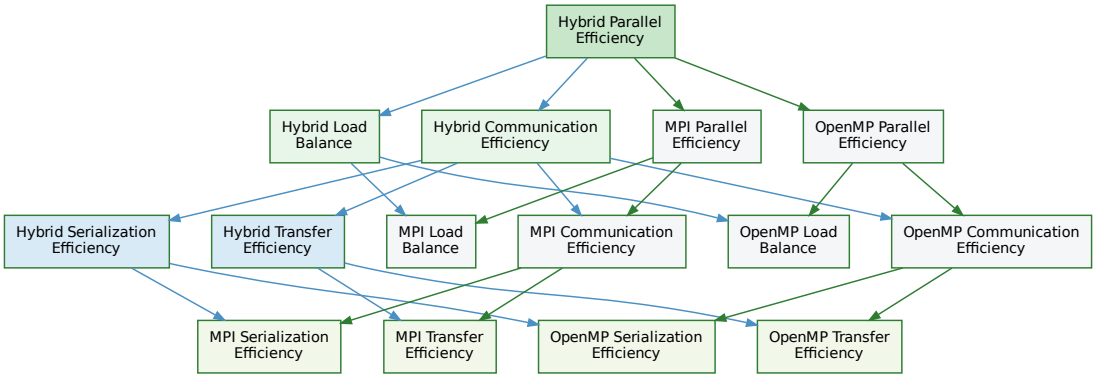
OpenMP Transfer Efficiency characterizes additional OpenMP runtime overheads associated with coordinating the parallel execution, including synchronization, scheduling, and thread-management activity.

The terminology therefore preserves the common Communication Efficiency decomposition while its runtime-specific interpretation reflects the interaction mechanisms of OpenMP.

The resulting hierarchy can be represented as:



The OpenMP factors derived through this extension describe the contribution of OpenMP to the communication-related factors observed in the hybrid execution. They remain relative contributions within the Parallel Runtime Model. The MPI+OpenMP metric hierarchy is therefore extended as follows:



A different perspective is required to characterize the OpenMP execution itself. For this purpose, BasicAnalysis provides an isolated OpenMP analysis that evaluates serial execution, load imbalance within parallel regions, and OpenMP scheduling and fork/join overhead.

5.3.2 Isolated OpenMP analysis

The OpenMP contributions derived through the Parallel Runtime Model describe how the inner runtime contributes to the performance factors observed for the complete hybrid execution. BasicAnalysis also provides an isolated OpenMP analysis that examines the efficiency of the OpenMP execution itself.

Efficient OpenMP execution requires more than distributing work among threads. The application must expose sufficient parallel work, distribute that work evenly among the threads participating in each parallel region, and manage the parallel execution without excessive runtime overhead.

The isolated OpenMP model therefore considers three main sources of inefficiency: serial execution outside

OpenMP parallel regions, load imbalance among threads inside parallel regions, and OpenMP runtime overhead associated with scheduling and fork/join activity.

Serial execution

Execution outside OpenMP parallel regions limits the amount of work that can be performed concurrently by the available threads. During these regions, only the master thread is active while the remaining OpenMP threads do not participate in the computation.

OpenMP Serial Efficiency characterizes the efficiency loss associated with this serial part of the OpenMP execution. A low value therefore indicates that a significant fraction of the available thread execution capacity is lost because execution remains outside OpenMP parallel regions.

Load imbalance in parallel regions

Entering an OpenMP parallel region does not by itself guarantee efficient use of the participating threads. The useful computational work must also be distributed evenly among them.

OpenMP Load Balance characterizes differences in useful computation among threads within each OpenMP parallel region. When some threads perform less useful work than others, they finish their assigned work earlier and remain without useful computation while other threads are still active.

A low OpenMP Load Balance therefore indicates an uneven distribution of useful work inside OpenMP parallel regions.

Scheduling and fork/join overhead

Even when sufficient parallel work is available and that work is well balanced, the OpenMP runtime introduces overhead to create, coordinate, and schedule the parallel execution.

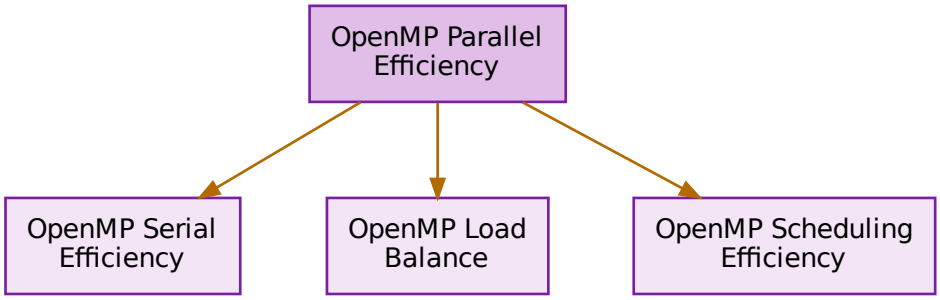
OpenMP Scheduling Efficiency characterizes runtime overhead associated with OpenMP scheduling and fork/join activity that is not attributed to serial execution or useful-work imbalance.

A low OpenMP Scheduling Efficiency therefore indicates that OpenMP runtime management represents a significant source of efficiency loss.

These three factors determine the isolated OpenMP Parallel Efficiency:

$$OMP_PE = OMP_Serial \times OMP_LB \times OMP_Sched$$

The metric hierarchy for the isolated OpenMP analysis can therefore be represented as:



5.4 Host/Device Model

Accelerator applications involve execution across two distinct but interacting domains: the **Host**, where the CPU-side execution takes place and work is prepared and submitted to the accelerator, and the **Device**, where the offloaded work is executed.

This distinction is independent of the particular parallel programming model. The Host may itself use a distributed- or shared-memory parallel runtime, while the Device execution is managed through an accelerator programming model. For example, the same Host/Device perspective can conceptually be applied to MPI+GPU, OpenMP+GPU, or OpenACC+GPU applications.

The performance behavior observed in these two domains is complementary but different. Inefficiency may originate from the Host execution or from the mechanisms used to supply work to the accelerator, while other losses may manifest on the Device through insufficient available work, data movement, workload imbalance, or computation scalability.

The **Host/Device Model** analysis separates these two perspectives in order to identify where accelerator-related inefficiencies manifest:

- **Host**, characterizing CPU-side execution and the interaction with the accelerator;
- **Device**, characterizing the execution of work on the accelerator.

BasicAnalysis currently implements this analysis for MPI+GPU applications, where MPI represents the Host-side parallel runtime and CUDA or HIP manages the accelerator execution.

The Host/Device efficiency model extends the POP methodology to heterogeneous accelerated systems. BasicAnalysis applies this model through post-mortem analysis of Paraver traces, deriving the Host- and Device-side execution information required to compute the corresponding metrics.

For compactness, the following notation is used for the Host and Device metrics:

- **Host_PE**: Host Parallel Efficiency
- **MPI_PE**: MPI Parallel Efficiency
- **Device_Offload**: Device Offload Efficiency
- **Host_GE**: Host Global Efficiency
- **Host_CompScale**: Host Computation Scalability
- **Device_GE**: Device Global Efficiency
- **Device_PE**: Device Parallel Efficiency
- **Device_LB**: Device Load Balance
- **Device_CommE**: Device Communication Efficiency
- **Device_OrchE**: Device Orchestration Efficiency
- **Device_CompScale**: Device Computation Scalability

5.4.1 Host Global Efficiency

The Host Global Efficiency characterizes the efficiency of the CPU-side execution. Performance losses at this level can arise from the parallel execution and the interaction with the accelerator, but also from changes in the efficiency of the computation performed on the Host as the execution configuration changes.

These two aspects are characterized by Host Parallel Efficiency and Host Computation Scalability, respectively. Host Global Efficiency is therefore defined as:

$$Host_GE = Host_PE \times Host_CompScale$$

Host Parallel Efficiency

From the Host perspective, parallel-efficiency losses can originate from two different components of the execution. The Host parallel runtime can introduce the usual load-distribution and communication overheads, while interaction with the accelerator can introduce offload overhead associated with launching work, transferring data, or waiting for the device.

In the current MPI+GPU implementation, the Host-side parallel runtime is MPI. Therefore, Host Parallel Efficiency is decomposed into an MPI contribution and an accelerator-offload contribution:

$$Host_PE = MPI_PE \times Device_Offload$$

MPI Parallel Efficiency characterizes the contribution associated with the MPI parallelization. When evaluating this contribution, time spent outside MPI, including Host activity associated with managing the accelerator, is considered computation from the MPI perspective.

Device Offload Efficiency characterizes the remaining Host-side loss associated with using the accelerator. This includes activity such as launching accelerator work, transferring data, and waiting for accelerator operations to complete.

Host Computation Scalability

Host Computation Scalability characterizes how the useful computation performed on the Host evolves when the execution configuration changes. It applies the computation-scalability branch of the performance model to Host-side useful computation.

As in the application-level model, Host Computation Scalability can be further decomposed into:

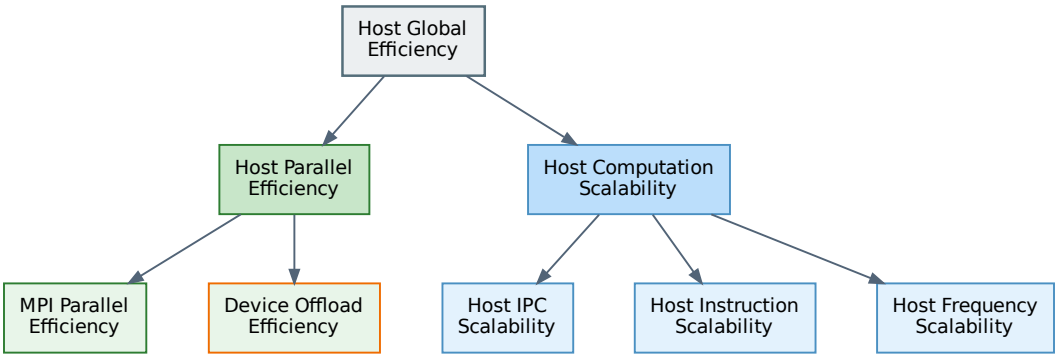
- **Host IPC Scalability**, characterizing changes in the number of instructions completed per processor cycle;
- **Host Instruction Scalability**, characterizing changes in the amount of instructions required to perform the useful Host computation;
- **Host Frequency Scalability**, characterizing changes in processor frequency.

These factors are evaluated considering only the Host-side execution and therefore characterize the scalability of the CPU-side computation independently of the Device execution.

The Host computation-scaling branch is therefore decomposed into:

$$Host_CompScale = Host_IPCScale \times Host_InstructionScale \times Host_FrequencyScale$$

Together, the Host Parallel Efficiency and Host Computation Scalability branches provide the complete decomposition of Host Global Efficiency. The resulting Host metric hierarchy for the current MPI+GPU implementation is:



5.4.2 Device Global Efficiency

Device Global Efficiency characterizes the efficiency of the accelerator-side execution. Performance losses at this level can arise from how effectively the available Device resources execute the work supplied by the Host, but also from changes in the efficiency of the computation performed on the Device as the execution configuration changes.

These two aspects are characterized by Device Parallel Efficiency and Device Computation Scalability, respectively. Device Global Efficiency is therefore defined as:

$$Device_GE = Device_PE \times Device_CompScale$$

Device Parallel Efficiency

A device may fail to perform useful computation for several different reasons. It may not have work available to execute, it may be involved in data movement, or the useful work may be distributed unevenly across the available devices. The Device hierarchy separates these effects into three performance factors: Orchestration Efficiency, Communication Efficiency, and Load Balance.

Device Load Balance

When several accelerators participate in the execution, useful work must also be distributed evenly among them.

Device Load Balance characterizes differences in useful computation across devices. A low value indicates that some devices perform less useful computation than others and therefore contribute less effectively to the parallel execution.

Device Communication Efficiency Even when work is available, device execution can be delayed by data movement. Transfers may occur between Host and Device or between accelerators, and may also involve communication mechanisms such as accelerator-aware MPI or device-communication libraries.

Device Communication Efficiency characterizes the loss associated with data movement that is not hidden by computation. Data transfers that overlap with useful computation do not contribute to this loss.

Device Orchestration Efficiency

A device can remain idle because executable work is not available when it is ready to execute. This may reflect inefficiencies in coordinating computation, communication, and Host offload, including delays in supplying work or dependencies between accelerator operations.

Device Orchestration Efficiency characterizes the loss associated with periods in which the accelerator remains inactive because neither useful computation nor data movement is being performed. A low value therefore indicates that the accelerator is not being supplied with executable work continuously.

Together, these factors determine Device Parallel Efficiency:

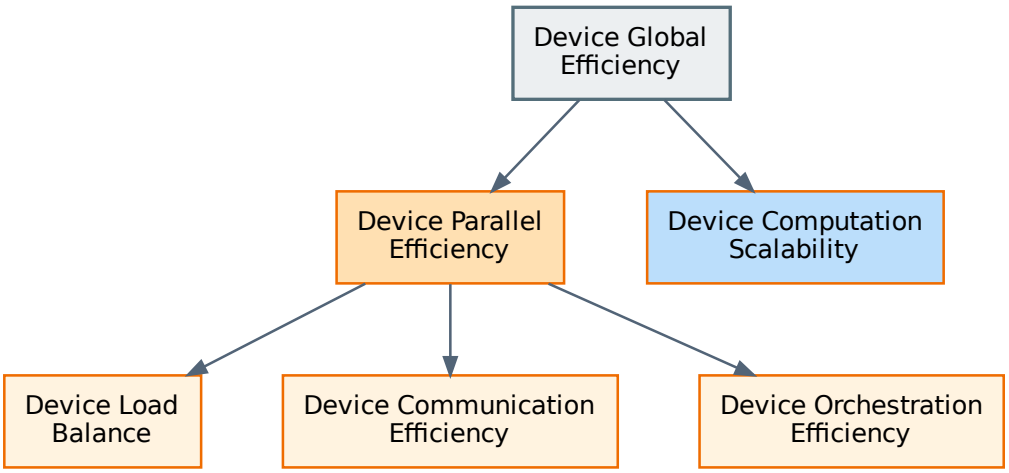
$$Device_PE = Device_LB \times Device_CommE \times Device_OrchE$$

Device Computation Scalability

Device Computation Scalability characterizes how the useful computation performed on the Device evolves when the execution configuration changes. It complements Device Parallel Efficiency by separating changes in the computation itself from losses associated with workload distribution, data movement, and the availability of work on the accelerator.

At present, BasicAnalysis provides Device Computation Scalability as the parent metric of the computation-scaling branch. A decomposition into lower-level Device computation-scaling factors is not currently defined in BasicAnalysis and is therefore not included in the metric hierarchy.

With the Device Parallel Efficiency factors described above and Device Computation Scalability completing the second branch, the Device Global Efficiency hierarchy for the current MPI+GPU implementation can be represented as:



5.4.3 Independent Host and Device efficiencies

The Host and Device hierarchies describe different execution domains. They are not successive levels of a single multiplicative hierarchy.

BasicAnalysis therefore computes two independent domain-level efficiencies:

- **Host Global Efficiency**, which characterizes CPU-side execution and the interaction with the accelerator;
- **Device Global Efficiency**, which characterizes execution on the accelerator itself.

A low Host Global Efficiency may indicate limitations in the CPU-side execution or offload path, while a low Device Global Efficiency may indicate insufficient work, data-movement overhead, imbalance, or poor computation scalability on the accelerator.

Warning

Host Global Efficiency and Device Global Efficiency must not be multiplied to obtain application Global Efficiency.

The Host/Device Model analysis should therefore be interpreted as a localization view: it helps determine **where** accelerator-related inefficiency manifests. The Parallel Runtime Model provides a different, complementary perspective by attributing efficiency losses to the participating parallel runtimes and their common performance factors.

5.5 I/O analysis

File I/O can represent an important component of application execution, but its contribution is not currently part of the hierarchical efficiency model used by BasicAnalysis. BasicAnalysis therefore provides **I/O Analysis** as a complementary analytical view.

The analysis characterizes two aspects of the observed File I/O activity:

- the relative contribution of File I/O to the measured execution activity;
- the distribution of File I/O activity across the execution units involved.

BasicAnalysis distinguishes File I/O performed through **MPI-I/O** from **POSIX and ANSI C File I/O** when the corresponding activity is available in the trace. The resulting metrics allow the user to determine whether File

I/O represents a significant component of the execution and whether the observed I/O activity is evenly distributed across the participating execution units.

The I/O metrics are interpreted as efficiency-oriented metrics, so higher values represent more favorable behavior. However, they should not be multiplied with Global Efficiency, Parallel Efficiency, or other factors of the hierarchical performance model.

When several execution configurations are analyzed, the evolution of the I/O metrics can also be compared across configurations to identify changes in the relative contribution or distribution of File I/O activity as the application scales.

The definitions, formulations, and availability conditions of the I/O metrics are provided in *Performance Metrics*.

5.6 Scaling analysis

When several execution configurations are analyzed together, BasicAnalysis examines how application performance and the efficiency factors identified by the hierarchical model evolve as the amount of parallel resources changes.

The **Scaling Analysis** adds the configuration dimension to the performance assessment. While the hierarchical analysis identifies the factors that contribute to an efficiency loss for a given execution, Scaling Analysis examines how those factors evolve across the analyzed configurations.

Scaling Analysis should not be confused with the **Computation Scalability** metric introduced earlier in this chapter. Computation Scalability is one factor of the efficiency hierarchy, characterizing how the computation itself scales relative to a reference execution. Scaling Analysis is the broader comparative analysis in which Computation Scalability, Parallel Efficiency, and their contributing factors can all be examined across configurations.

The interpretation of the scaling behavior depends on the scaling model. BasicAnalysis supports both **strong scaling**, where the problem size remains constant while the computational resources increase, and **weak scaling**, where the amount of work increases together with the computational resources.

5.6.1 Determining the scaling model

When automatic scaling detection is enabled, BasicAnalysis determines whether the analyzed executions exhibit behavior consistent with strong or weak scaling from the execution measurements.

The first execution in the ordered trace list is used as the reference. BasicAnalysis evaluates three indicators across the remaining configurations:

- the growth in useful instructions relative to the growth in the number of processes;
- the evolution of execution time relative to the reference;
- the evolution of average useful computation relative to the reference.

For each indicator, BasicAnalysis computes its average relative behavior across the non-reference executions. An indicator is considered consistent with weak scaling when its average ratio is greater than 0.9.

Weak scaling is selected when at least two of the three indicators exhibit weak-scaling behavior. Otherwise, the executions are classified as strong scaling.

This detection provides an execution-based estimate of the scaling model rather than determining the application problem size directly. The scaling model can therefore be selected explicitly when the intended scaling experiment is known. If the manually selected model differs from the automatically detected one, BasicAnalysis reports the discrepancy.

The detected scaling model, the model finally used for the analysis, and whether the selection was automatic or manual are included in the scaling information reported by BasicAnalysis.

5.6.2 User-defined workload

Strong and weak scaling are two particular cases of a more general experiment, in which the workload and the resources change independently. In strong scaling the workload is constant; in weak scaling it grows with the resources. Experiments in which the workload changes while the resources do not, for example the batch size of an inference run on the same GPU, are neither.

For these experiments, the workload of each execution can be given with `--workload` or `--ratios-workload` (see *Running BasicAnalysis*). BasicAnalysis then uses the weak-scaling formulas of the scalability metrics, replacing the resource ratio of each metric (processes, Host threads, devices, or processes with instructions) with the workload ratio

$$w(n) = \frac{W(n)}{W_{ref}}$$

where $W(n)$ is the workload of execution n. In the execution models with a single domain (MPI, MPI+OpenMP, OpenMP or pthreads, Burst mode) the same ratio is used for all metrics, because it describes the size of the whole problem, not one type of resource: if the workload doubles, the ideal is twice the useful computation time and twice the instructions. With $w(n) = 1$ the metrics are those of strong scaling, and with $w(n) = P(n)/P_{ref}$ they are those of weak scaling.

In the Host/Device model, the Host and the device are two execution domains whose work can grow differently. In an inference run, for example, the GPU work grows with the batch size, while the Host performs about the same steps and kernel launches for every batch. Each domain therefore has its own workload ratio, $w_H(n)$ for the Host metrics and $w_D(n)$ for the device metrics, which are the ratio of the application, $w(n)$, unless the user gives them (`--host-workload`, `--device-workload`). The Application Computation Scalability compares the useful time with the ideal useful time of both domains, and Speedup and Scaling Efficiency use the ratio of the application (see *Performance Metrics*). The resource ratios of weak scaling follow the same pattern: processes for the application, Host threads for the Host, and devices for the device.

The automatic detection is not used in this case: the report shows it as *Not used*.

5.6.3 Reference execution

The reference execution used during scaling detection also provides the baseline for evaluating relative scalability metrics, such as Computation Scalability and its contributing factors.

By default, BasicAnalysis orders the traces according to their parallel configuration and uses the first execution in this ordering as the reference. Users can preserve the input ordering when a different reference execution is required.

The choice of reference affects the numerical values of the relative scalability metrics and their interpretation. The selected configuration should therefore provide a meaningful baseline for evaluating how performance evolves as the application scales.

5.6.4 Interpreting scaling trends

Scaling Analysis preserves the relationships defined by the metric hierarchy, but examines them across execution configurations. The evolution of a parent metric can therefore be interpreted together with the evolution of its contributing factors.

For example, if Global Efficiency decreases as the application scales, Parallel Efficiency and Computation Scalability can be compared across the same configurations to determine which branch is responsible for the observed degradation. If Parallel Efficiency deteriorates, its Load Balance and Communication Efficiency trends can then be inspected. Similarly, a degradation in Computation Scalability can be investigated through the evolution of its IPC, Instruction, and Frequency Scalability factors.

This comparative perspective is important because the scaling behavior of a metric is not determined only by its value at a single configuration. A metric may remain relatively high while progressively degrading as resources increase, revealing an emerging scalability limitation. Conversely, a metric that is low but remains stable may represent an existing performance limitation without being the factor responsible for the observed scaling degradation.

5.7 Comparison of programming models

Executions that use different programming models can be compared, for example to evaluate a new MPI+CUDA implementation of an application against its legacy MPI or MPI+OpenMP version. Such a comparison is not a scaling study: the executions use different kinds of parallel resources and their efficiency metrics are defined by different models.

BasicAnalysis therefore describes each execution with the efficiency metrics of its own programming model and presents them side by side, aligning the metrics that represent the same performance factor.

5.7.1 Compared metrics

The comparison is organized around the parallel-efficiency hierarchy, which is defined for every MPI-based programming model:

Metric	MPI	MPI+OpenMP	MPI+GPU
Parallel Efficiency	Parallel Efficiency	Hybrid Parallel Efficiency	Host Parallel Efficiency
MPI Parallel Efficiency, Load Balance, Communication, Serialization, and Transfer Efficiency	Classic MPI metrics	MPI metrics	Host MPI metrics
OpenMP Parallel Efficiency, Load Balance, and Communication Efficiency	—	OpenMP contribution	—
Device Offload Efficiency	—	—	Host metric
Device Parallel Efficiency, Load Balance, Communication, and Orchestration Efficiency	—	—	Device metrics

In each programming model, the top-level metric is the product of the MPI contribution and the contribution of the second level, when present:

- MPI: Parallel Efficiency equals MPI Parallel Efficiency.
- MPI+OpenMP: Parallel Efficiency = MPI Parallel Efficiency × OpenMP Parallel Efficiency.
- MPI+GPU: Host Parallel Efficiency = MPI Parallel Efficiency × Device Offload Efficiency.

The top-level metric is therefore comparable across programming models. When at least one MPI+GPU execution is compared, it is labeled **Host Parallel Efficiency**, because it describes the Host side of the accelerator executions; when only CPU-based models are compared, it is labeled **Parallel Efficiency**.

For MPI executions, the MPI metrics are the classic POP metrics, based on useful computation. For hybrid executions, the MPI metrics are based on the time spent outside MPI, as in the Parallel Runtime Model. For an MPI execution, both formulations differ only by the time outside MPI that is not useful computation, such as File I/O. The classic formulation is used because it is the one from which BasicAnalysis obtains Serialization and Transfer Efficiency for MPI executions, and it gives the same values as the analysis of the MPI execution alone.

Metrics that do not apply to a programming model are left empty. The Metric Details of each value describe the metric of the corresponding execution's programming model.

5.7.2 Reference execution and Speedup

Scalability metrics, such as Computation Scalability and its IPC, Instruction, and Frequency factors, compare the computation of an execution with that of a reference execution. Across programming models, this comparison mixes different kinds of computation, for example CPU-only useful computation and CPU+GPU useful computation, and the resulting values would not be meaningful. For the same reason, scaling detection is not applied.

BasicAnalysis therefore does not compute these metrics, nor the Global Efficiency and Efficiency metrics that depend on them, across programming models.

Elapsed time and Speedup remain available. Speedup is the ratio between the elapsed time of the first execution and that of each execution, and directly answers whether an implementation is faster than the reference. IPC and processor frequency are reported for each execution as general metrics.

5.7.3 Analysis per programming model

The comparison is complemented by the analysis of the executions of each programming model as an independent set. When several executions use the same programming model, this analysis includes the usual scaling analysis between them. The comparison therefore identifies how the implementations differ, while the per-model analysis explains the behavior of each implementation in detail.

5.8 Complementary analytical views

The BasicAnalysis analytical views address different performance questions and provide complementary perspectives on the execution. They are not intended to be interpreted as independent analyses, but as different levels of evidence that can be combined according to the objective of the performance assessment.

A useful way to interpret them is:

Hierarchical Efficiency Analysis

Which performance factor contributes to the observed efficiency loss?

Parallel Runtime Model

When multiple parallel runtimes participate in the execution, which runtime contributes to that performance factor?

Runtime-Specific Analysis

Which behavior within the identified runtime helps explain the observed inefficiency?

Host/Device Model

For accelerator applications, where does the inefficiency manifest: Host, offload path, or Device?

I/O Analysis

How significant is File I/O activity, and how is that activity distributed across the execution units involved?

Scaling Analysis

How do the performance factors and their contributions evolve as the execution configuration changes?

Comparison of Programming Models

How do executions that use different programming models compare?

These perspectives can be followed progressively during a comprehensive performance assessment or selected individually when investigating a specific performance question. Their combination allows the analysis to move from identifying an efficiency loss, to attributing it to a performance factor or parallel runtime, and, when supported by the available metrics, to further characterizing where and how the inefficiency manifests. Complementary views can additionally expose execution aspects, such as File I/O activity, that are relevant to the performance assessment without forming part of the hierarchical efficiency model.

BasicAnalysis provides the efficiency metrics and interpretation guidance needed to support this assessment. The metrics identify and characterize performance losses, but they do not necessarily establish their underlying cause. Detailed trace inspection or specialized performance-analysis tools may therefore be required to validate the observations and determine why the identified behavior occurs.

CHAPTER

SIX

PERFORMANCE METRICS

BasicAnalysis computes a hierarchy of efficiency and scalability metrics from the performance information extracted from the analyzed Paraver traces.

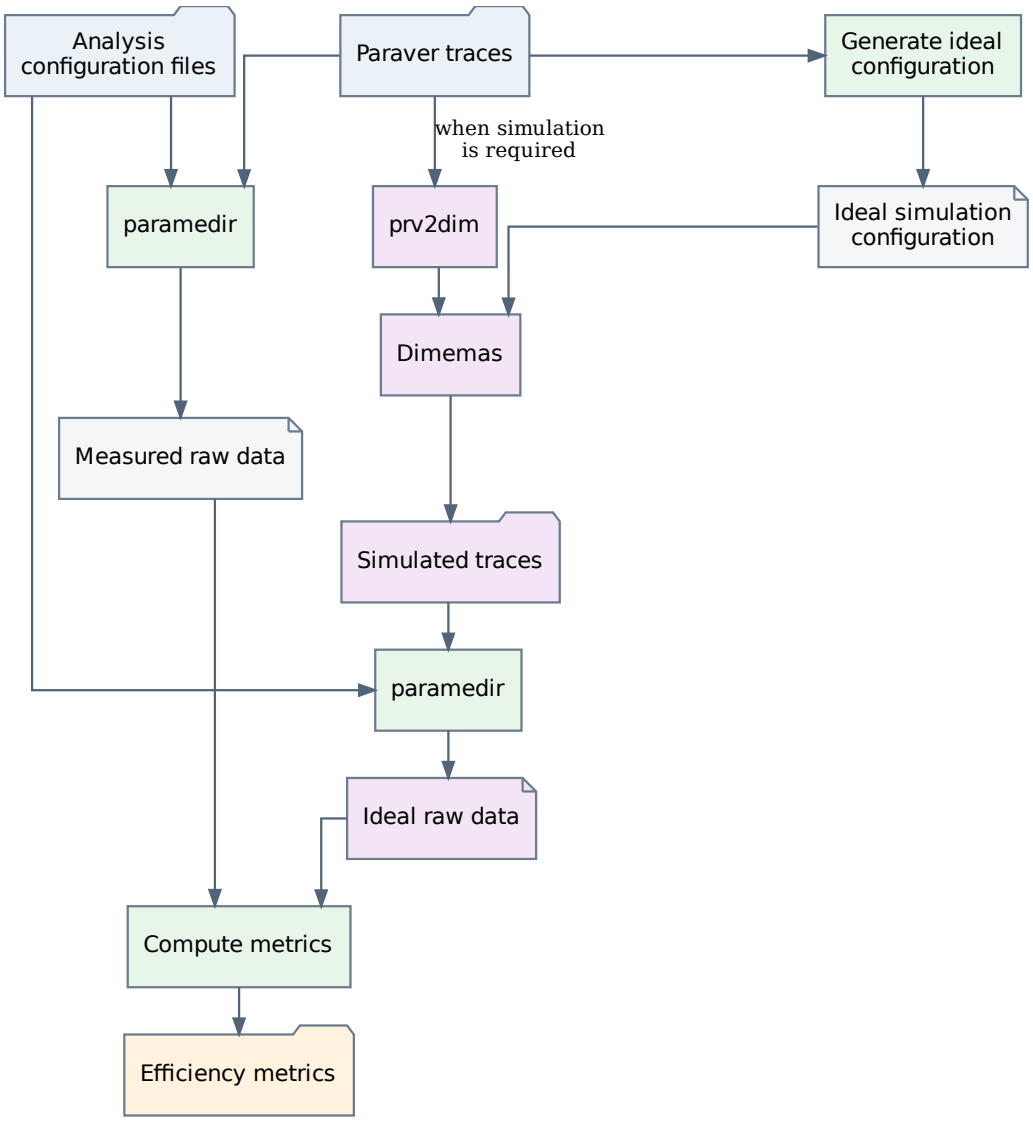
This chapter describes the metrics as they are implemented by BasicAnalysis. For each metric, the formulation corresponds to the quantities actually used by the tool. The conceptual relationships between the metrics and their role in the performance-analysis methodology are described in *Performance Analysis Methodology*.

Metric values are reported as percentages in the efficiency tables and Performance Report unless otherwise indicated.

6.1 From traces to efficiency metrics

BasicAnalysis derives its performance metrics from information extracted from Paraver traces. Depending on the metric and programming model, the required information can come directly from the measured execution or from an idealized execution generated through simulation.

The internal analysis workflow combines Paraver/paramedir data extraction and, when required, Dimemas simulation:



6.1.1 Measured execution

The original Paraver trace represents the measured application execution. BasicAnalysis uses `paramedir` together with a set of Paraver configuration files to extract the raw information required by the different metric models.

The extracted information depends on the programming model and may include execution times, useful computation, communication behavior, runtime activity, and accelerator activity.

6.1.2 Idealized execution

Some communication-efficiency decompositions require a comparison with an idealized execution.

When these metrics are required, BasicAnalysis converts the relevant trace information for Dimemas and simulates an idealized execution. `paramedir` is then used again to extract the corresponding simulated information.

Measured and simulated data are subsequently combined to compute the efficiency factors that depend on this comparison.

6.1.3 Dimemas ideal configuration

The Dimemas simulation used by BasicAnalysis is controlled by an ideal configuration that represents the assumptions of the idealized communication model.

At the MPI communication level, the ideal configuration assumes zero network latency and infinite network bandwidth. This removes the communication cost associated with message transfer through the interconnection network and provides the idealized reference execution required by the communication- efficiency decomposition.

The purpose of this configuration is not to reproduce the measured machine exactly, but to provide the reference execution required by the corresponding efficiency decomposition.

When simulation is disabled with `--skip-simulation`, metrics requiring idealized execution information cannot be computed and are reported as unavailable.

6.2 Simple Metrics

BasicAnalysis uses the simple metric model for applications whose performance can be analyzed using a single parallel execution domain. This includes, for example, MPI-only and OpenMP-only applications.

The model follows the hierarchical decomposition:

$$Global_Eff = Par_Eff \cdot CompScale$$

and:

$$Par_Eff = LB_Eff \cdot Comm_Eff$$

For MPI applications, Communication Efficiency can additionally be decomposed as:

$$Comm_Eff = Ser_Eff \cdot Transfer_Eff$$

Serialization Efficiency and Transfer Efficiency require the MPI ideal simulation described above and are therefore available only when the required simulation can be performed.

6.2.1 Notation

The following notation is used for the simple metric definitions.

Symbol	Description
T	Application execution time represented by the analyzed trace. BasicAnalysis obtains this value using the `runtime_app.cfg` Paraver configuration.
P	Number of parallel execution units considered by the simple metric model.
i	Index of a parallel execution unit, $i = 1, \ldots, P$.
$Useful_i$	Useful computation duration of parallel execution unit i.
T_{ideal}	Application execution time of the idealized trace generated by Dimemas.
$\max_{ideal}(Useful)$	Maximum useful computation duration obtained from the idealized Dimemas trace.
$Instructions$	Number of processor instructions collected during useful computation.
$Cycles$	Number of processor cycles collected during useful computation.
IPC	Instructions per cycle during useful computation.
$Frequency$	Average CPU frequency during useful computation.
(0)	Reference execution.
(n)	Execution being evaluated.

For the generic simple model, P denotes the parallel execution units used by the corresponding analysis. Its concrete meaning therefore depends on the programming model.

Hardware-counter metrics are available only when the corresponding information is present in the trace.

6.2.2 Parallel Efficiency

Parallel Efficiency measures the fraction of the application execution that corresponds, on average, to useful computation across the parallel execution units.

BasicAnalysis computes:

$$Par_Eff = \frac{\sum_{i=1}^{P} Useful_i}{P \cdot T}$$

Parallel Efficiency is also computed through the multiplicative decomposition:

$$Par_Eff = LB_Eff \cdot Comm_Eff$$

Load Balance

Load Balance measures how evenly useful computation is distributed across the parallel execution units.

BasicAnalysis computes:

$$LB_Eff = \frac{\frac{1}{P}\sum_{i=1}^{P} Useful_i}{\max(Useful_1, \ldots, Useful_P)}$$

A value close to 100% indicates that useful computation is similarly distributed among the parallel execution units. Lower values indicate increasing workload imbalance.

Communication Efficiency

Communication Efficiency measures the part of the application execution time that remains after accounting for the longest useful-computation path.

BasicAnalysis computes:

$$Comm_Eff = \frac{\max(Useful_1, \ldots, Useful_P)}{T}$$

A value close to 100% indicates that little execution time is lost outside the longest useful-computation path. Lower values indicate increasing overhead associated with communication, synchronization, or other parallel-runtime effects.

For MPI applications, BasicAnalysis can further decompose Communication Efficiency into Serialization Efficiency and Transfer Efficiency.

Serialization Efficiency

Serialization Efficiency characterizes the communication-related loss that remains after the MPI communication environment has been idealized.

BasicAnalysis computes:

$$Ser_Eff = \frac{\max_{ideal}(Useful)}{T_{ideal}}$$

where $\max_{ideal}(Useful)$ is the maximum useful-computation duration obtained from the idealized execution and T_{ideal} is its application execution time.

Lower Serialization Efficiency indicates increasing losses associated with dependencies and serialization among MPI processes.

If the resulting Serialization Efficiency exceeds 100%, the measured and simulated executions do not provide a valid conventional efficiency decomposition. BasicAnalysis reports `Warning!` instead of the numerical value.

Transfer Efficiency

Transfer Efficiency characterizes the additional communication degradation present in the measured execution relative to the idealized communication scenario.

Under the standard decomposition, BasicAnalysis derives Transfer Efficiency from Communication Efficiency and Serialization Efficiency:

$$Transfer_Eff = \frac{Comm_Eff}{Ser_Eff}$$

therefore preserving the multiplicative relationship:

$$Comm_Eff = Ser_Eff \cdot Transfer_Eff$$

If Serialization Efficiency is reported as `Warning!`, BasicAnalysis cannot use this decomposition and instead computes Transfer Efficiency directly from the measured and idealized execution times:

$$Transfer_Eff = \frac{T_{ideal}}{T}$$

where T is the measured application execution time.

A lower Transfer Efficiency indicates increasing communication-transfer cost relative to the idealized execution.

If the resulting Transfer Efficiency exceeds 100%, BasicAnalysis reports `Warning!` instead of the numerical value.

Serialization Efficiency and Transfer Efficiency are available when the required MPI ideal simulation can be performed.

The consistency and validity of simulation-derived communication submetrics are discussed in *Simulation Validity*.

Simulation Validity

Serialization Efficiency and Transfer Efficiency depend on the consistency of the measured and idealized executions.

BasicAnalysis checks the resulting simulation-derived values. When MPI Serialization Efficiency exceeds 100%, BasicAnalysis reports `Warning!` instead of interpreting the value as a conventional efficiency. In this case, MPI Transfer Efficiency is also reported as `Warning!` because the multiplicative communication-efficiency decomposition is not considered valid.

When MPI Serialization Efficiency is valid but the resulting MPI Transfer Efficiency exceeds 100%, MPI Transfer Efficiency is reported as `Warning!`.

The communication submetrics are reported as `Non-Avail` when the required simulation is unavailable, explicitly skipped, or unsupported for the detected programming model.

Computation Scalability

Computation Scalability characterizes how the useful computation evolves relative to a reference execution as the execution configuration changes.

BasicAnalysis uses the total useful-computation duration, $Useful_{tot}$, and distinguishes between strong- and weak-scaling experiments.

For strong scaling:

$$CompScale = \frac{Useful_{tot,ref}}{Useful_{tot}}$$

For weak scaling:

$$CompScale = \frac{Useful_{tot,ref}}{Useful_{tot}} \cdot \frac{P}{P_{ref}}$$

where P is the number of parallel execution units and the `ref` subscript denotes the reference execution.

When the workload of each execution is given with `--workload` or `--ratios-workload` (see *Performance Analysis Methodology*), the weak-scaling formulas of this chapter are used with the resource ratio (P/P_{ref}, or the ratio of Host threads, devices, Host threads plus devices, or processes with instructions) replaced by the workload ratio $w = W/W_{ref}$. In the models with a single domain it is the same for all metrics:

$$CompScale = \frac{Useful_{tot,ref}}{Useful_{tot}} \cdot w$$

In the Host/Device model, the Host metrics (Host Computation Scalability and Instruction Scalability) use the Host ratio w_H, and Device Computation Scalability the device ratio w_D; both are w unless given with `--host-workload` or `--device-workload`. The Application Computation Scalability compares the useful time of the application with the ideal useful time of each domain:

$$AppCompScale = \frac{U^H_{ref} \cdot w_H + U^D_{ref} \cdot w_D}{U^H + U^D}$$

where U^H and U^D are the Host and device useful times. With $w_H = w_D = w$ it is the formula above applied to the application useful time, and with ratios of 1 the strong-scaling formula.

In strong scaling, the problem size is assumed to remain constant, so changes in total useful-computation time characterize the scalability of the computation.

In weak scaling, the amount of work is expected to grow with the number of parallel resources. The factor P/P_{ref} compensates for this expected increase, allowing Computation Scalability to characterize changes in the computational behavior relative to the weak-scaling assumption.

A value close to 100% indicates that the useful computation scales as expected under the selected scaling model. Lower values indicate increasing computation-related scalability loss.

Computation Scalability is a relative metric and therefore requires multiple execution configurations. The reference execution establishes the baseline and has a Computation Scalability of 100%.

6.2.3 Computation Scalability submetrics

When the required hardware counters are available, BasicAnalysis provides additional factors that decompose changes in Computation Scalability into contributions associated with instruction throughput, the amount of executed instructions, and processor frequency.

The three factors form the computation-scalability decomposition used by BasicAnalysis:

$$CompScale = IPCScale \cdot InstructionScale \cdot FrequencyScale$$

The factors should be interpreted together when investigating a degradation in Computation Scalability.

IPC Scalability

IPC Scalability characterizes changes in instruction throughput during useful computation relative to the reference execution.

BasicAnalysis computes:

$$IPCScale = \frac{IPC}{IPC_{ref}}$$

The IPC used by BasicAnalysis is obtained from the hardware counters collected during useful computation:

$$IPC = \frac{Instructions}{Cycles}$$

where $Instructions$ is the number of instructions executed during useful computation and $Cycles$ is the corresponding number of processor cycles.

The hardware counters (instructions, cycles, and frequency) are read on the host threads only. In GPU traces, the rows of the GPU streams are excluded: Extrae may write a few counter values on them, which do not measure GPU work.

Values below 100% indicate that fewer instructions are completed per cycle than in the reference execution. Values above 100% indicate an improvement in instruction throughput relative to the reference.

Instruction Scalability

Instruction Scalability characterizes changes in the amount of processor instructions required to perform the useful computation.

For strong scaling:

$$InstructionScale = \frac{Instructions_{ref}}{Instructions}$$

For weak scaling:

$$InstructionScale = \frac{Instructions_{ref}}{Instructions} \cdot \frac{P_{ins}}{P_{ins,ref}}$$

where P_{ins} is the number of execution units contributing valid instruction-counter measurements.

The weak-scaling correction accounts for the expected increase in the amount of work as the number of parallel resources grows.

Values below 100% indicate that the execution requires more instructions than expected relative to the reference configuration. This may reflect changes in the executed code path, additional computational work, or other effects that increase the instruction count.

Frequency Scalability

Frequency Scalability characterizes changes in average processor frequency during useful computation relative to the reference execution.

BasicAnalysis computes:

$$FrequencyScale = \frac{Frequency}{Frequency_{ref}}$$

The average processor frequency used by BasicAnalysis is derived from the number of processor cycles measured during useful computation and the corresponding useful-computation time:

$$Frequency = \frac{Cycles}{Useful\ Time}$$

BasicAnalysis reports the resulting average frequency in GHz after applying the corresponding unit conversion.

Values below 100% indicate that the processor operates at a lower average frequency during useful computation than in the reference execution. Values above 100% indicate a higher average frequency relative to the reference.

Frequency changes may result from hardware power-management mechanisms, thermal constraints, processor utilization, or other system-level effects. The metric identifies the contribution of frequency variation to Computation Scalability but does not by itself determine the cause of that variation.

6.3 Hybrid MPI+X Metrics for CPU-Based Applications

For CPU-based hybrid applications, BasicAnalysis extends the application-level performance model by separating the contribution of MPI from that of the inner parallel runtime.

The hybrid application-level model follows the hierarchy:

$$Global_Eff = Hybrid_Par_Eff \cdot CompScale$$

Hybrid Parallel Efficiency is decomposed into the contributions of the outer MPI runtime and the inner parallel runtime:

$$Hybrid_Par_Eff = MPI_Par_Eff \cdot Inner_Par_Eff$$

MPI metrics characterize the outer MPI level, while derived runtime contributions quantify the multiplicative contribution of the inner parallel level to Hybrid Parallel Efficiency.

Computation Scalability and its IPC, Instruction, and Frequency scalability submetrics follow the same formulation used by the simple metric model. For CPU-based hybrid applications, the number of parallel units is the sum of the parallel units of both parallel runtime levels.

This section describes the Computation Scalability, MPI runtime metrics, the Dimemas simulations used to derive communication submetrics, and the multiplicative decomposition used to obtain the contribution of the inner parallel level.

6.3.1 Computation Scalability

For CPU-based MPI+X applications, **Computation Scalability** and its associated IPC, Instruction, and Frequency scalability submetrics are computed using the same formulation defined for the simple metric model.

The difference lies in the parallel units used by the hybrid model. For a CPU-based hybrid execution, the total number of parallel units combines the parallel units associated with both parallel runtime levels.

BasicAnalysis uses the total useful-computation duration, $Useful_{tot}$.

For strong scaling:

$$CompScale = \frac{Useful_{tot,ref}}{Useful_{tot}}$$

For weak scaling:

$$CompScale = \frac{Useful_{tot,ref}}{Useful_{tot}} \cdot \frac{P}{P_{ref}}$$

where P represents the total parallel units of the hybrid execution and the `ref` subscript denotes the reference execution.

The corresponding IPC Scalability, Instruction Scalability, and Frequency Scalability factors are computed using the same definitions described in *Computation Scalability submetrics* for the simple metric model.

These factors preserve the computation-scalability decomposition:

$$CompScale = IPCScale \cdot InstructionScale \cdot FrequencyScale$$

6.3.2 MPI runtime metrics

At the MPI level, BasicAnalysis evaluates the execution from the perspective of the MPI ranks using the time spent outside MPI calls. This allows the MPI contribution to Hybrid Parallel Efficiency to be characterized independently of the inner parallel runtime.

The MPI component follows the hierarchy:

$$MPI_Par_Eff = MPI_LB_Eff \cdot MPI_Comm_Eff$$

When the required Dimemas ideal simulation is available, MPI Communication Efficiency can additionally be decomposed as:

$$MPI_Comm_Eff = MPI_Ser_Eff \cdot MPI_Transfer_Eff$$

The MPI component of the hybrid model introduces the following notation.

Symbol	Description
T	Application execution time represented by the analyzed trace.
P	Number of MPI ranks.
i	MPI-rank index, $i = 1, \ldots, P$.
$OutsideMPI_i$	Time spent outside MPI calls by MPI rank i.
t_i	Number of host execution units associated with MPI rank i for the OutsideMPI measurement.
T_{ideal}	Application execution time of the MPI-ideal trace generated by Dimemas.
$\max_{ideal}(OutsideMPI)$	Maximum time outside MPI calls obtained from the MPI-ideal simulated trace.

The $OutsideMPI_i$ quantity is used instead of the useful-computation time employed by the simple metric model because the purpose of this decomposition is to isolate the contribution of the MPI runtime within a hybrid application.

BasicAnalysis extracts one representative OutsideMPI duration for each MPI rank. When MPI ranks contain different numbers of host execution units, the OutsideMPI contribution is weighted by the number of host execution units associated with each rank when computing the average contribution used by MPI Parallel Efficiency and MPI Load Balance.

6.3.3 MPI Parallel Efficiency

MPI Parallel Efficiency measures the efficiency associated with the MPI component of the hybrid execution.

For the general case, where MPI ranks may have different numbers of host execution units, BasicAnalysis computes:

$$MPI_Par_Eff = \frac{\sum_{i=1}^{P} OutsideMPI_i \cdot t_i}{T \cdot \sum_{i=1}^{P} t_i}$$

The weighting by t_i accounts for configurations in which different MPI ranks contain different numbers of host execution units.

When all MPI ranks have the same number of host execution units, the expression reduces to:

$$MPI_Par_Eff = \frac{\sum_{i=1}^{P} OutsideMPI_i}{P \cdot T}$$

MPI Parallel Efficiency is decomposed as:

$$MPI_Par_Eff = MPI_LB_Eff \cdot MPI_Comm_Eff$$

MPI Load Balance

MPI Load Balance measures how evenly the time outside MPI calls is distributed across the MPI ranks.

For the general case, BasicAnalysis computes:

$$MPI_LB_Eff = \frac{\frac{\sum_{i=1}^{P} OutsideMPI_i \cdot t_i}{\sum_{i=1}^{P} t_i}}{\max(OutsideMPI_1, \ldots, OutsideMPI_P)}$$

The numerator therefore represents the OutsideMPI duration averaged over the host execution units associated with the MPI ranks.

When all MPI ranks have the same number of host execution units, this reduces to:

$$MPI_LB_Eff = \frac{\frac{1}{P}\sum_{i=1}^{P} OutsideMPI_i}{\max(OutsideMPI_1, \ldots, OutsideMPI_P)}$$

A value close to 100% indicates that the time outside MPI is similarly distributed among MPI ranks. Lower values indicate increasing imbalance between ranks from the MPI-runtime perspective.

MPI Communication Efficiency

MPI Communication Efficiency characterizes the communication and synchronization overhead observed from the MPI perspective by comparing the maximum time spent outside MPI among the ranks with the application execution time.

BasicAnalysis computes:

$$MPI_Comm_Eff = \frac{\max(OutsideMPI_1, \ldots, OutsideMPI_P)}{T}$$

A value close to 100% indicates that the rank with the largest OutsideMPI duration spends most of the application execution outside MPI. Lower values indicate increasing time associated with MPI communication or synchronization on the critical MPI path.

Together with MPI Load Balance, this preserves the multiplicative relationship:

$$MPI_Par_Eff = MPI_LB_Eff \cdot MPI_Comm_Eff$$

When a supported Dimemas ideal simulation is available, MPI Communication Efficiency can be further decomposed into MPI Serialization Efficiency and MPI Transfer Efficiency.

MPI Serialization Efficiency

MPI Serialization Efficiency characterizes the communication-related loss that remains from the MPI perspective after the MPI communication environment has been idealized.

BasicAnalysis computes:

$$MPI_Ser_Eff = \frac{\max_{ideal}(OutsideMPI)}{T_{ideal}}$$

where $\max_{ideal}(OutsideMPI)$ is the maximum time spent outside MPI among the MPI ranks in the idealized execution and T_{ideal} is the application execution time of that execution.

A lower MPI Serialization Efficiency indicates increasing communication loss associated with dependencies and temporal serialization among MPI ranks that remain after idealizing the MPI communication environment.

MPI Transfer Efficiency

MPI Transfer Efficiency characterizes the additional MPI communication degradation associated with the non-ideal communication environment of the measured execution.

BasicAnalysis derives MPI Transfer Efficiency from MPI Communication Efficiency and MPI Serialization Efficiency:

$$MPI_Transfer_Eff = \frac{MPI_Comm_Eff}{MPI_Ser_Eff}$$

therefore preserving the multiplicative relationship:

$$MPI_Comm_Eff = MPI_Ser_Eff \cdot MPI_Transfer_Eff$$

A lower MPI Transfer Efficiency indicates an increasing contribution from the communication-transfer costs represented by the difference between the measured and MPI-ideal communication environments.

Unlike the corresponding simple-model expression, the hybrid formulation cannot in general be simplified to:

$$\frac{T_{ideal}}{T}$$

because MPI Communication Efficiency and MPI Serialization Efficiency are defined using the maximum OutsideMPI duration of the measured and idealized executions, respectively:

$$MPI_Transfer_Eff = \frac{\max(OutsideMPI)/T}{\max_{ideal}(OutsideMPI)/T_{ideal}}$$

and $\max(OutsideMPI)$ is not assumed to be equal to $\max_{ideal}(OutsideMPI)$.

MPI Serialization Efficiency and MPI Transfer Efficiency are available when the required MPI-ideal Dimemas simulation can be performed for the corresponding hybrid programming model.

The consistency and validity of simulation-derived communication submetrics are discussed in *Simulation Validity*.

6.3.4 Dimemas simulation

For CPU-based hybrid applications, the MPI-level simulation uses the ideal MPI communication configuration described above while disabling the inner parallel runtime.

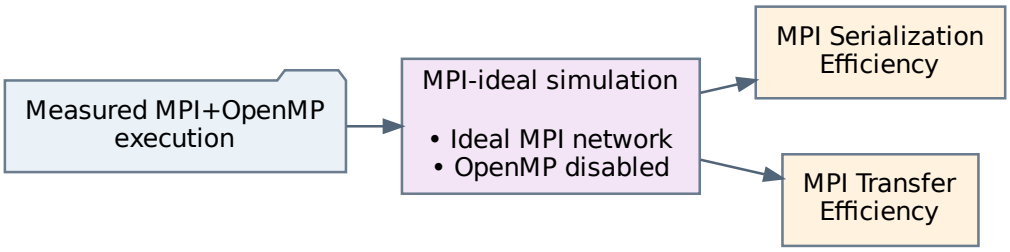
This allows the simulation-derived quantities to be interpreted from the MPI perspective, consistently with the use of `OutsideMPI` in MPI Serialization Efficiency and MPI Transfer Efficiency.

BasicAnalysis currently applies this standard simulation methodology to MPI+OpenMP applications.

MPI+OpenMP

For the standard MPI-level analysis of an MPI+OpenMP application, BasicAnalysis invokes Dimemas with OpenMP simulation disabled.

Conceptually, the experiment is:



The resulting idealized execution is used to obtain T_{ideal} and $\max_{ideal}(OutsideMPI)$, which are required to compute MPI Serialization Efficiency and MPI Transfer Efficiency.

OpenMP is disabled in this experiment because the objective is to isolate the MPI communication contribution rather than to model the combined MPI+OpenMP execution.

Optional MPI+OpenMP Hybrid Simulation

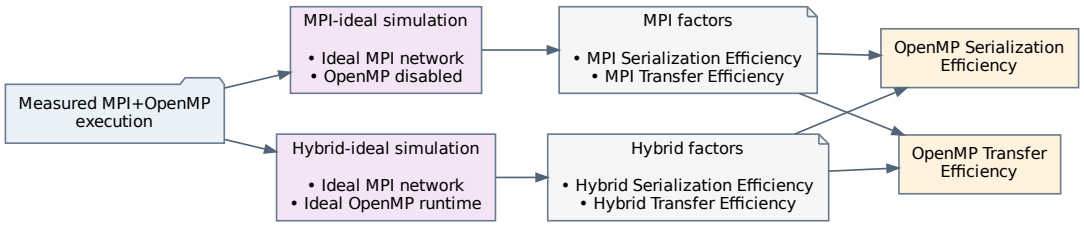
For MPI+OpenMP applications, BasicAnalysis can optionally perform a second ideal simulation when `--hyb-mpiomp` is enabled.

This experiment is different from the standard MPI-level simulation described above. The standard simulation disables OpenMP in order to isolate the MPI communication contribution. The optional hybrid simulation instead enables the Dimemas ideal-OpenMP model while retaining the idealized MPI communication environment.

BasicAnalysis therefore generates two different idealized executions:

- an **MPI-ideal execution**, used to compute MPI Serialization Efficiency and MPI Transfer Efficiency; and
- a **hybrid-ideal execution**, used to characterize the communication decomposition when both the MPI communication environment and the OpenMP execution are idealized according to the corresponding Dimemas models.

Conceptually:



From the hybrid-ideal trace, BasicAnalysis extracts the maximum useful- computation duration and the application execution time. These quantities are used to compute an intermediate hybrid Serialization Efficiency:

$$Hybrid_Ser_Eff = \frac{\max_{ideal,hybrid}(Useful)}{T_{ideal,hybrid}}$$

where $\max_{ideal,hybrid}(Useful)$ is the maximum useful-computation duration obtained from the hybrid-ideal trace and $T_{ideal,hybrid}$ is its execution time.

BasicAnalysis then derives the corresponding hybrid Transfer Efficiency from the application-level Communication Efficiency of the measured hybrid execution:

$$Hybrid_Transfer_Eff = \frac{Hybrid_Comm_Eff}{Hybrid_Ser_Eff}$$

These hybrid quantities are intermediate values used to derive the OpenMP contribution to the communication decomposition. They should not be confused with the MPI Serialization and Transfer Efficiencies obtained from the standard MPI-level simulation.

Their use in deriving the OpenMP runtime contribution is described in *OpenMP communication submetrics*.

Advanced OpenMP Dimemas Simulation Options

BasicAnalysis provides advanced options that allow users to modify how OpenMP is treated in the Dimemas simulation.

These options alter the simulated experiment. Consequently, metrics derived from the resulting simulated trace do not necessarily have the same interpretation as the standard BasicAnalysis Serialization Efficiency and Transfer Efficiency metrics described above.

`-somp, --simulation_openmp`

> Enables the simulation of OpenMP events for MPI+OpenMP applications. Without this option, OpenMP simulation is disabled in the standard MPI-level experiment.

`--ideal-omp`

> Uses the Dimemas ideal-OpenMP model during an MPI+OpenMP simulation. OpenMP runtime-event duration is ignored, while the remaining duration is associated with implicit synchronization.
>
> This option modifies how OpenMP is modeled and is meaningful when OpenMP simulation is enabled.

These options are intended for users who understand the corresponding Dimemas simulation models and want to perform customized simulation experiments.

The standard BasicAnalysis methodology uses the simulation configuration selected internally for each supported programming model. Enabling these advanced options changes the assumptions of the idealized execution and may therefore change the meaning and interpretation of the simulation-derived metrics.

6.3.5 Derived Inner-Runtime Contributions

For CPU-based hierarchical hybrid applications, BasicAnalysis derives the contribution of the inner parallel level from the application-level and MPI runtime metrics.

The model assumes the multiplicative decomposition:

$$Hybrid_Par_Eff = MPI_Par_Eff \cdot Inner_Par_Eff$$

Therefore:

$$Inner_Par_Eff = \frac{Hybrid_Par_Eff}{MPI_Par_Eff}$$

Similarly:

$$Inner_LB_Eff = \frac{Hybrid_LB_Eff}{MPI_LB_Eff}$$

and:

$$Inner_Comm_Eff = \frac{Hybrid_Comm_Eff}{MPI_Comm_Eff}$$

These quantities are derived multiplicative contributions of the inner parallel level and do not require the inner runtime to be isolated directly from the trace.

MPI+OpenMP Runtime Contribution

For the currently supported MPI+OpenMP case, the generic inner-runtime contributions defined above correspond to the OpenMP runtime contribution:

$$Inner_Par_Eff \equiv OMP_Par_Eff$$

$$Inner_LB_Eff \equiv OMP_LB_Eff$$

$$Inner_Comm_Eff \equiv OMP_Comm_Eff$$

Therefore, the application-level decomposition can be expressed as:

$$Hybrid_Par_Eff = MPI_Par_Eff \cdot OMP_Par_Eff$$

$$Hybrid_LB_Eff = MPI_LB_Eff \cdot OMP_LB_Eff$$

$$Hybrid_Comm_Eff = MPI_Comm_Eff \cdot OMP_Comm_Eff$$

The application-level metrics in these expressions are those measured for the complete MPI+OpenMP execution, while the MPI metrics represent the MPI contribution isolated through the `OutsideMPI`-based decomposition described earlier.

The resulting OpenMP factors therefore represent the multiplicative contribution required to relate the MPI-level metrics to the corresponding application-level metrics.

OpenMP communication submetrics

When the optional MPI+OpenMP hybrid simulation is enabled with `--hyb-mpiomp`, BasicAnalysis can also derive the OpenMP contribution to Serialization Efficiency and Transfer Efficiency.

The OpenMP Serialization Efficiency contribution is derived as:

$$OMP_Ser_Eff = \frac{Hybrid_Ser_Eff}{MPI_Ser_Eff}$$

and the OpenMP Transfer Efficiency contribution as:

$$OMP_Transfer_Eff = \frac{Hybrid_Transfer_Eff}{MPI_Transfer_Eff}$$

Therefore, the multiplicative relationships are:

$$Hybrid_Ser_Eff = MPI_Ser_Eff \cdot OMP_Ser_Eff$$

and:

$$Hybrid_Transfer_Eff = MPI_Transfer_Eff \cdot OMP_Transfer_Eff$$

The OpenMP Serialization and Transfer Efficiencies obtained in this way are **derived runtime contributions**. They are not independent measurements of an OpenMP-only execution.

These metrics are available only for MPI+OpenMP applications when `--hyb-mpiomp` is explicitly requested, the required Dimemas simulations complete successfully, and the corresponding simulated-trace measurements can be obtained.

Interpretation of Derived Runtime Contributions

The runtime factors above are multiplicative contributions relative to the MPI component. They are not standalone efficiencies obtained from an independent OpenMP execution.

Because they are obtained as ratios between application-level and MPI-level metrics, a derived contribution is not mathematically restricted to 100%.

For example:

$$OMP_LB_Eff = \frac{Hybrid_LB_Eff}{MPI_LB_Eff}$$

can exceed 100% when the application-level Load Balance is greater than the MPI Load Balance.

A value above 100% must therefore not be interpreted as an execution efficiency above the theoretical maximum. Instead, it indicates that the application-level metric is greater than the corresponding MPI-level metric, resulting in a multiplicative inner-runtime contribution above 100%.

6.4 Hybrid MPI+GPU Metrics

For MPI+GPU applications, BasicAnalysis extends the hybrid performance model to account for both the Host and accelerator components of the execution.

The analysis provides two complementary views:

- the **Parallel Runtime Model**, which separates the contributions of the MPI and CUDA/HIP runtimes to the hybrid application-level efficiency; and
- the **Host/Device Execution Domains**, which characterize where accelerator-related inefficiency manifests in the physical execution domains.

The construction of the GPU-related application-level and execution-domain quantities relies on the correct association of GPU stream activity with the corresponding physical devices.

6.4.1 Stream-to-device activity aggregation

GPU traces may contain several execution streams associated with the same physical accelerator. BasicAnalysis therefore does not treat individual GPU streams as independent physical execution resources when constructing the MPI+GPU metrics.

Instead, GPU stream activity is first mapped to the corresponding physical devices using the device information available in the Paraver trace metadata.

For each physical device d, BasicAnalysis collects the useful- computation intervals from all streams mapped to that device and computes their temporal union. Overlapping intervals are therefore counted only once.

The useful-computation intervals of a GPU stream are the intervals in which the stream is in the *Running* state, which on GPU streams corresponds to kernel execution. This definition does not depend on the event code used by the tracing tool to record kernel execution, which can differ between Extrae versions, and applies to both CUDA and HIP. The *Running* state of host threads corresponds to host computation and is not considered device activity.

The resulting useful-computation duration of device d is:

$$Useful_d = \left| \bigcup_{s \in Streams(d)} UsefulIntervals_s \right|$$

where $Streams(d)$ denotes the set of GPU streams mapped to physical device d.

The aggregate useful device computation is then:

$$UsefulDevice = \sum_{d=1}^{D} Useful_d$$

where D is the number of physical devices.

BasicAnalysis also obtains:

$$UsefulDevice_{max} = \max_d(Useful_d)$$

This stream-to-device mapping and interval flattening prevents concurrent activity occurring on different streams of the same physical device from being double-counted.

Device memory-transfer activity

BasicAnalysis applies the same physical-device mapping principle to memory-transfer activity.

Memory-transfer intervals are the intervals in which a GPU stream is in the *Memory transfer* state. This definition does not depend on the event code used by the tracing tool to record the transfer, which can differ between Extrae versions, and applies to both CUDA and HIP. The *Memory transfer* state can also appear on host threads, representing the host side of a synchronous copy; these intervals are not device activity and are not considered.

For each physical device, memory-transfer intervals from all associated GPU streams are collected and flattened. BasicAnalysis then removes any portion of the resulting transfer activity that overlaps useful device computation.

Only memory-transfer activity occurring outside useful computation therefore contributes additional device active time.

For device d:

$$UsefulMemTransfer_d = Useful_d + NonOverlappingTransfer_d$$

where $NonOverlappingTransfer_d$ is the duration of memory-transfer activity that does not overlap useful computation on that physical device.

BasicAnalysis then obtains:

$$UsefulMemTransfer_{max} = \max_d(UsefulMemTransfer_d)$$

This construction prevents useful computation and memory-transfer activity occurring concurrently on different streams of the same physical device from being double-counted.

6.4.2 MPI+GPU Application-Level Metrics

For MPI+GPU applications, BasicAnalysis constructs hybrid application-level metrics that account for useful computation performed in both the Host and Device execution domains.

Let:

- $UsefulHost$ denote the aggregate useful Host computation;
- $UsefulDevice$ denote the aggregate useful Device computation after stream-to-device mapping and interval flattening;
- H denote the number of Host execution units;
- D denote the number of physical devices; and
- T denote the application execution time.

The hybrid number of parallel units is therefore:

$$H + D$$

rather than the number of Host execution units plus the number of GPU streams.

Computation Scalability

For MPI+GPU applications, **Computation Scalability** characterizes how the useful computation of the complete hybrid execution evolves relative to the reference execution.

The application-level useful computation combines Host and Device useful computation:

$$Useful_{app} = UsefulHost + UsefulDevice$$

where $UsefulDevice$ is the total useful Device computation obtained after GPU streams have been mapped to their corresponding physical devices and overlapping useful-computation intervals on each device have been flattened as described in *Stream-to-device activity aggregation*.

GPU streams are therefore not treated as independent parallel execution units when constructing either the Device useful-computation quantity or the application-level parallel units.

For strong scaling, BasicAnalysis computes:

$$CompScale = \frac{Useful_{app,ref}}{Useful_{app}}$$

For weak scaling, the application-level parallel units is:

$$P_{app} = H + D$$

where H is the number of Host execution units and D is the number of physical devices.

BasicAnalysis therefore computes:

$$CompScale = \frac{Useful_{app,ref}}{Useful_{app}} \cdot \frac{H + D}{H_{ref} + D_{ref}}$$

This normalization uses the same physical parallel units as the MPI+GPU application-level model. In particular, GPU streams are not counted as independent Device resources; the Device contribution is represented by the number of physical devices.

Within the Parallel Runtime Model, Computation Scalability is reported as a single factor for the complete MPI+GPU execution. Unlike the CPU-based hybrid model, BasicAnalysis does not decompose it into IPC, Instruction, and Frequency scalability submetrics.

The Host/Device Execution Domains provide a complementary execution-domain view of Computation Scalability. In that view, Host Computation Scalability and Device Computation Scalability are computed independently using the parallel units associated with each physical execution domain: Host execution units for the Host domain and physical devices for the Device domain.

Hybrid Parallel Efficiency

BasicAnalysis computes:

$$Hybrid_Par_Eff = \frac{UsefulHost + UsefulDevice}{(H + D)T}$$

Hybrid Load Balance

Let:

$$Useful_{max} = \max(UsefulHost_{max}, UsefulDevice_{max})$$

where $UsefulHost_{max}$ is the maximum useful-computation duration among the Host execution units and $UsefulDevice_{max}$ is the maximum flattened useful-computation duration among the physical devices.

BasicAnalysis computes:

$$Hybrid_LB_Eff = \frac{\frac{UsefulHost + UsefulDevice}{H + D}}{Useful_{max}}$$

Hybrid Communication Efficiency

BasicAnalysis computes:

$$Hybrid_Comm_Eff = \frac{Useful_{max}}{T}$$

The hybrid application-level metrics preserve the multiplicative decomposition:

$$Hybrid_Par_Eff = Hybrid_LB_Eff \cdot Hybrid_Comm_Eff$$

These quantities characterize the complete MPI+GPU execution and provide the application-level reference used by the Parallel Runtime Model.

6.4.3 MPI+GPU Parallel Runtime Model

The Parallel Runtime Model separates the contribution of the outer MPI runtime from that of the accelerator runtime.

The MPI contribution is characterized using the MPI runtime metrics described for the hybrid model. For MPI+GPU applications, however, `OutsideMPI` is treated strictly as an MPI-rank quantity. BasicAnalysis uses one representative `OutsideMPI` value per MPI rank, independently of the number of GPU streams associated with that rank. Consequently, MPI Parallel Efficiency, MPI Load Balance, and MPI Communication Efficiency are computed over the MPI ranks; GPU streams do not contribute to the number of parallel units of these MPI-level metrics.

The MPI-level decomposition remains:

$$MPI_Par_Eff = MPI_LB_Eff \cdot MPI_Comm_Eff$$

The accelerator-runtime contribution is then derived as the multiplicative contribution required to relate the MPI-level metrics to the corresponding hybrid application-level metrics.

For Parallel Efficiency:

$$Hybrid_Par_Eff = MPI_Par_Eff \cdot GPU_Par_Eff$$

therefore:

$$GPU_Par_Eff = \frac{Hybrid_Par_Eff}{MPI_Par_Eff}$$

For Load Balance:

$$Hybrid_LB_Eff = MPI_LB_Eff \cdot GPU_LB_Eff$$

therefore:

$$GPU_LB_Eff = \frac{Hybrid_LB_Eff}{MPI_LB_Eff}$$

For Communication Efficiency:

$$Hybrid_Comm_Eff = MPI_Comm_Eff \cdot GPU_Comm_Eff$$

therefore:

$$GPU_Comm_Eff = \frac{Hybrid_Comm_Eff}{MPI_Comm_Eff}$$

Here `GPU` represents the CUDA or HIP runtime according to the programming model detected in the trace.

These accelerator-runtime factors are **derived multiplicative contributions**, analogous to the derived inner-runtime contributions of the CPU-based hybrid model. They are not independent measurements of a CUDA-only or HIP-only execution and are therefore not mathematically restricted to 100%.

The runtime contributions must not be confused with the metrics of the Device Execution Domain. The Parallel Runtime Model attributes efficiency losses to the MPI and CUDA/HIP runtime contributions, whereas the Host/Device analysis characterizes where accelerator-related inefficiency manifests in the physical execution domains.

6.4.4 Host and Device Execution Domains

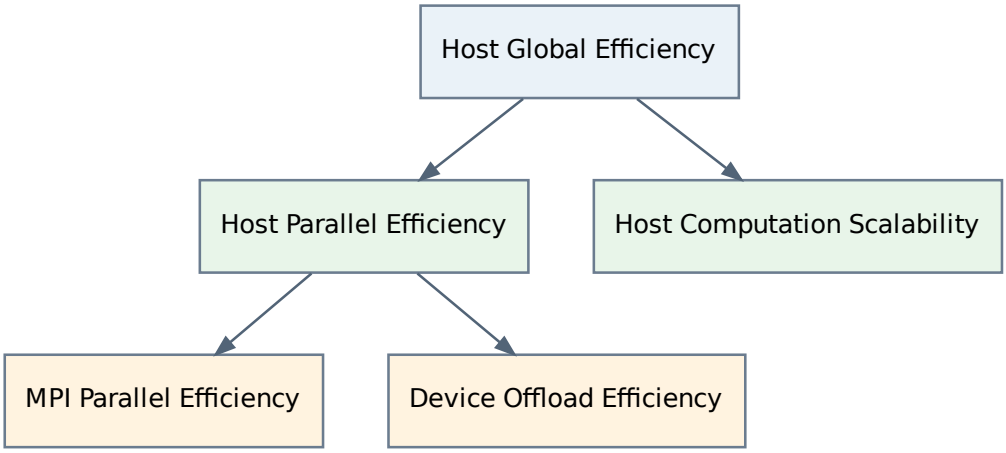
For MPI+CUDA and MPI+HIP applications, BasicAnalysis provides an additional Host/Device decomposition.

This view is complementary to the Parallel Runtime Model. The Parallel Runtime Model attributes efficiency losses to the MPI and CUDA/HIP runtime contributions, while the execution-domain analysis identifies whether accelerator-related inefficiency manifests in the Host or Device execution domain.

Host Execution Domain

The Host domain characterizes useful Host computation and the Host-side interaction with the accelerator.

Its hierarchy is:



The Host Parallel Efficiency hierarchy is:

$$Host_Par_Eff = MPI_Par_Eff \cdot DeviceOffload_Eff$$

with:

$$MPI_Par_Eff = MPI_LB_Eff \cdot MPI_Comm_Eff$$

and, when the required MPI-ideal simulation is available:

$$MPI_Comm_Eff = MPI_Ser_Eff \cdot MPI_Transfer_Eff$$

Device Offload Efficiency

Device Offload Efficiency characterizes the fraction of the time outside MPI that corresponds to useful Host computation.

Let $UsefulHost$ denote the accumulated useful Host-computation time and $OutsideMPI$ the accumulated OutsideMPI duration across the MPI ranks.

BasicAnalysis computes:

$$DeviceOffload_Eff = \frac{UsefulHost}{OutsideMPI}$$

A low Device Offload Efficiency indicates that a significant fraction of the Host execution outside MPI does not correspond to useful Host computation.

This time can include accelerator-runtime activity such as kernel launches, synchronization, memory-management operations, or other Host-side activity associated with supplying and coordinating work on the accelerator.

Device Offload Efficiency does not measure the efficiency of GPU kernel execution itself. Device-side behavior is characterized separately by the Device Execution Domain.

Host Parallel Efficiency

Host Parallel Efficiency measures the fraction of the available Host execution capacity corresponding to useful Host computation.

BasicAnalysis decomposes it as:

$$Host_Par_Eff = MPI_Par_Eff \cdot DeviceOffload_Eff$$

For a homogeneous Host configuration, this relationship can be expressed as:

$$MPI_Par_Eff = \frac{OutsideMPI}{HT}$$

and:

$$DeviceOffload_Eff = \frac{UsefulHost}{OutsideMPI}$$

therefore:

$$Host_Par_Eff = \frac{UsefulHost}{HT}$$

where H is the number of Host execution units.

Host Computation Scalability

Host Computation Scalability measures how the aggregate amount of useful Host computation evolves across execution configurations.

For strong scaling, using configuration 0 as the reference:

$$Host_CompScale_n = \frac{UsefulHost_0}{UsefulHost_n}$$

with:

$$Host_CompScale_0 = 1$$

Under ideal strong scaling, the total Host computational work remains approximately constant and Host Computation Scalability therefore remains close to 100%.

For weak scaling:

$$Host_CompScale_n = \frac{UsefulHost_0}{UsefulHost_n} \cdot \frac{H_n}{H_0}$$

where H_n/H_0 is the ratio of Host execution units between configuration n and the reference execution.

The number of Host execution units is obtained from the Host rank/threads identified in the trace.

Under ideal weak scaling, aggregate useful Host computation grows proportionally with the Host execution resources, resulting in a value close to 100%.

Host Computation Scalability Submetrics

When the required Host-side hardware-counter information is available, BasicAnalysis further decomposes Host Computation Scalability into IPC, Instruction, and Frequency scalability factors:

$$Host_CompScale = Host_IPCScale \cdot Host_InstructionScale \cdot Host_FrequencyScale$$

These factors apply the computation-scalability decomposition to the hardware-counter information associated with useful Host computation.

For weak scaling, Host Instruction Scalability is normalized using the number of Host execution units that provide valid instruction-counter measurements. Under normal trace conditions, this parallel-unit count corresponds to the Host execution units contributing useful computation and the multiplicative relationship is preserved.

If instruction-counter measurements are missing for some Host execution units, the reported child factors may not reproduce Host Computation Scalability exactly. Such a discrepancy indicates incomplete hardware-counter information rather than a different definition of the scalability model.

Host IPC Scalability

Host IPC Scalability characterizes changes in Host instruction throughput during useful Host computation relative to the reference execution.

BasicAnalysis computes:

$$Host_IPCScale = \frac{HostIPC}{HostIPC_{ref}}$$

where:

$$HostIPC = \frac{UsefulInstructions}{UsefulCycles}$$

Values below 100% indicate a reduction in Host instruction throughput relative to the reference execution.

Host Instruction Scalability

Host Instruction Scalability characterizes changes in the amount of processor instructions associated with useful Host computation.

For strong scaling:

$$Host_InstructionScale = \frac{UsefulInstructions_{ref}}{UsefulInstructions}$$

For weak scaling:

$$Host_InstructionScale = \frac{UsefulInstructions_{ref}}{UsefulInstructions} \cdot \frac{P_{ins}}{P_{ins,ref}}$$

where P_{ins} is the number of Host execution units contributing valid instruction-counter measurements.

The weak-scaling correction therefore uses the parallel units of Host execution units contributing valid instruction-counter measurements. Under normal trace conditions, this number corresponds to the number of Host execution units used by Host Computation Scalability.

Host Frequency Scalability

Host Frequency Scalability characterizes changes in the effective processor frequency during useful Host computation relative to the reference execution.

BasicAnalysis computes:

$$Host_FrequencyScale = \frac{HostFrequency}{HostFrequency_{ref}}$$

The effective Host frequency is derived from the Host-side cycle and useful-computation measurements used by the Host model.

Values below 100% indicate that useful Host computation executes at a lower effective processor frequency than in the reference configuration.

The effective Host frequency used for Host Frequency Scalability should not be confused with the Average Frequency reported by BasicAnalysis as a general performance indicator.

Host Global Efficiency

Host Global Efficiency combines Host Parallel Efficiency and Host Computation Scalability:

$$Host_Global_Eff = Host_Par_Eff \cdot Host_CompScale$$

Therefore:

$$Host_Global_Eff = MPI_Par_Eff \cdot DeviceOffload_Eff \cdot Host_CompScale$$

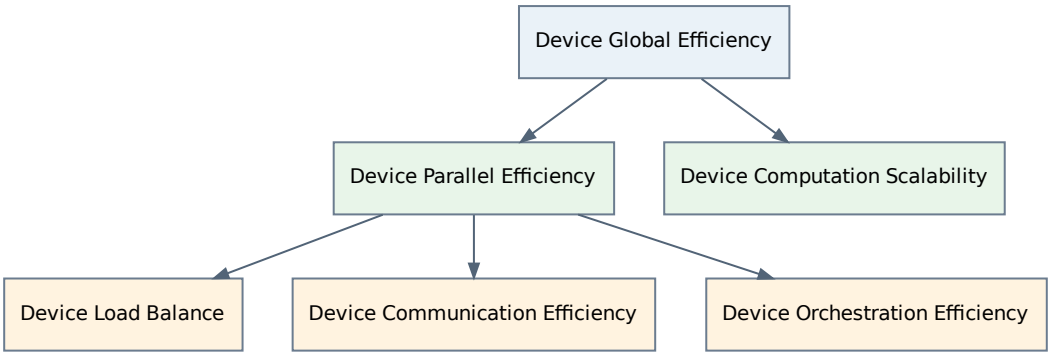
For a single-trace analysis, where Host Computation Scalability cannot be evaluated relative to another execution configuration, BasicAnalysis reports:

$$Host_Global_Eff = Host_Par_Eff$$

Device Execution Domain

The Device domain evaluates useful accelerator activity after all GPU streams have been mapped and flattened to their corresponding physical devices.

Its hierarchy is:



Device Parallel Efficiency

Device Parallel Efficiency measures the fraction of the available physical Device execution capacity corresponding to useful device computation.

BasicAnalysis computes:

$$Device_Par_Eff = \frac{UsefulDevice}{DT}$$

where D is the number of physical devices after stream-to-device mapping.

Device Load Balance

Device Load Balance measures how evenly useful computation is distributed among the physical devices:

$$Device_LB_Eff = \frac{UsefulDevice/D}{UsefulDevice_{max}}$$

A value close to 100% indicates that useful computation is similarly distributed among the physical devices. Lower values indicate increasing imbalance in useful device activity.

Device Communication Efficiency

Device Communication Efficiency characterizes the additional active device time associated with memory transfers that do not overlap useful computation.

BasicAnalysis computes:

$$Device_Comm_Eff = \frac{UsefulDevice_{max}}{UsefulMemTransfer_{max}}$$

The numerator and denominator are independently computed maxima across the physical devices and are not required to correspond to the same device.

A low Device Communication Efficiency indicates that non-overlapped memory-transfer activity contributes substantially to the maximum device active time relative to the maximum useful-computation time observed across the devices.

Device Orchestration Efficiency

Device Orchestration Efficiency measures how much of the application execution time is covered by the longest useful-computation plus non-overlapped-memory-transfer device activity.

BasicAnalysis computes:

$$Device_Orch_Eff = \frac{UsefulMemTransfer_{max}}{T}$$

Lower values indicate increasing periods during which the devices are not performing useful computation or non-overlapped memory-transfer activity.

Such periods can reflect limitations in supplying work to the accelerators, Host-side orchestration delays, synchronization, dependencies, or other effects that leave the devices without active work. The metric identifies the presence of these inactive periods but does not by itself determine their cause.

Device Parallel-Efficiency Decomposition

The three Device factors preserve the multiplicative decomposition:

$$Device_Par_Eff = Device_LB_Eff \cdot Device_Comm_Eff \cdot Device_Orch_Eff$$

Indeed:

$$Device_Par_Eff = \frac{UsefulDevice/D}{UsefulDevice_{max}} \cdot \frac{UsefulDevice_{max}}{UsefulMemTransfer_{max}} \cdot \frac{UsefulMemTransfer_{max}}{T} = \frac{UsefulDevice}{DT}$$

Device Computation Scalability

Device Computation Scalability measures how aggregate useful device computation evolves across execution configurations.

The useful-device quantity used by this metric is the aggregate physical- device activity obtained after stream-to-device mapping and interval flattening:

$$UsefulDevice_n = \sum_{d=1}^{D_n} Useful_{d,n}$$

It is therefore not the sum of the raw useful times of all GPU streams.

For strong scaling:

$$Device_CompScale_n = \frac{UsefulDevice_0}{UsefulDevice_n}$$

with:

$$Device_CompScale_0 = 1$$

Under ideal strong scaling, aggregate useful device computation remains approximately constant.

For weak scaling:

$$Device_CompScale_n = \frac{UsefulDevice_0}{UsefulDevice_n} \cdot \frac{D_n}{D_0}$$

where D_n/D_0 is the ratio of physical devices between configuration n and the reference execution.

Under ideal weak scaling, aggregate useful device computation grows proportionally with the number of physical devices, resulting in a value close to 100%.

Device Work and Throughput Scalability

A low Device Computation Scalability can mean that the devices do more work or that they do the same work more slowly. With a measure W_n of the work done by all the devices in configuration n, BasicAnalysis splits it exactly into two factors, as Host Computation Scalability is split into Instruction, IPC, and Frequency Scalability:

$$Device_CompScale_n = Device_WorkScale_n \cdot Device_ThroughputScale_n$$

$$Device_WorkScale_n = \frac{W_0 \cdot r_n}{W_n}, \qquad Device_ThroughputScale_n = \frac{W_n/UsefulDevice_n}{W_0/UsefulDevice_0}$$

where r_n is the work ratio of the scaling model: 1 in strong scaling, D_n/D_0 in weak scaling, and the device workload ratio in the user-defined workload mode (see *Running BasicAnalysis*). Device Throughput Scalability is computed as Device Computation Scalability divided by Device Work Scalability, so the product is exact.

- **Device Work Scalability** below 100% means that the devices do more work than the scaling model expects: replicated or redundant work, halo or boundary kernels that grow with the decomposition, or different kernels for different problem sizes.
- **Device Throughput Scalability** below 100% means that the same work is done more slowly: smaller kernels with lower occupancy, launch latency, a lower clock, or more pressure on the memory system.

The work measure depends on the trace source, and the same measure is used for all the traces of an analysis:

Issued instructions (nsys2prv traces with GPU metrics)

$$W_n = \sum_d \int SMIssue_d(t) \cdot f_{GPC,d}(t)\, dt$$

SM Issue is the percentage of cycles in which the SMs issue an instruction, averaged over the SMs, and f_{GPC} the GPC clock. The product is proportional to the instructions issued; the constant (number of SMs, issue width) cancels when the GPU model is the same. SM Issue is 0 outside the kernels.

GPU threads launched (all other traces)

$$W_n = \sum_{kernels} blocks \cdot threads\ per\ block$$

from the blocks per grid and threads per block recorded with each kernel (Extrae events `63000011` and `63000012` for CUDA, `63500011` and `63500012` for HIP; grid and block dimensions `9101`–`9106` in nsys2prv traces). It measures the work when each GPU thread processes a fixed amount of data.

If one trace of the analysis has no GPU metrics, all traces use the GPU threads launched. The work measure is printed under the Host/Device table.

With the issued instructions, the throughput is split once more:

$$Device_ThroughputScale_n = Device_IssueScale_n \cdot Device_FreqScale_n$$

$$Device_IssueScale_n = \frac{\overline{Issue}_n}{\overline{Issue}_0}, \quad \overline{Issue} = \frac{\sum_d \int SMIssue_d\, dt}{UsefulDevice}$$

$$Device_FreqScale_n = \frac{\overline{f}_n}{\overline{f}_0}, \quad \overline{f} = \frac{\sum_d \int SMIssue_d \cdot f_{GPC,d}\, dt}{\sum_d \int SMIssue_d\, dt}$$

$\overline{Issue}$ is the mean SM Issue during the kernels and $\overline{f}$ the mean clock while the SMs issue instructions. Without GPU metrics, these two rows are not shown.

The submetrics are shown when at least two traces are analyzed and every trace has the work measure. The limitations of the two measures are described in *Limitations*.

Kernel contributions

To identify the kernels that change the device time, the kernels are grouped by name and, for each execution n other than the reference, the share of each kernel k in the change from the ideal of the scaling model is:

$$Share_{k,n} = \frac{U_{k,n} - U_{k,0} \cdot r_n}{U_n - U_0 \cdot r_n}$$

where $U_{k,n}$ is the time of kernel k, and U_n the time of all kernels (the sum of the kernel durations, which counts kernels that overlap on different streams of a device twice). The shares add up to 100%. When the execution is faster than the ideal ($U_n < U_0 \cdot r_n$), a positive share is a contribution to the gain.

Libraries such as cuBLAS select a different kernel for each problem size; the name ends with the tile configuration (`nvjet_sm90_tst_192x192_64x4_2x1_v_bz_coopB_TNN`). These kernels are grouped by family: the name up to the first `_<n>x<m>` (`nvjet_sm90_tst`).

For each kernel, the table also gives the calls, the time, the ratio of GPU threads launched, and the ratio of the time per thread; a ratio of the time per thread above 1 means that the kernel does its work more slowly. The ten kernels with the largest change are printed on the standard output and shown in the report; all kernels are written to `device_kernels.csv` (see *Output*).

Device Global Efficiency

Device Global Efficiency combines Device Parallel Efficiency and Device Computation Scalability:

$$Device_Global_Eff = Device_Par_Eff \cdot Device_CompScale$$

Expanding the Device Parallel Efficiency decomposition:

$$\begin{aligned} Device_Global_Eff = {} & Device_LB_Eff \cdot Device_Comm_Eff \\ & \cdot Device_Orch_Eff \cdot Device_CompScale \end{aligned}$$

For a single-trace analysis, where Device Computation Scalability cannot be evaluated relative to another execution configuration, BasicAnalysis reports:

$$Device_Global_Eff = Device_Par_Eff$$

Interpretation of Host and Device Metrics

The Host and Device execution-domain metrics are complementary views of an MPI+GPU execution and should be interpreted together with the application- level and runtime-specific metrics.

The absolute values of Host Global Efficiency and Device Global Efficiency should not be directly compared to determine which execution domain "contributes more" to application performance. The two metrics characterize different parallel resources and different aspects of the heterogeneous execution.

The application-level metrics identify the overall performance loss. The Parallel Runtime Model identifies the runtime contribution associated with parallel inefficiency, while the Host and Device execution-domain metrics localize where accelerator-related inefficiency manifests.

Within the Host domain, Device Offload Efficiency helps distinguish useful Host computation from Host-side accelerator interaction, while the Host Computation Scalability factors characterize changes in the computational behavior of useful Host work.

Within the Device domain, Device Load Balance characterizes the distribution of useful computation across physical accelerators, Device Communication Efficiency characterizes additional active time associated with non-overlapped memory transfers, and Device Orchestration Efficiency characterizes periods in which the devices are not supplied with useful computation or non-overlapped transfer activity.

These views should therefore be used as complementary evidence when investigating the source of a performance loss rather than as interchangeable efficiency measurements.

6.4.5 Dimemas simulation

As for MPI+X CPU-based applications, BasicAnalysis uses Dimemas in MPI+GPU applications to obtain the idealized execution required for the MPI communication-efficiency decomposition.

The MPI-level simulation follows the same methodology: the MPI communication environment is idealized using infinite bandwidth and zero latency, while accelerator simulation is disabled. The resulting simulation-derived quantities can therefore be interpreted from the MPI perspective, consistently with the use of `OutsideMPI` in MPI Serialization Efficiency and MPI Transfer Efficiency.

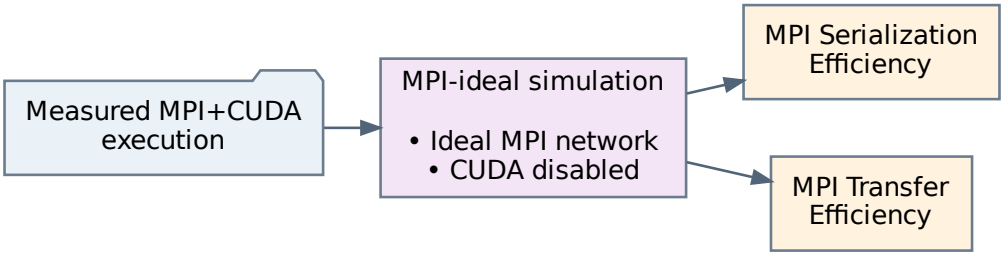
For MPI+GPU applications, these MPI communication submetrics are used wherever the MPI contribution is represented in the analysis. This includes both the MPI contribution of the Parallel Runtime Model and the MPI component of the Host execution-domain hierarchy.

BasicAnalysis currently applies this simulation methodology to MPI+CUDA applications. MPI+HIP is treated differently because the current BasicAnalysis-Dimemas workflow does not provide the corresponding simulation-derived communication submetrics.

MPI+CUDA

For the standard MPI-level analysis of an MPI+CUDA application, BasicAnalysis invokes Dimemas with CUDA simulation disabled.

Conceptually, the experiment is:



The resulting idealized execution provides T_{ideal} and $\max_{ideal}(OutsideMPI)$, which are used to compute MPI Serialization Efficiency and MPI Transfer Efficiency.

Disabling CUDA simulation allows this experiment to isolate the MPI communication contribution without introducing accelerator-simulation effects into the MPI Serialization and Transfer Efficiency submetrics.

CUDA Simulation Option

BasicAnalysis provides the `-scuda, --simulation_cuda` option to enable CUDA simulation in Dimemas.

This option changes the simulated experiment from the standard MPI-level simulation used by the BasicAnalysis performance model. In the standard methodology, CUDA simulation is disabled so that the idealized execution isolates the MPI communication contribution.

When CUDA simulation is explicitly enabled, the resulting simulated execution also incorporates the CUDA simulation model. Consequently, performance metrics derived from this simulated execution do not represent the standard BasicAnalysis MPI communication-efficiency decomposition and should not be interpreted as the MPI Serialization Efficiency and MPI Transfer Efficiency defined by the performance model described in this chapter.

This option is therefore intended for customized simulation experiments rather than for obtaining the standard BasicAnalysis performance metrics.

MPI+HIP

MPI Communication Efficiency for MPI+HIP applications is computed directly from the measured execution and therefore does not require Dimemas.

However, MPI Serialization Efficiency and MPI Transfer Efficiency require the MPI-ideal simulation described above. The current BasicAnalysis-Dimemas workflow does not provide the corresponding simulation-derived communication submetrics for HIP executions. These metrics are therefore reported as `Non-Avail` for MPI+HIP.

This limitation affects only the simulation-derived MPI communication submetrics. Metrics computed directly from the measured MPI+HIP trace, including the available Parallel Runtime Model factors and the Host and Device execution-domain metrics, remain available when their required trace information is present.

6.5 GPU Traces Generated with nsys2prv

BasicAnalysis also analyzes Paraver traces generated with *nsys2prv* from NVIDIA Nsight Systems reports of CUDA applications. These traces differ from Extrae traces in their content: they contain **no state records**, only events, and some event types reuse Extrae numbers with a different meaning. BasicAnalysis therefore detects them automatically and computes their metrics with their own Paraver configurations, without changing the analysis of Extrae traces.

A trace is identified as an *nsys2prv* trace when its `.pcf` file has no `STATES` section and contains event labels that Extrae does not use (`CUDA memcpy kernel` or `NCCL kernel`). Its trace mode is reported as `nsys2prv` with the `CUDA` programming model.

The metrics use the same formulas as for GPU applications without MPI in Extrae traces. Only the definition of useful time and communication changes, because it must be obtained from events instead of states. The assumptions of these definitions are summarized in *Limitations*.

6.5.1 Execution units

BasicAnalysis classifies the threads of the trace from the `.row` file:

- **GPU streams**: threads labelled `CUDA-D<device>.S<stream>`. The node of each stream is taken from the node of its process in the trace header, and the streams with the same node and device number belong to the same GPU. Traces merged from several nodes may place every process on the same node. Because each physical GPU is sampled by a single profiling session, a trace with more `Metrics GPU<n>` threads than GPUs identified by node and device number is missing the node information: BasicAnalysis then assumes that each process uses its own GPUs, and prints a message.
- **GPU metrics threads**: threads labelled `Metrics GPU<n>`, which contain sampled GPU hardware counters. They are not execution units; their counters are used in *GPU Hardware Counter Metrics*.
- **Host threads**: the remaining threads of the processes that have GPU streams. Processes without GPU streams do not offload work to the accelerator; they are reported in the standard output and are not analyzed.

The configuration label uses the same format as MPI+GPU traces: `units (processes x streams per process) [devices]`, where the units are the host threads and GPU streams of the processes with GPU streams, for example `5 (1x4) [1D]`.

6.5.2 Device metrics

Following the TALP device model, in which a device executes kernel computation, performs memory operations, or is idle, the device activity is classified from the events of the GPU streams:

Activity	Event	Classification
CUDA kernels	`63000006`	Useful computation
Memory copies (Host-to-Device, Device-to-Host, Device-to-Device)	`63000001` = 3, 4, 5	Memory operation (communication)
`memset`	`63000001` = 6	Memory operation (communication)
NCCL kernels	`63000007`	Communication

Useful computation and communication are then aggregated per physical device as for Extrae traces (see *Stream-to-device activity aggregation*): the intervals of all the streams of a device are flattened, and only communication that does not overlap useful computation on the same device contributes additional device active time. Device Parallel Efficiency, Load Balance, Communication Efficiency, Orchestration Efficiency, and Computation Scalability are computed with the same formulas.

These metrics provide the first level of the device-side performance assessment by identifying whether efficiency is limited by device parallel execution or by Device Computation Scalability.

The execution time T of an *nsys2prv* trace is the end time declared in its header. *nsys2prv* can write GPU metric samples after that end (each process's Nsight Systems session ends at a different time); Paraver then extends the trace to the last event, but that period contains no application activity, so BasicAnalysis uses the declared end and prints a message.

The timelines are exported in nanoseconds, so that the sum of many short kernels is not affected by rounding.

6.5.3 Host metrics

Without state records, the useful Host computation of each host thread h is estimated from the CUDA runtime and driver calls (event `63000000`) recorded on that thread:

$$UsefulHost_h = T - W_h$$

where T is the elapsed time and W_h the time of thread h inside CUDA calls, including the time waiting for the accelerator in synchronization calls.

With H host threads, BasicAnalysis computes:

$$DeviceOffload_Eff = \frac{\sum_h UsefulHost_h}{H \cdot T}$$

There is no MPI level, so Host Parallel Efficiency equals Device Offload Efficiency. Host Computation Scalability and Host Global Efficiency are computed from $UsefulHost$ as for Extrae traces. IPC, Instruction, and Frequency Scalability are not available, because *nsys2prv* traces contain no Host hardware counters.

6.5.4 Parallel Runtime Model

The Parallel Runtime Model is the single-runtime model (CUDA Parallel Runtime Model) over all execution units, as for GPU applications without MPI in Extrae traces. The useful time of a host thread is $UsefulHost_h$, and the useful time of a GPU stream is the time of its own kernels (the *Running* state of each stream in Extrae traces). Parallel Efficiency, Load Balance, and Communication Efficiency are computed with the Simple Metrics formulas. Serialization and Transfer Efficiency do not apply without MPI.

6.5.5 GPU Hardware Counter Metrics

When the Nsight Systems report includes GPU metrics (`nsys profile --gpu-metrics-devices`), each GPU has a `Metrics GPU<n>` thread with hardware counters sampled for the whole GPU, every 100 µs by default (event types `94xx`). The Device metrics characterize how effectively the accelerator is kept supplied with useful work over the execution, whereas the GPU hardware-counter metrics characterize how the GPU resources are exercised while the Graphics/Compute Engine is active. They are complementary and are **not** part of the multiplicative Host/Device model.

GPU activity represented by the counters

The sampled hardware counters describe activity at different levels of the GPU execution hierarchy. Understanding these levels is important when interpreting the metrics derived from them.

For CUDA compute workloads, `GR Active` represents activity of the GPU Graphics/Compute Engine. When this engine is active, the GPU is processing graphics or compute work. CUDA kernels execute through this engine and their thread blocks are scheduled on the Streaming Multiprocessors (SMs).

The SM-related counters then characterize progressively more detailed aspects of this compute activity:

- **SMs Active** characterizes how much of the available SM capacity has at least one warp in flight.
- **Compute Warps in Flight** characterizes how much of the available warp capacity is occupied by resident compute warps on the active SMs.
- **SM Issue** characterizes instruction-issue activity within the active SMs.

These counters therefore provide complementary information. A GPU may have its Graphics/Compute Engine active while only part of its SM capacity is active. Similarly, an SM may contain resident warps without those warps issuing instructions continuously, for example while instructions are waiting on dependencies or memory accesses.

Other sampled counters characterize particular GPU resources or operating conditions. **Tensor Active** represents activity of the Tensor Core pipelines, while the **DRAM Read** and **DRAM Write** counters characterize throughput through the device-memory system. **GPC Clock** reports the Graphics Processing Cluster clock frequency and therefore describes an operating condition of the GPU rather than the utilization of a resource.

`GR Active` should not be interpreted as meaning that every component of the GPU is idle when it is zero. In particular, GPU memory copies can be performed by independent copy engines and are not represented by `GR Active`. Consequently, the hardware-counter metrics based on `GR Active` characterize Graphics/Compute Engine activity rather than all possible activity occurring on the device.

Metric computation

BasicAnalysis derives execution-level GPU hardware-counter metrics from the sampled counters recorded in the `Metrics GPU<n>` threads.

For each sampled counter x, BasicAnalysis integrates its value over the complete execution:

$$I_x = \int x(t)\,dt$$

The integration is performed with the corresponding `cfgs/nsys2prv/hwc_<counter>.cfg` Paraver configuration using the *Integral* statistic. One value is obtained for each `Metrics GPU<n>` thread, and the integrals of all devices with sampled counters are added.

The derived metrics are computed from these accumulated quantities rather than by averaging a metric independently at each sampling point. For example, SM Coverage is computed as:

$$SM\ Coverage = \frac{I_{SMs\ Active}}{I_{GR\ Active}}$$

This formulation naturally weights the result by GPU activity. Periods with more Graphics/Compute Engine activity contribute proportionally more to the execution-level metric, while periods in which `GR Active` is zero do not contribute to metrics normalized by GPU-active time.

For sampled quantities that are not themselves nested within `GR Active`, such as DRAM bandwidth and GPC clock frequency, BasicAnalysis first weights the sampled value by `GR Active`. For example:

$$DRAM\ Read = \frac{\int DRAM^{Read}(t)\,GR^{Active}(t)\,dt}{\int GR^{Active}(t)\,dt}$$

and:

$$GPC\ Clock = \frac{\int f_{GPC}(t)\,GR^{Active}(t)\,dt}{\int GR^{Active}(t)\,dt}$$

These quantities therefore represent activity-weighted execution averages during Graphics/Compute Engine activity.

`rawdata.csv` also contains the integrals of the DRAM read and write bandwidth without the `GR Active` weighting, I_{DRAM}^{Read} and I_{DRAM}^{Write} (`hwc_dram_read_unweighted.cfg` and `hwc_dram_write_unweighted.cfg`). The metrics do not use them. Because the counters are percentages of the peak DRAM bandwidth, these integrals are proportional to the amount of data read and written: the amount in bytes is $I_{DRAM}/100$, converted from nanoseconds to seconds, multiplied by the peak bandwidth of the GPU in bytes per second, which the trace does not record. Their ratio between two executions on the same GPU model is therefore the ratio of the data moved, and dividing them by the elapsed time gives the average bandwidth over the whole execution, as a percentage of the peak.

This execution-level aggregation is particularly useful because the hardware counters are sampled periodically (every 100 µs by default), while individual GPU kernels may be shorter than the sampling period. BasicAnalysis therefore does not attribute hardware-counter samples to individual kernels. The resulting metrics characterize the complete analyzed execution and should not be interpreted as per-kernel measurements.

For multi-GPU executions, summing the counter integrals before computing the ratios also weights the aggregate metric by the corresponding activity of each device. The reported value is therefore an execution-level aggregate across the devices with sampled counters, rather than an unweighted average of independently computed per-device metrics.

GPU hardware-counter metrics in BasicAnalysis

From the integrated hardware counters, BasicAnalysis derives a set of execution-level metrics that characterize different aspects of GPU activity.

The metrics serve two different purposes:

- **SM Utilization**, **SM Coverage**, and **SM Issue Efficiency** are efficiency metrics. They characterize how effectively the SM resources are exercised while the Graphics/Compute Engine is active.
- **GPU Active**, **Achieved Occupancy**, **Tensor Core Activity**, **DRAM Read Bandwidth**, **DRAM Write Bandwidth**, and **GPC Clock** are performance indicators. They provide complementary information for interpreting the execution, but their values are not intrinsically good or bad.

The metrics are summarized below:

Metric	Formula	Meaning
SM Utilization	$I_{SM\ Issue}/I_{GR\ Active}$	Fraction of GPU-active time during which the SMs are issuing instructions.
– SM Coverage	$I_{SMs\ Active}/I_{GR\ Active}$	Fraction of the GPU's SM capacity that is active while the Graphics/Compute Engine is active.
– SM Issue Efficiency	$I_{SM\ Issue}/I_{SMs\ Active}$	Fraction of active-SM activity during which the SM sub-partitions issue instructions.
GPU Active (HW counters)	$I_{GR\ Active}/(N \cdot T)$	Average fraction of execution time during which the Graphics/Compute Engine is active across the devices with sampled hardware counters.
Achieved Occupancy (run average)	$I_{Warps\ in\ Flight}/I_{SMs\ Active}$	Average fraction of the maximum warp capacity occupied by resident compute warps while the SMs are active.
Tensor Core Activity	$I_{Tensor\ Active}/I_{GR\ Active}$	Fraction of GPU-active time during which the Tensor Core pipelines are active.
DRAM Read Bandwidth	$I_{DRAM\ Read \times GR}/I_{GR\ Active}$	Average DRAM read bandwidth during GPU-active periods, as a percentage of the peak sustained rate.
DRAM Write Bandwidth	$I_{DRAM\ Write \times GR}/I_{GR\ Active}$	Average DRAM write bandwidth during GPU-active periods, as a percentage of the peak sustained rate.
GPC Clock	$I_{Clock \times GR}/I_{GR\ Active}$	Average Graphics Processing Cluster clock frequency during GPU-active periods.

where N is the number of devices with sampled hardware counters and T is the application execution time.

The sampled counters used by these metrics are `GR Active` (`9402`), `SMs Active` (`9403`), `SM Issue` (`9404`), `Tensor Active` (`9405`), `Compute Warps in Flight` (`9412`), `DRAM Read Bandwidth` (`9418`), `DRAM Write Bandwidth` (`9419`), and `GPC Clock Frequency` (`9400`).

The DRAM bandwidth counters are already reported by NVIDIA as percentages of a hardware-specific peak sustained rate. BasicAnalysis uses these normalized counter values when computing the execution-level indicators; it does not derive the percentage from a GPU peak-bandwidth specification or provide the corresponding peak bandwidth in GB/s.

SM efficiency decomposition

The three SM efficiency metrics form a hierarchical decomposition of the instruction-issue activity observed while the GPU Graphics/Compute Engine is active.

The sampled SM counters follow the relationship:

$$SM\ Issue \leq SMs\ Active \leq GR\ Active$$

where `GR Active` represents Graphics/Compute Engine activity, `SMs Active` represents SM activity with at least one warp in flight, and `SM Issue` represents instruction-issue activity within the active SMs.

BasicAnalysis uses these quantities to distinguish two complementary aspects of SM utilization:

- **SM Coverage** characterizes how broadly the available SM capacity is active while the Graphics/Compute Engine is active:

$$SM\ Coverage = \frac{I_{SMs\ Active}}{I_{GR\ Active}}$$

- **SM Issue Efficiency** characterizes how much instruction-issue activity occurs within the active SM capacity:

$$SM\ Issue\ Efficiency = \frac{I_{SM\ Issue}}{I_{SMs\ Active}}$$

Their product gives **SM Utilization**:

$$\begin{aligned} SM\ Utilization &= SM\ Coverage \times SM\ Issue\ Efficiency \\ &= \frac{I_{SMs\ Active}}{I_{GR\ Active}} \times \frac{I_{SM\ Issue}}{I_{SMs\ Active}} \\ &= \frac{I_{SM\ Issue}}{I_{GR\ Active}} \end{aligned}$$

The decomposition separates two different reasons for low SM Utilization. Low **SM Coverage** indicates that a significant fraction of the available SM capacity is inactive while the Graphics/Compute Engine is active. Low **SM Issue Efficiency** indicates that the active SMs spend a significant fraction of their activity without issuing instructions. Both effects can occur simultaneously.

SM Utilization therefore characterizes the degree to which the SMs issue instructions while the Graphics/Compute Engine is active. It should not be interpreted as the fraction of the architecture's peak instruction throughput that is achieved.

Similarly, SM Issue Efficiency is an instruction-issue activity ratio, not an instructions-per-cycle (IPC) metric. It can play a diagnostic role similar to CPU IPC when investigating whether active execution resources are making progress, but its numerical value has a different meaning: CPU IPC counts instructions executed per cycle, whereas SM Issue Efficiency represents the fraction of active-SM activity during which instructions are issued.

Complementary performance indicators

The remaining hardware-counter metrics provide complementary information that helps interpret the SM efficiency decomposition.

Unlike the SM efficiency metrics, these performance indicators do not have a universal interpretation in which higher values are necessarily better. Their values depend on the workload, GPU architecture, and execution conditions, and should therefore be interpreted together with the efficiency metrics and with knowledge of the application.

GPU Active

`GPU Active` measures the average fraction of the execution during which the Graphics/Compute Engine is active across the devices with sampled hardware counters. It is an activity indicator rather than an efficiency metric.

It can be compared with the fraction of the execution covered by GPU kernels as a consistency check between the sampled hardware counters and the kernel-based trace information. GPU idle periods and their impact on application performance should instead be assessed using the Device metrics of the Host/Device Model.

Achieved Occupancy (run average)

`Achieved Occupancy` measures the average fraction of the maximum warp capacity occupied by resident compute warps while the SMs are active. It complements SM Issue Efficiency by indicating how many warps are available on the active SMs to help hide execution and memory latency.

Higher occupancy is not necessarily better. A kernel can perform efficiently at relatively low occupancy when the available warps provide sufficient latency hiding or when substantial instruction-level parallelism is available. Conversely, low occupancy can become relevant when there are too few resident warps to hide latency, particularly when SM Issue Efficiency is also low.

The BasicAnalysis metric is an execution-level indicator computed from the sampled hardware counters over the analyzed execution. It is therefore not the theoretical occupancy determined from a kernel launch configuration, nor should it be interpreted as the achieved occupancy of an individual kernel.

Tensor Core Activity

`Tensor Core Activity` measures the fraction of GPU-active time during which the Tensor Core pipelines are active.

Its interpretation is workload dependent. Low Tensor Core activity is expected for applications that do not use Tensor Core operations. For workloads expected to benefit from Tensor Cores, low activity may indicate that these resources are not being effectively used.

Conversely, high Tensor Core activity indicates substantial use of the Tensor Core pipelines, but does not by itself establish that Tensor Core execution is a performance bottleneck.

DRAM Read and Write Bandwidth

`DRAM Read Bandwidth` and `DRAM Write Bandwidth` characterize the use of the device DRAM memory system during GPU-active periods.

These indicators are workload dependent. High DRAM bandwidth indicates intensive use of the device-memory system, but does not by itself demonstrate that execution is memory-bandwidth bound. Similarly, low DRAM bandwidth is not necessarily inefficient: an application may obtain most of its data from the GPU caches, perform little memory traffic, or be limited by another part of the execution. DRAM bandwidth is most informative together with SM Issue Efficiency (step 4 of the *Diagnostic workflow*).

GPC Clock

`GPC Clock` reports the average Graphics Processing Cluster clock frequency during GPU-active periods. As an operating condition, its absolute value is not good or bad on its own. It is most useful when comparing executions on the same GPU model under comparable system conditions: an unexpectedly lower clock may contribute to a performance difference, but it does not explain why the frequency differs and is not sufficient evidence of power or thermal throttling.

Diagnostic workflow

The GPU hardware-counter metrics can be used as a diagnostic path to identify which aspect of GPU execution should be investigated further. The metrics describe execution-level behavior and should therefore be used to formulate performance hypotheses rather than to assign a root cause directly.

A useful starting point is **SM Utilization**, which is decomposed into **SM Coverage** and **SM Issue Efficiency**:

$$SM\ Utilization = SM\ Coverage \times SM\ Issue\ Efficiency$$

The steps below follow this decomposition and then use the complementary indicators to explain what limits instruction issue.

1. Inspect SM Coverage

If **SM Coverage is low**, a significant fraction of the available SM capacity is inactive while the Graphics/Compute Engine is active. This suggests that the workload may not expose enough parallel work to use the available SMs effectively.

Possible causes include:

- a grid containing too few thread blocks relative to the number of SMs;
- insufficient parallelism in the work decomposition; or
- kernel tail effects, where only a small number of thread blocks remain active near the end of a kernel.

Inspect the kernel grid dimensions and the number of thread blocks relative to the number of SMs. The nsys2prv trace can also be inspected in Paraver to determine whether low coverage is associated with particular kernels, short kernel executions, or kernel tail effects.

2. Inspect SM Issue Efficiency

If **SM Coverage is high but SM Issue Efficiency is low**, work is broadly distributed across the SMs, but instruction-issue activity remains low within the active SM capacity.

This indicates that the next step is to determine why the resident warps are not providing more instruction-issue activity. Possible explanations include insufficient resident warps to hide latency, memory latency, instruction or data dependencies, synchronization, or control-flow effects.

3. Inspect Achieved Occupancy

When SM Issue Efficiency is low, **Achieved Occupancy** shows whether the active SMs contain relatively few resident warps. If it is also low, too few resident warps to hide execution or memory latency is a hypothesis to investigate; the kernel launch characteristics in the trace, such as grid and block dimensions, provide additional context. Low occupancy alone does not indicate a problem (see *Complementary performance indicators*).

4. Inspect DRAM activity

If SM Issue Efficiency remains low despite sufficient resident warp occupancy, inspect **DRAM Read Bandwidth** and **DRAM Write Bandwidth**. High DRAM activity together with low SM Issue Efficiency makes device-memory behavior a relevant hypothesis for the limited instruction issue. Low or moderate DRAM bandwidth does not exclude memory-related effects: execution may instead be affected by memory latency, cache behavior, access patterns, or dependencies.

5. Inspect workload-specific resources

For workloads expected to use Tensor Cores, inspect **Tensor Core Activity** (see *Complementary performance indicators*). The trace shows which kernel intervals coincide with Tensor Core activity and whether it is concentrated in the expected parts of the execution.

6. Inspect the GPU operating conditions

When comparing executions on the same GPU model, a lower **GPC Clock** together with degraded performance may contribute to the performance change (see *Complementary performance indicators*). The clock timeline in the trace shows whether the frequency change is persistent or associated with particular execution regions.

The terms *low* and *high* in this workflow are relative rather than fixed thresholds. They should be interpreted in the context of the application, GPU architecture, and, when possible, comparable executions.

Because the nsys2prv trace contains both GPU execution events and the sampled hardware-counter timelines, the next analysis step can be performed directly in Paraver. The analyst can correlate the behavior identified by BasicAnalysis with the corresponding kernel, memory-operation, and other GPU events to determine when it occurs and which kernels or execution regions are associated with it. The hardware counters are sampled at device level and are not associated with individual kernels, so this temporal correlation should be interpreted carefully for short kernels and for kernels that overlap on different streams. The dominant kernels can then be profiled with Nsight Compute to confirm the cause.

Example: combining Device metrics and GPU hardware counters

The GPU hardware-counter metrics complement the Device metrics of the Host/Device Model. The Device metrics first identify where efficiency is being lost, while the hardware counters provide additional information about how the GPU resources are exercised during Graphics/Compute Engine activity.

The following example compares two vLLM executions on the same GPU configuration: T1 uses batch size 1 and T2 uses batch size 256.

The Device metrics reported by BasicAnalysis are shown in Fig. 6.1.

T1 shows very low Device Parallel Efficiency (12.55%). Device Load Balance is 100%, while Device Communication Efficiency is 47.16% and Device Orchestration Efficiency is only 26.62%. The main loss on the device side is therefore associated with keeping the device supplied with useful computation rather than with imbalance between devices.

In T2, Device Parallel Efficiency improves substantially to 54.56%. Device Communication Efficiency increases to 88.09% and Device Orchestration Efficiency to 61.94%, while Device Load Balance remains at 100%. The larger batch therefore keeps the device supplied with useful computation more effectively.

However, Device Global Efficiency decreases from 12.55% to 6.46%, because Device Computation Scalability falls from 100% in the reference execution T1 to 11.85% in T2: T2 spends about 8.4 times more useful device computation time than T1. The two executions use the same resources but different problem sizes, so this value compares amounts of work rather than measuring a scaling experiment. It does not by itself indicate a loss of

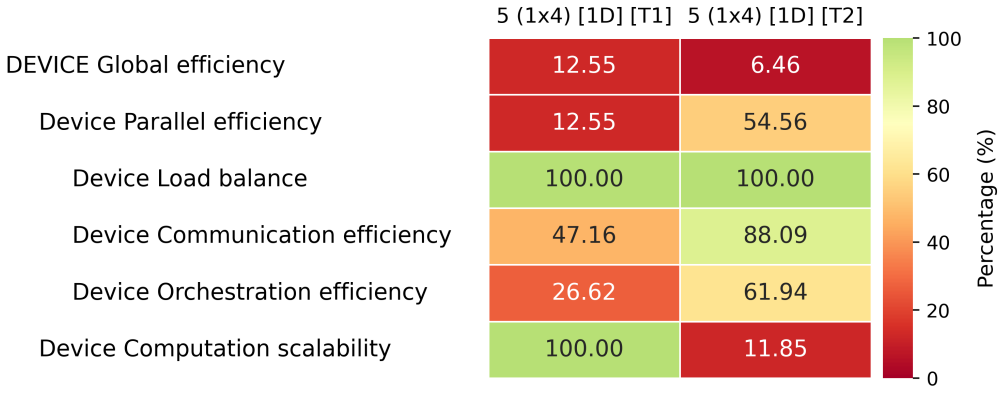


Fig. 6.1: Device metrics of the Host/Device Model.

efficiency; it shows that the device computation time grows with the batch size, and the hardware counters can be used to characterize how the GPU resources are exercised during this computation.

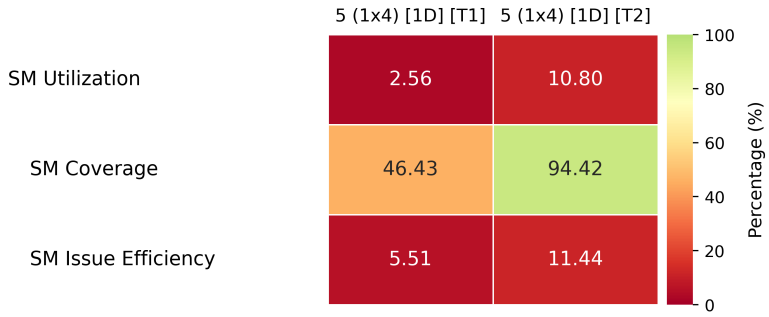


Fig. 6.2: SM Utilization and its decomposition.

Fig. 6.2 shows SM Utilization and its decomposition, and Fig. 6.3 the complementary indicators. As a consistency check, GPU Active increases from 15.10% to 55.72%, in the same direction as Device Parallel Efficiency, although the two metrics characterize different aspects of the execution.

SM Utilization increases from 2.56% to 10.80%, with both factors improving. Following the *Diagnostic workflow*:

1. SM Coverage. SM Coverage increases from 46.43% to 94.42%. In T1, about half of the SM capacity is inactive while the Graphics/Compute Engine is active, which suggests that the kernels of batch size 1 do not expose enough parallel work for the available SMs. In T2, the work is distributed over almost all the SM capacity, so SM Coverage no longer limits SM Utilization.

2. SM Issue Efficiency. SM Issue Efficiency increases from 5.51% to 11.44%, but remains low in both executions. In T2, the work is broadly distributed while the active SMs spend most of their activity without issuing instructions, so the next steps investigate why.

3. Achieved Occupancy. Achieved Occupancy increases only moderately, from 14.97% to 19.51%. Low occupancy together with low SM Issue Efficiency suggests that the active SMs may not contain enough resident warps to hide execution or memory latency. The grid and block dimensions, registers per thread, and shared memory per block of the dominant kernels should be inspected to determine what limits the number of resident warps.

4. DRAM activity. DRAM Read Bandwidth increases from 54.12% to 71.56%, and DRAM Write Bandwidth from 1.55% to 8.25%. The device-memory system is used intensively during GPU-active periods, so device-memory behavior is also a relevant hypothesis for the limited instruction issue. It does not by itself demonstrate that the execution is memory-bandwidth bound.

5. Tensor Core Activity. Tensor Core Activity increases from 2.30% to 34.28%. Batch size 1 makes little use of the Tensor Core pipelines, whereas batch size 256 uses them substantially, which is consistent with the larger matrix operations of a larger batch.

	5 (1x4) [1D] [T1]	5 (1x4) [1D] [T2]
GPU Active (HW counters)	15.10%	55.72%
Achieved Occupancy (run average)	14.97%	19.51%
Tensor Core Activity	2.30%	34.28%
DRAM Read Bandwidth	54.12%	71.56%
DRAM Write Bandwidth	1.55%	8.25%
GPC Clock (MHz)	1905	1734

Fig. 6.3: Complementary GPU hardware-counter performance indicators.

6. GPC Clock. The average GPC Clock decreases from 1905 MHz to 1734 MHz. The executions use the same GPU model, so the lower frequency may contribute to the performance of T2, but it does not by itself demonstrate power or thermal throttling.

The workflow therefore leads to two hypotheses for T2: too few resident warps to hide latency, and intensive device-memory activity. Both should be investigated for the dominant kernels.

As the last step of the *Diagnostic workflow*, the dominant kernels of T2 can be located in Paraver by correlating their execution intervals with the sampled SM, occupancy, Tensor Core, DRAM, and clock activity, and then profiled with Nsight Compute to confirm these hypotheses.

6.6 OpenMP Runtime-Specific Metrics

BasicAnalysis provides a set of OpenMP runtime-specific efficiency metrics for **OpenMP-only** and **MPI+OpenMP** applications. These metrics are based on the OpenMP efficiency model introduced by TALP and provide an isolated view of the OpenMP execution.

These metrics must not be confused with the derived OpenMP contribution of the Parallel Runtime Model described in *MPI+OpenMP Runtime Contribution*. That contribution quantifies the multiplicative contribution of OpenMP to the application-level efficiency decomposition.

In contrast, the OpenMP Runtime-Specific metrics are computed from OpenMP-specific timing information extracted from the trace. Their purpose is to characterize the OpenMP execution itself and identify which OpenMP mechanisms contribute to its inefficiency.

The analysis separates three main sources of OpenMP inefficiency:

- serial execution outside OpenMP parallel regions;
- useful-work imbalance among threads inside OpenMP parallel regions; and
- OpenMP runtime scheduling and fork/join overhead.

6.6.1 OpenMP Parallel Efficiency

OpenMP Parallel Efficiency characterizes the efficiency of the OpenMP execution after accounting for the main sources of OpenMP-specific thread-capacity loss.

BasicAnalysis decomposes it as:

$$OMP_PE = OMP_Serial \cdot OMP_LB \cdot OMP_Sched$$

The formulation uses four timing components:

Symbol	Description
T_{noOMP}	Thread-capacity time not attributed to OpenMP-specific losses. For OpenMP-only applications, it corresponds to useful computation. For MPI+OpenMP applications, MPI execution time is also included because MPI activity is not considered an OpenMP runtime loss.
T^{OMP}_{serial}	Thread-capacity time lost because OpenMP parallelism is not active.
T^{OMP}_{Imbal}	Thread-capacity time lost because useful work is unevenly distributed among OpenMP threads inside parallel regions.
T^{OMP}_{sched}	Thread-capacity time associated with OpenMP runtime scheduling and fork/join overhead.

The total OpenMP execution capacity represented by the model is therefore:

$$T^{OMP}_{capacity} = T_{noOMP} + T^{OMP}_{serial} + T^{OMP}_{Imbal} + T^{OMP}_{sched}$$

and OpenMP Parallel Efficiency is:

$$OMP_PE = \frac{T_{noOMP}}{T_{noOMP} + T^{OMP}_{serial} + T^{OMP}_{Imbal} + T^{OMP}_{sched}}$$

OpenMP Serial Efficiency

OpenMP Serial Efficiency characterizes the loss of thread capacity caused by execution outside OpenMP parallel regions.

BasicAnalysis does not interpret the duration of a serial region itself as the complete loss. Instead, it estimates the capacity that remains unused because the other OpenMP threads cannot contribute useful parallel work during that interval.

For each MPI rank in an MPI+OpenMP execution, or each OpenMP process in an OpenMP-only execution, BasicAnalysis first determines the duration for which OpenMP parallelism is not active.

Let r denote an MPI rank for an MPI+OpenMP execution or an OpenMP process for an OpenMP-only execution.

Symbol	Description
$T^{active}_{serial,r}$	Duration for which OpenMP parallelism is not active for r.
$N_{threads,r}$	Number of OpenMP threads associated with r.

The corresponding lost thread capacity is:

$$T^{OMP}_{serial,r} = T^{active}_{serial,r} \cdot (N_{threads,r} - 1)$$

and the total serial loss is:

$$T^{OMP}_{serial} = \sum_{r} T^{OMP}_{serial,r}$$

For example, if an OpenMP process with four threads spends 10 ms outside parallel regions, the serial-region duration is 10 ms, but the corresponding lost worker-thread capacity is:

$$10\,ms \cdot (4 - 1) = 30\,ms$$

This distinction is important because OpenMP Serial Efficiency characterizes the loss of available parallel capacity, rather than simply the elapsed duration of serial regions.

BasicAnalysis computes:

$$OMP_Serial = \frac{T_{noOMP}}{T_{noOMP} + T^{OMP}_{serial}}$$

A value close to 100% indicates that little thread capacity is lost because OpenMP parallelism is inactive. Lower values indicate an increasing contribution from serial execution outside OpenMP parallel regions.

OpenMP Load Balance

OpenMP Load Balance characterizes how evenly useful computation is distributed among OpenMP threads inside parallel regions.

For each OpenMP parallel region p, let:

$$U_{p,i}$$

be the useful-computation duration of thread i, and:

$$U_p^{max} = \max_i(U_{p,i})$$

the maximum useful-computation duration among the threads participating in that region.

BasicAnalysis estimates the thread-capacity loss associated with useful-work imbalance as:

$$T_{Imbal,p}^{OMP} = \sum_i \left(U_p^{max} - U_{p,i}\right)$$

The imbalance contribution is then accumulated over the OpenMP parallel regions represented in the trace:

$$T_{Imbal}^{OMP} = \sum_p T_{Imbal,p}^{OMP}$$

OpenMP Load Balance is computed as:

$$OMP_LB = \frac{T_{noOMP} + T_{serial}^{OMP}}{T_{noOMP} + T_{serial}^{OMP} + T_{Imbal}^{OMP}}$$

A value close to 100% indicates that useful work is similarly distributed among the OpenMP threads. Lower values indicate increasing useful-work imbalance.

Therefore, this metric measures **useful-work imbalance** rather than directly measuring time spent in an OpenMP waiting state.

OpenMP Scheduling Efficiency

OpenMP Scheduling Efficiency characterizes the OpenMP runtime overhead associated with scheduling and fork/join activity, after serial-execution and useful-work-imbalance losses have been represented separately in the multiplicative decomposition.

The T_{sched}^{OMP} component defined above represents this remaining OpenMP runtime-related thread-capacity loss.

BasicAnalysis computes:

$$OMP_Sched = \frac{T_{noOMP} + T_{serial}^{OMP} + T_{Imbal}^{OMP}}{T_{noOMP} + T_{serial}^{OMP} + T_{Imbal}^{OMP} + T_{sched}^{OMP}}$$

A value close to 100% indicates that little thread capacity is lost to the remaining OpenMP runtime overhead. Lower values indicate an increasing contribution from scheduling and fork/join overhead.

Multiplicative decomposition

The three OpenMP runtime-specific factors form a multiplicative decomposition of OpenMP Parallel Efficiency:

$$OMP_PE = OMP_Serial \cdot OMP_LB \cdot OMP_Sched$$

Substituting the individual definitions gives:

$$OMP_PE = \frac{T_{noOMP}}{T_{noOMP} + T_{serial}^{OMP}} \cdot \frac{T_{noOMP} + T_{serial}^{OMP}}{T_{noOMP} + T_{serial}^{OMP} + T_{Imbal}^{OMP}} \cdot \frac{T_{noOMP} + T_{serial}^{OMP} + T_{Imbal}^{OMP}}{T_{noOMP} + T_{serial}^{OMP} + T_{Imbal}^{OMP} + T_{sched}^{OMP}}$$

which reduces to:

$$OMP_PE = \frac{T_{noOMP}}{T_{noOMP} + T_{serial}^{OMP} + T_{Imbal}^{OMP} + T_{sched}^{OMP}}$$

This decomposition allows a reduction in OpenMP Parallel Efficiency to be attributed to serial execution, useful-work imbalance, or the remaining OpenMP runtime scheduling and fork/join overhead.

6.6.2 OpenMP-Only and MPI+OpenMP Applications

The OpenMP Runtime-Specific metrics are available for both **OpenMP-only** and **MPI+OpenMP** applications. The same OpenMP efficiency decomposition is used in both cases, but the quantities used to construct the model differ according to the programming model.

In particular, the difference affects T_{noOMP} and the active master-thread time used to compute the serial-loss component.

For an OpenMP-only application:

$$T_{noOMP} = T_{useful}$$

For an MPI+OpenMP application:

$$T_{noOMP} = T_{useful} + T_{MPI}$$

MPI execution is included in T_{noOMP} because the OpenMP Runtime-Specific metrics are designed to isolate inefficiencies attributable to the OpenMP runtime. Time spent executing MPI calls is therefore not classified as an OpenMP runtime loss.

MPI execution can nevertheless contribute indirectly to the OpenMP serial-loss component through unused worker-thread capacity. When the master thread executes an MPI call outside an OpenMP parallel region, the remaining OpenMP worker threads are inactive. The corresponding unused worker-thread capacity contributes to T^{OMP}_{serial}.

This treatment allows the same OpenMP efficiency decomposition to be applied to OpenMP-only and MPI+OpenMP executions while preserving the runtime-specific interpretation of the metrics.

6.6.3 Interpretation

The OpenMP Runtime-Specific metrics distinguish the main sources of OpenMP-related efficiency loss:

- A low OMP_Serial indicates a significant loss of OpenMP thread capacity while OpenMP parallelism is not active.
- A low OMP_LB indicates uneven useful-work distribution among OpenMP threads inside parallel regions.
- A low OMP_Sched indicates a significant contribution from the remaining OpenMP runtime scheduling and fork/join overhead.
- A low OMP_PE indicates that one or more of these OpenMP-specific factors are limiting the efficiency of the OpenMP execution.

For MPI+OpenMP applications, this runtime-specific view complements the OpenMP contribution reported by the Parallel Runtime Model. The two views address different performance questions:

- The **Parallel Runtime Model** quantifies the OpenMP contribution within the application-level multiplicative efficiency decomposition.
- The **OpenMP Runtime-Specific Analysis** characterizes the OpenMP execution itself and identifies which OpenMP-specific factor contributes to its inefficiency.

Using both views is particularly useful for hybrid applications because the runtime-specific analysis can expose OpenMP behavior that may not be apparent from the application-level runtime decomposition alone.

6.7 I/O Metrics

BasicAnalysis provides complementary metrics for characterizing the File I/O observed in the trace: the POSIX, ANSI C, and MPI-IO calls recorded by Extrae.

These metrics are reported independently from the hierarchical performance-efficiency model. They do not contribute to Global Efficiency, Parallel Efficiency, Computation Scalability, or the runtime-specific efficiency decompositions.

The I/O metrics start from **time**, because BasicAnalysis observes the cost paid by the application on a specific I/O system. A low I/O efficiency identifies I/O that deserves investigation; it does not by itself show that the I/O is

inefficient, because the application may need to move a large amount of data. Quantities in other units, such as operation counts, are diagnostics that help explain the observed time.

6.7.1 Classification of the I/O calls

Every traced I/O call is classified by its meaning, with the same rule for every API:

- **Data Access**: calls that move or commit application data.
- **File Management**: calls that manage the file without moving application data (open, close, positioning, and control).

API	Data Access	File Management
POSIX	`read`, `write`, `pread`, `pwrite`, `readv`, `writev`, `preadv`, `pwritev`	`open`, `close`, `lseek`, `ioctl`
ANSI C	`fread`, `fwrite`	`fopen`, `fclose`, `fseek`
MPI-IO	`MPI_File_read*`, `MPI_File_write*` (all variants)	`MPI_File_open`, `MPI_File_close`

Within Data Access, the **read** calls (`read`, `fread`, `pread`, `readv`, `preadv`, `MPI_File_read*`) and the **write** calls are also reported separately. Within File Management, the calls are grouped as **open**, **close**, **seek** (`lseek`, `fseek`), and **control** (`ioctl`).

A call that the classification does not know (for example, a call instrumented by a future Extrae version) is counted as File Management, and BasicAnalysis prints a warning naming it.

By default, Extrae does not trace the POSIX calls performed inside MPI-IO calls. The MPI-IO and POSIX times are therefore not counted twice.

6.7.2 Notation

Symbol	Description
U	Total useful computation time of the Host execution units, the units that perform File I/O. In GPU applications, the kernel time of the GPU streams is excluded: the I/O time is compared with the useful time of the same units.
DA	Total time in Data Access calls (POSIX, ANSI C, and MPI-IO).
FM	Total time in File Management calls (POSIX, ANSI C, and MPI-IO).
IO_i	Time in I/O calls of execution unit i.
P	Number of Host execution units. For hybrid applications, accelerator execution contexts are excluded because File I/O is performed by the Host.
T	Application execution time.

6.7.3 I/O Efficiency

I/O Efficiency measures the cost of File I/O relative to useful computation: the fraction of the useful computation plus File I/O time of the Host units that is useful computation:

$$IO_Eff = \frac{U}{U + DA + FM}$$

It is the product of two factors:

$$IO_Eff = DA_Eff \times FM_Eff$$

Data Access Efficiency measures the cost of reading and writing application data: the fraction of the useful computation plus Data Access time that is not spent in Data Access:

$$DA_Eff = \frac{U}{U + DA}$$

File Management Efficiency measures the additional cost of managing the files, which moves no application data: the fraction of the useful computation plus I/O time that is not spent in File Management calls:

$$FM_Eff = \frac{U + DA}{U + DA + FM}$$

The lower factor identifies the I/O component to investigate. A low Data Access Efficiency means that read and write calls take significant time relative to useful computation; a low File Management Efficiency means that open, close, seek, or control calls add significant time after the Data Access time is accounted for.

File Management Efficiency includes DA in its denominator: with the same File Management time, it is higher when the Data Access time is higher. When comparing executions, a change of File Management Efficiency should therefore be read together with the File Management time and its contributions.

I/O Efficiency compares I/O time with useful time, not with the execution time. The fraction of the execution time spent in I/O is given by the I/O time fractions below.

The I/O efficiencies are classified with the same thresholds as the other efficiencies: Critical below 60%, Attention from 60% to 85%, and Good from 85%. An I/O Efficiency of 85%, for example, means that I/O takes 15% of the useful plus I/O time. For applications where such an I/O cost already matters, the analyst can use stricter thresholds for the I/O efficiencies. The thresholds are defined per metric in `metrics/definitions.yaml`; for example, to classify I/O Efficiency as Critical below 90% and Good from 95%, add to its entry (`io_eff`):

```
thresholds:
  critical: 90.0
  attention: 95.0
```

The cell colors and the efficiency scale of the report follow the thresholds of each metric, so the colors of the other metrics do not change.

The metrics are reported as `Non-Avail` when no File I/O is detected.

6.7.4 I/O Critical-Path Efficiency and I/O Load Balance

I/O Efficiency uses the average I/O time per unit. When the units synchronize after their I/O (for example, in read and write phases separated by barriers or collectives), the time paid is that of the slowest unit, and the other units wait for it. **I/O Critical-Path Efficiency** measures the cost of File I/O on the critical path of the execution, when each kind of I/O call costs the time of the slowest unit:

$$IO_CP_Eff = \frac{U_{avg}}{U_{avg} + IO_{cp}}, \qquad IO_{cp} = \sum_{op} \max_{i} IO_{op,i}$$

where U_{avg} is the average useful time per unit and IO_{cp} adds, for each kind of I/O call (read, write, open, close, seek, control), the time of the unit that spends the most time in it. It is the product of I/O Efficiency and **I/O Load Balance**:

$$IO_CP_Eff = IO_Eff \times IO_LB, \qquad IO_LB = \frac{U_{avg} + IO_{avg}}{U_{avg} + IO_{cp}}$$

I/O Load Balance measures the cost of the uneven distribution of the I/O time across the units: the useful plus I/O time of the average unit divided by that of the critical path. A low value indicates that some units spend much more time than others in some kind of I/O call (for example, one rank writes the output while the others wait). It complements the per-operation load balances of the diagnostics, which show which kind of call is unevenly distributed.

Using the maximum of each kind of call assumes that the units synchronize after each kind (separate phases). If the I/O of a unit overlaps the computation of the others, its imbalance costs less than estimated, and I/O Load Balance underestimates the efficiency. The waiting caused by the imbalance already appears in the hierarchical model (as Load Balance or Communication Efficiency); I/O Load Balance attributes part of that loss to I/O, and is not an additional loss.

In a VTKHDF execution with 112 ranks, I/O Load Balance drops from 88.49% to 76.19% when the chunk size changes from 100k to 2M, while I/O Efficiency hardly changes (68.26% and 65.19%): the writes become unbalanced (Write Load Balance 61.48% and 21.39%). The model predicts 0.38 s more waiting per rank, and the measured MPI time per rank grows by 0.44 s.

All the I/O efficiencies use the thresholds given above.

6.7.5 I/O diagnostics

The I/O diagnostics explain where the I/O time is spent. They are not efficiencies; their values must be interpreted in context.

I/O time fractions

$$IOFraction_{avg} = \frac{DA + FM}{PT}, \qquad IOFraction_{max} = \frac{\max_i IO_i}{T}$$

$IOFraction_{avg}$ is the fraction of the execution time spent in I/O on average, and $IOFraction_{max}$ the I/O fraction of the execution unit with the most I/O time. Averages hide I/O concentrated in a few units: when one rank writes while the others wait, I/O Efficiency can be close to 100% while $IOFraction_{max}$ is high. The waiting time of the other units appears in the main efficiency model (for example, as MPI time).

I/O participation is the percentage of the P execution units that perform I/O. A low participation can be an intentional strategy (I/O delegated to a few ranks); together with a high $IOFraction_{max}$, it identifies I/O that may be on the critical path.

Contributions are shares of time:

- MPI-IO and POSIX/ANSI C contributions to the I/O time;
- Read and Write contributions to the Data Access time;
- Open, Close, Seek, and Control contributions to the File Management time.

They indicate which part of the I/O should be investigated first; a high contribution does not mean that those calls are inefficient.

Load balance is computed among the execution units that perform the corresponding calls (participants):

$$LB = \frac{\overline{X}_{participants}}{\max_i X_i}$$

for X = Data Access, read, write, File Management, and open time per unit. The Read and Write Load Balance are more informative than the Data Access Load Balance when the application reads and writes in different phases: an imbalance in the write phase costs time on its own, and adding the read and write times per unit dilutes it. Units that perform no I/O do not lower the load balance; they are reflected in the participation.

Per-operation time, count, and mean duration for the read, write, open, close, seek, and control calls. The mean duration (time divided by count) distinguishes the two explanations of a high operation time: many operations (same mean duration, higher count) or more expensive operations (same count, higher mean duration). The latter points to the interaction with the I/O system rather than to the application I/O pattern.

Requests of the read and write calls of every API: the POSIX and ANSI C read and write families (`read`, `fread`, `pread`, `readv`, `preadv`, and the corresponding write calls) and the MPI-IO read and write calls (`MPI_File_read*`, `MPI_File_write*`):

- the **volume** read and written (bytes);
- the **mean request size**, the volume divided by the number of requests;
- the **request-size coverage**, the percentage of the read or write calls that have a non-zero request size in the trace and are therefore included in the volume and the distribution;
- the **request-size distribution**: for each power-of-two size interval, the share of the requests, of the time, and of the bytes of the read or write calls.

The intervals are $[2^{j-1}, 2^j)$ bytes (`256-512 B` holds the requests from 256 to 511 bytes), from 1 byte to 2 GiB or more. They are a scale, not a classification of the requests as small or large: they are the same for every execution, so executions can be compared directly, and they resolve small requests even when large requests exist. The mean alone is not enough, because different distributions can have the same mean.

The size of a request is the one recorded by Extrae for the call: the bytes requested by a POSIX or ANSI C call, and the count times the size of the datatype of an MPI-IO call (also the requested bytes, which may be more than

the bytes read at the end of a file). An MPI-IO split collective (`*_begin` and `*_end`) is one request, with the size of the `*_begin` call and the time of both calls. The requests of every API are added in the distribution, as in the classification of the calls; `io_request_sizes.csv` gives them by API. Extrae does not trace, by default, the POSIX calls performed inside the MPI-IO calls, so the MPI-IO requests are the requests of the application, not those that the MPI-IO library issues to the file system.

The distribution shows which request sizes cost the time. In a cfdem execution, the writes of 256–511 bytes are 1.3% of the writes but 84% of the write time; in a VTKHDF execution with chunks of 2M, the writes of 8–16 MiB are 3.1% of the writes and 60% of the write time. An interval with a small share of the requests and a large share of the time holds the expensive requests.

BasicAnalysis obtains the requests from a temporary trace that keeps only the I/O call and size events of POSIX/ANSI C and MPI-IO (written by Paraver with `cfgs/io-requests-filter.xml` and removed after the analysis). The histograms `cfgs/io-{read,write}-size-pow2-{count,time,bytes}.cfg` show the same intervals in Paraver for the POSIX and ANSI C calls.

Files accessed by the POSIX and ANSI C calls:

- the **files accessed**, the files with at least one call in the trace;
- the **processes per file, maximum**, the largest number of processes that call on one file;
- the **volume moved by the main process of each file**: for each file, the bytes read and written by the process that moves the most (its *main process*), added over the files and divided by the bytes read and written in all the files. It is 100% when each file is read and written by one process (one file per process), and $100/N$ % when every file is shared evenly by N processes;
- the **file-name coverage of the volume**, the percentage of the bytes read and written by the POSIX and ANSI C calls that are attributed to a file.

The number of processes per file alone does not tell whether the processes share the data of the files: in a VTKHDF execution of 112 processes, every process opens every file and reads its metadata, so all 113 files have 112 processes, but each part file is written and read almost entirely by one process (97% of its bytes), and the volume moved by the main processes is 96%. Only the common file is shared (its main process moves 1.4% of its bytes).

Extrae records the descriptor of every POSIX and ANSI C call (for the ANSI C calls, the descriptor of the stream), and the file name only in the `open` and `fopen` calls, together with the returned descriptor. A call is attributed to the file of the last `open` or `fopen` of the same process that returned its descriptor, until the descriptor is closed. The threads of a process share its descriptors. Calls on descriptors without an `open` or `fopen` in the trace are collected in one group, *descriptors without a traced open*: the standard input, output, and error, files opened before the tracing starts, and files opened with calls that Extrae does not instrument (such as `openat`, `creat`, or `dup`). MPI-IO records neither the file name nor the file handle, so the MPI-IO calls are not attributed to files.

For each file, BasicAnalysis computes the opens, the I/O time (the time of all the POSIX and ANSI C calls on the file), the processes that call on it, and the requests, bytes, time, and processes of the reads and writes. The files are read from the same temporary trace as the requests, which also keeps the descriptor and file name events. The bytes, requests, and time of the reads and writes of all the files, including the descriptors without a traced open, are those of the request-size distribution of the POSIX and ANSI C calls.

In a cfdem execution, 80% of the POSIX I/O time is spent in one probe file of 61 KiB, written by one process in 223 writes of about 280 bytes (12 ms per write).

Access pattern of the POSIX and ANSI C reads and writes to regular files. Each request is compared with the previous request of the same kind (read or write) on the same descriptor of the same process, through the gap between its offset and the end of the previous request:

- **first**: the first read (write) on the descriptor after it is opened; there is no previous request to compare with;
- **consecutive**: starts exactly where the previous read (write) ended (gap 0): the file is read (written) in order, without gaps;
- **strided**: leaves the same gap after the previous request as that request left after the one before it: a regular pattern, such as one record every N bytes;
- **forward**: jumps ahead, skipping part of the file, by a gap different from the previous one;

- **backward**: jumps back to an earlier offset, by a gap different from the previous one; it includes reading (writing) the same region again;
- **unknown offset**: the offset cannot be rebuilt from the trace (see below).

Consecutive, positive strided, and forward requests are *sequential* in the sense used by Darshan. For each class, BasicAnalysis gives the share of the requests, of the bytes, and of the time of the reads and of the writes: the classes that hold most requests may move few bytes. In a VTKHDF execution of 112 processes, 78% of the read requests jump forward or backward, but 93% of the bytes read are consecutive: the jumps are small reads, consistent with HDF5 metadata lookups, and they take 74% of the read time.

Extrae does not record the offset of the reads and writes. BasicAnalysis rebuilds it from the position of each descriptor: an `open` or `fopen` sets it to 0, a seek (`lseek`, `fseek`) to the offset recorded by Extrae, and each read or write starts at the position and advances it by its request size. The offset is unknown for the `pread`/`pwrite` family (the offset argument is not recorded), for descriptors without a traced `open` until their first seek, and after a seek without a recorded offset. The seek offset is recorded only by recent Extrae versions; with older versions, the position is lost at the first seek of each descriptor. The **offset coverage** is the share of the requests to regular files with a known offset. Requests on sockets, pipes, and terminals (the descriptor type recorded by Extrae) have no position and are not classified. MPI-IO records no offset, so its requests are not classified.

Two more diagnostics come from the same positions:

- the **volume re-accessed**: the share of the bytes read (written) with a known offset that the same process had already read (written) in the same file. In the VTKHDF execution, 2.5% of the bytes read are re-read (metadata read again after each open of a file);
- the **redundant seeks**: the share of the seeks with an offset that go to the position the descriptor already has. They move nothing, so their time is File Management time that the application or the I/O library could avoid. In the VTKHDF execution, 9.2% of the 417,596 seeks are redundant.

The **volume in consecutive requests** of the reads and of the writes is reported with the I/O diagnostics. With chunks of 2M instead of 100k, the VTKHDF writes go from 94% to 29% of the bytes in consecutive requests: most writes then follow a forward gap.

Each request is classified within its own file, but the shares of the execution add the requests of all the files and processes, so a file with a different pattern can be hidden by the others. The pattern is therefore also given **per file**: the share of the bytes read and written in consecutive requests of each file is shown with the files (see *files*), and `io_files.csv` has the shares of the requests and of the bytes of every class for each file. In the VTKHDF execution with chunks of 100k, the part files have 93% of the bytes read and 94% of the bytes written in consecutive requests, while the common file has 84% and 55%: its accesses jump more, although they are hidden in the totals because the part files hold 99% of the requests.

The I/O diagnostics are shown in the cards of the I/O Analysis view of the report and printed in the standard output (in both, rows that are zero for all the executions are omitted), and written to `io_metrics.csv`. The request-size distribution is shown in the histograms of the view and in the standard output (share of the requests and of the time), and written to `io_request_sizes.csv` (requests, time, bytes, and their shares). The files with the most I/O time are shown in the view and in the standard output, and all the files are written to `io_files.csv`. The access pattern is shown in the view and in the standard output (share of the requests, of the bytes, and of the time of each class), and written to `io_access_pattern.csv`.

These metrics quantify the time of the File I/O captured in the trace. They do not measure achieved bandwidth or storage-system utilization. When File I/O is a significant performance factor, the trace should be inspected in Paraver to locate the I/O phases and calls responsible.

6.8 General Metrics

BasicAnalysis also reports the classic parallel-performance indicators **Speedup** and **Scaling Efficiency** when multiple execution configurations are analyzed.

These quantities describe the overall scaling behavior of the application and are computed with respect to the reference execution. They are general scaling indicators and do not form part of the multiplicative performance-efficiency model described in the preceding sections.

Let:

Symbol	Description
$T(0)$	Execution time of the reference configuration.
$T(n)$	Execution time of configuration n.
$P(0)$	Application-level parallel-unit count of the reference configuration.
$P(n)$	Application-level parallel-unit count of configuration n.

The parallel-unit count is the application-level execution-unit count used by BasicAnalysis to represent each configuration in the overall scaling experiment. It should not be confused with the Host execution-unit count or the number of physical devices used by the corresponding execution-domain scalability metrics.

6.8.1 Speedup

For strong scaling, BasicAnalysis computes:

$$Speedup(n) = \frac{T(0)}{T(n)}$$

Under ideal strong scaling, Speedup grows proportionally with the increase in parallel resources.

For weak scaling, BasicAnalysis computes:

$$Speedup(n) = \frac{T(0)}{T(n)} \cdot \frac{P(n)}{P(0)}$$

The resource ratio accounts for the increase in workload expected in a weak-scaling experiment. This normalization allows the reported Speedup to retain the resource-growth interpretation: under ideal weak scaling, execution time remains constant while workload and resources increase proportionally, and Speedup therefore follows the resource-growth ratio.

6.8.2 Scaling Efficiency

Scaling Efficiency expresses the achieved scaling behavior relative to the ideal behavior defined by the selected scaling model.

For strong scaling, BasicAnalysis computes:

$$Efficiency(n) = \frac{T(0)}{T(n)} \cdot \frac{P(0)}{P(n)}$$

or equivalently:

$$Efficiency(n) = \frac{Speedup(n)}{P(n)/P(0)}$$

For weak scaling, BasicAnalysis computes:

$$Efficiency(n) = \frac{T(0)}{T(n)}$$

With a user-defined workload ratio $w(n)$, BasicAnalysis computes:

$$Speedup(n) = \frac{T(0)}{T(n)} \cdot w(n)$$

$$Efficiency(n) = \frac{T(0)}{T(n)} \cdot \frac{w(n)}{P(n)/P(0)}$$

Both reduce to the strong-scaling formulas when $w(n) = 1$ and to the weak-scaling formulas when $w(n) = P(n)/P(0)$.

For both scaling models, a value of 1 represents ideal scaling behavior relative to the reference execution. Values below 1 indicate increasing loss of scaling efficiency, while values above 1 indicate scaling behavior better than the corresponding ideal reference, for example in a superlinear strong-scaling case.

Unlike the efficiency factors of the hierarchical performance model, which are reported as percentages, Scaling Efficiency is reported as a dimensionless value, where 1 corresponds to 100% efficiency.

6.9 Metric Availability

Some BasicAnalysis metrics require information that may not be available for every trace or analysis configuration.

BasicAnalysis reports `Non-Avail` when the information required to compute a metric is not available or when the metric is not defined for the corresponding analysis.

Typical cases include:

- application-level Computation Scalability and its associated scalability factors when only one execution configuration is analyzed;
- IPC Scalability, Instruction Scalability, and Frequency Scalability, including their Host-side counterparts when applicable, when the required hardware-counter information is not available;
- Host and Device Computation Scalability when only one MPI+GPU execution configuration is analyzed;
- MPI Serialization Efficiency and MPI Transfer Efficiency when the required Dimemas simulation is unavailable, explicitly skipped, or unsupported for the detected programming model;
- derived OpenMP Serialization and Transfer contributions when the optional MPI+OpenMP hybrid simulation is not requested or cannot be completed;
- MPI I/O Efficiency and MPI I/O Load Balance when no MPI-I/O activity is detected;
- POSIX I/O Efficiency and POSIX I/O Load Balance when no POSIX or ANSI C File I/O activity is detected; and
- metrics that are not defined for the detected trace mode or programming model.

A value reported as `Non-Avail` therefore indicates that BasicAnalysis cannot evaluate that metric from the information available for the current analysis. It should not be interpreted as zero efficiency.

The set of available metrics depends on the trace mode, programming model, number of analyzed execution configurations, available hardware counters, detected runtime and I/O activity, availability of the required simulation, and the selected analysis options.

CHAPTER

SEVEN

PERFORMANCE REPORT

BasicAnalysis generates an interactive performance report that organizes the analysis into complementary views. Each view addresses a different question about the application's performance and helps progressively narrow down the factors responsible for efficiency loss.

The report is intended to support a hierarchical analysis workflow. Start with the Execution Overview to verify the analyzed configurations, then use the Parallel Runtime Model to identify the main sources of efficiency loss. Runtime-Specific Analysis provides a more detailed view of the active parallel runtimes, while the Host/Device Model identifies where accelerator-related inefficiencies manifest. For *nsys2prv* traces with GPU metrics, GPU Hardware Counters shows how the GPU resources are exercised while the device is active. I/O Analysis provides a complementary view of the weight of File I/O and of its Data Access and File Management components. Finally, the Scaling view shows how performance and efficiency factors evolve across the analyzed configurations.

Throughout this workflow, Metric Details provides contextual guidance to help interpret the reported metrics and continue the performance diagnosis.

These views are complementary and may use different analytical scopes. Not all metrics shown in the report belong to the same multiplicative efficiency model.

The following sections describe the report interface, the purpose of each analysis view, and the functionality available to support the performance analysis.

7.1 Report Interface

The BasicAnalysis performance report is organized around the following primary analysis views, shown when they apply to the analyzed traces:

- **Execution Overview**
- **Parallel Runtime Model**
- **Host/Device Model**
- **GPU Hardware Counters** (*nsys2prv* traces with GPU metrics)
- **I/O Analysis**
- **Scaling**

Use the navigation bar shown in Fig. 7.1 to move between the different report views.

Runtime-specific analysis is accessed from the Parallel Runtime Model through the **Runtime Analysis** selector rather than as a separate primary view.

Metric values in the interactive efficiency tables can be clicked to open the **Metric Details** view. This view provides the metric definition, interpretation of the observed value, and recommended next diagnostic steps. The Metric Details functionality is described later in this chapter.

The Execution Overview starts with the note **How to use this report**, and the analytical views with the note **How to read this view**, which also recalls how to open the Metric Details. The notes are collapsed by default; click the title to expand them.

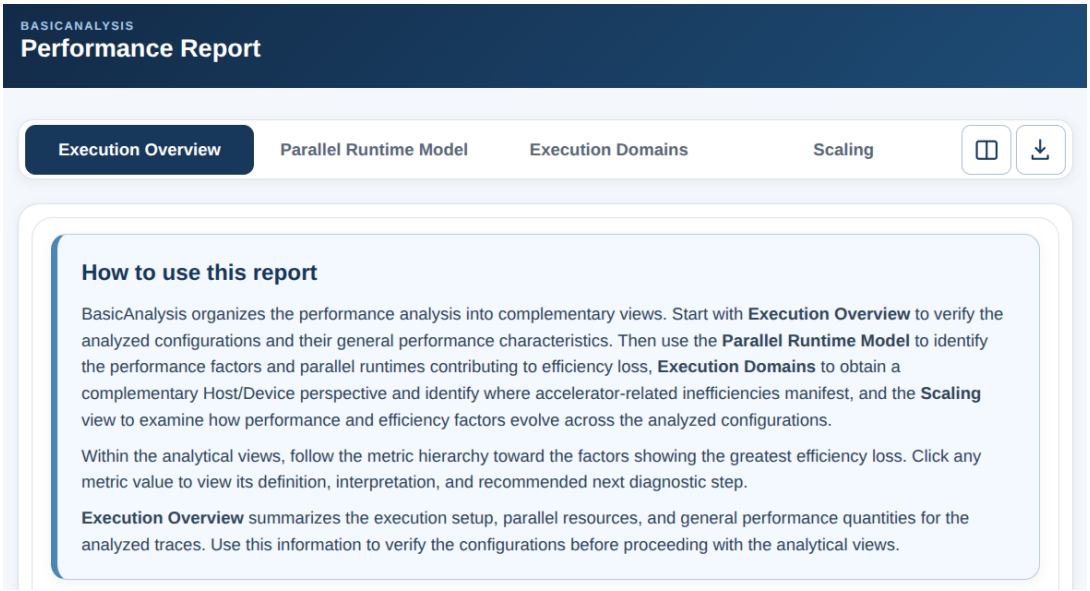


Fig. 7.1: Performance report navigation.

The controls on the right side of the navigation bar provide access to the complementary split view and to the report export functions. These features are also described later in this chapter.

7.2 Execution Overview

The **Execution Overview**, shown in Fig. 7.2, is the starting point of the analysis. It summarizes the execution configuration and general performance characteristics of the analyzed traces.

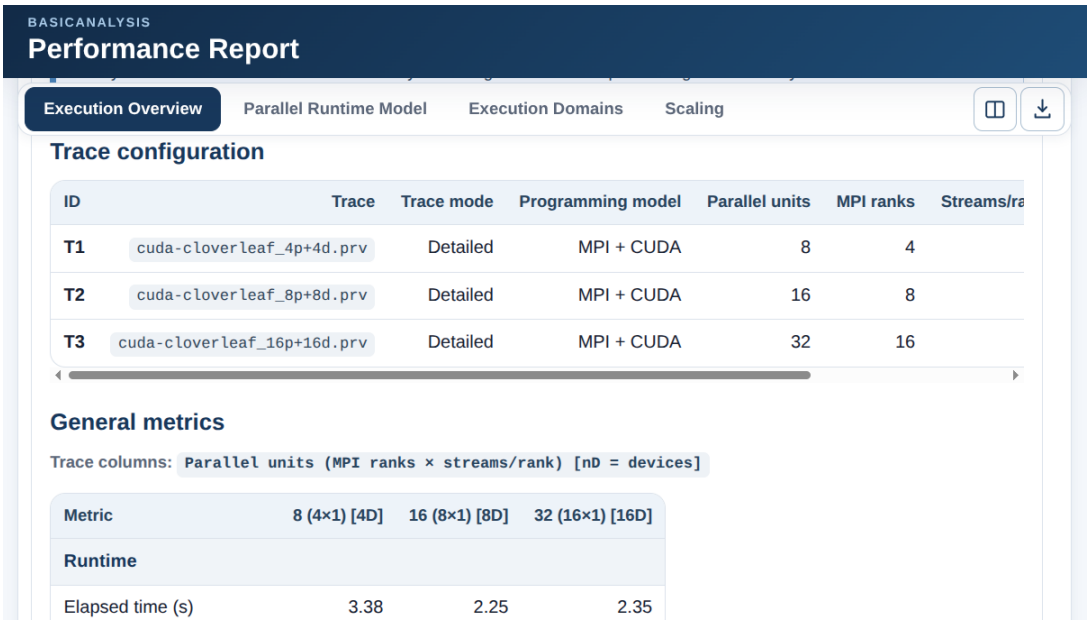


Fig. 7.2: Execution Overview.

Before interpreting efficiency metrics, verify that the reported trace configuration corresponds to the intended experiment. In particular, check the programming model, parallel resources, and accelerator configuration when applicable.

The General Metrics table provides the main execution quantities used to compare the analyzed configurations, including elapsed time, Speedup, Efficiency, IPC, and processor frequency when available.

When several execution configurations are analyzed, these values provide an initial indication of whether application performance improves as resources increase. The detailed efficiency and scaling views should then be used to identify which factors explain the observed behavior.

7.3 Parallel Runtime Model

The **Parallel Runtime Model**, shown in Fig. 7.3, identifies the performance factors and parallel runtimes contributing to efficiency loss.

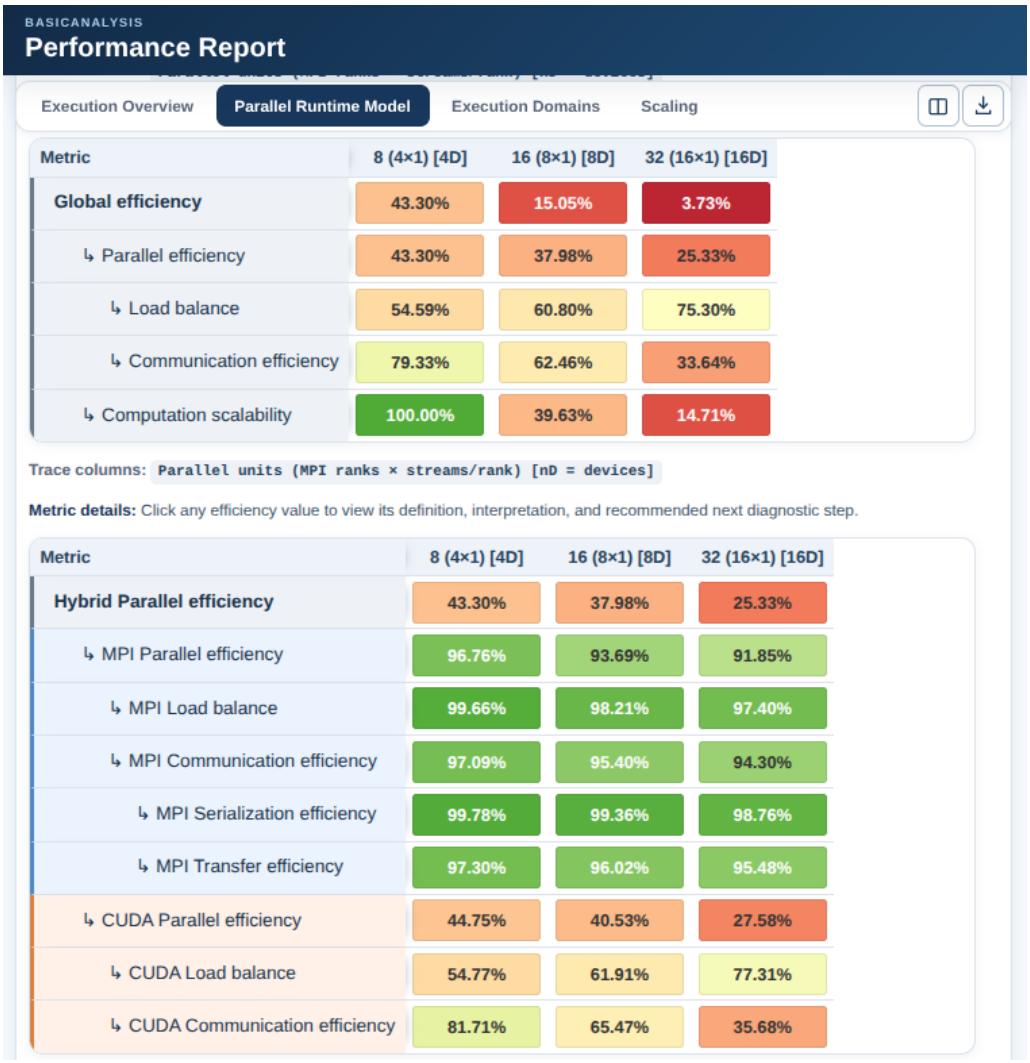


Fig. 7.3: Parallel Runtime Model view.

Start with **Global Efficiency** and follow its child metrics toward the lowest-efficiency factors. Global Efficiency separates losses associated with Parallel Efficiency from changes in Computation Scalability.

For hybrid applications, the Parallel Runtime Model additionally decomposes Parallel Efficiency across the active parallel runtimes.

The model can therefore be read in two complementary ways:

- **by performance factor**, following Global Efficiency toward Parallel Efficiency, Load Balance, Communication Efficiency, and Computation Scalability; or
- **by runtime**, comparing the contributions of MPI and the inner parallel runtime.

The performance-factor hierarchy identifies what type of efficiency loss is present, while the runtime decomposition helps determine which parallel runtime contributes to the observed loss.

In the interactive report, metric values are selectable. Clicking a metric value opens its definition, interpretation, and recommended next diagnostic step.

7.4 Runtime-Specific Analysis

When a runtime requires closer investigation, select it from the **Runtime Analysis** selector. A split view opens, displaying the metrics specific to the selected runtime while keeping the Parallel Runtime Model visible for context. An example for the OpenMP parallel runtime is shown in Fig. 7.4.

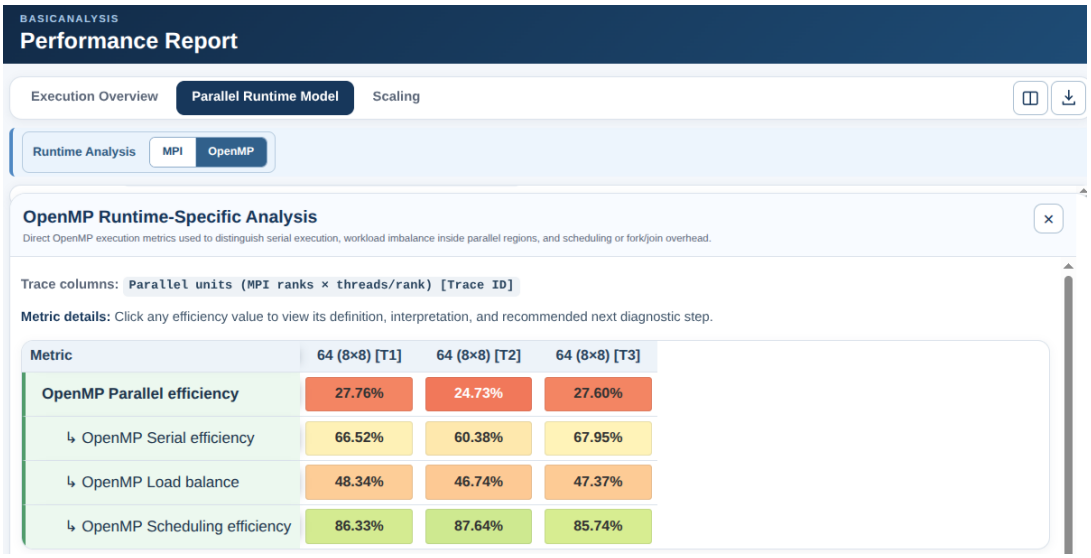


Fig. 7.4: Runtime-specific analysis.

The selected runtime opens in the detailed-analysis panel while the Parallel Runtime Model remains visible for context.

The available runtime-specific metrics depend on the programming model. For example:

- MPI analysis examines MPI Load Balance and Communication Efficiency, including Serialization and Transfer Efficiency when available.
- OpenMP analysis provides the isolated OpenMP Serial, Load Balance, and Scheduling efficiencies.
- CUDA or HIP analysis displays the corresponding accelerator-runtime contribution defined by the selected hybrid model.

Runtime-Specific Analysis is intended to explain the runtime contribution identified in the Parallel Runtime Model. Its metrics may therefore use a different analytical scope from the application-level metrics.

7.5 Host/Device Model

For accelerator applications, the **Host/Device Model** view provides a complementary analysis of where performance inefficiencies manifest by separating the execution into **Device** and **Host** domains, following the TALP model. The Device metrics are shown first.

Fig. 7.5 shows an example of the Host and Device execution-domain analysis.

The Device domain characterizes accelerator execution through Device Parallel Efficiency and its Load Balance, Communication, and Orchestration components, together with Device Computation Scalability. With two or more traces, Device Computation Scalability is split into Device Work and Throughput Scalability (and, for *nsys2prv* traces with GPU metrics, Throughput into Issue and Frequency Scalability), shown in the Device table and in the *Device Computation Scalability* plot of the Scaling view (see *Device Work and Throughput Scalability*), followed by the card **Which kernels change the device time?** (see *Scaling*).

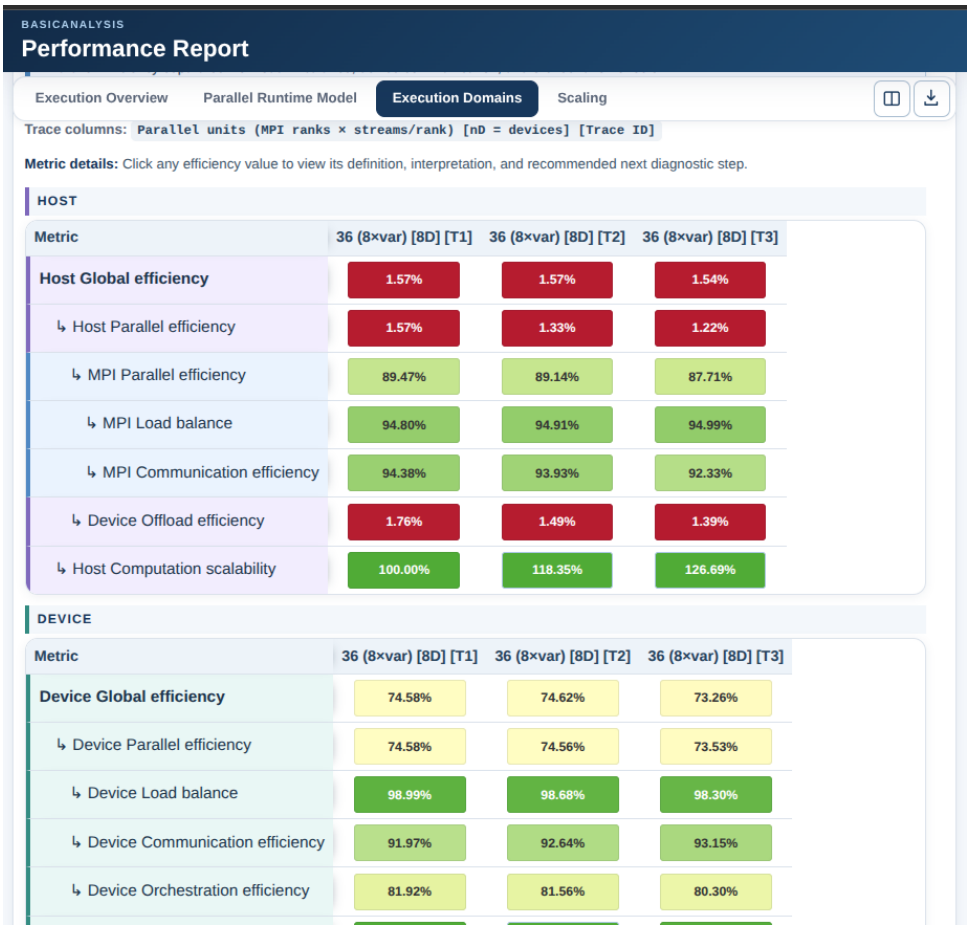


Fig. 7.5: Host and Device execution-domain analysis.

The Host domain characterizes CPU-side execution, including Host Parallel Efficiency, accelerator offload behavior, and Host Computation Scalability.

For Device metrics, GPU stream activity is first mapped and flattened to the corresponding physical devices. Consequently, the Device analysis represents physical accelerator activity rather than treating individual GPU streams as independent parallel execution units.

Host and Device are complementary execution domains and are **not** components of a common multiplicative efficiency model. Therefore, Host Global Efficiency and Device Global Efficiency must be interpreted independently and must not be multiplied to obtain the application Global Efficiency.

Use the Parallel Runtime Model to determine **which runtime contributes to the efficiency loss**, and the Host/Device Model view to determine **where accelerator-related inefficiencies manifest**.

A low execution-domain metric identifies where a performance loss becomes visible. Detailed runtime or trace analysis may still be required to determine the underlying cause.

7.6 GPU Hardware Counters

For *nsys2prv* traces whose Nsight Systems report includes GPU metrics, the **GPU Hardware Counters** view characterizes how the GPU resources are exercised while the device is active. It complements the Device metrics of the Host/Device Model, which characterize how effectively the accelerator is kept supplied with useful work over the execution.

The view, shown in Fig. 7.6, includes:

- **SM Utilization**, the degree to which the SMs issue instructions while the GPU is active (not a fraction of the peak instruction throughput), and its factors **SM Coverage** (are the available SMs given work?) and **SM Issue Efficiency** (do the active SMs issue instructions?). They use the same efficiency colour scale as the other views.
- **Indicators** that explain the SM Utilization chain: GPU Active (a consistency check with the kernel events), Achieved Occupancy (run average, not per kernel), Tensor Core Activity, DRAM Read and Write Bandwidth, and GPC Clock. A higher indicator value is not necessarily better, so they are shown in a single neutral colour, and their Metric Details explain how to read each indicator and what low and high values mean, instead of giving an efficiency verdict.
- A **Diagnostic path** that summarizes the diagnostic workflow of *GPU Hardware Counter Metrics*: SM Coverage, then SM Issue Efficiency, explained with Achieved Occupancy and the DRAM bandwidth, and finally Tensor Core Activity and the GPC Clock as context. The path helps formulate hypotheses to investigate in the trace with Paraver, not assign a root cause.
- A trend plot of SM Utilization and its factors when several executions are analyzed.

As in the other views, click any value to open its Metric Details.

The metrics are averages over the whole execution weighted by the GPU activity. They are not part of the multiplicative efficiency model and are not valid per kernel. See *GPU Hardware Counter Metrics* for their definitions.

7.7 I/O Analysis

The **I/O Analysis** view provides a complementary characterization of the File I/O (POSIX, ANSI C, and MPI-IO calls) captured in the analyzed traces.

The **I/O Analysis** view is shown in Fig. 7.7. Its efficiency table starts with **I/O Critical-Path Efficiency**, the I/O cost on the critical path of the execution, which is the product of:

- **I/O Efficiency**, the time spent in I/O, itself the product of **Data Access Efficiency** (read and write calls) and **File Management Efficiency** (open, close, seek, and control calls); and
- **I/O Load Balance**, the waiting caused by an uneven distribution of the I/O time across the units.

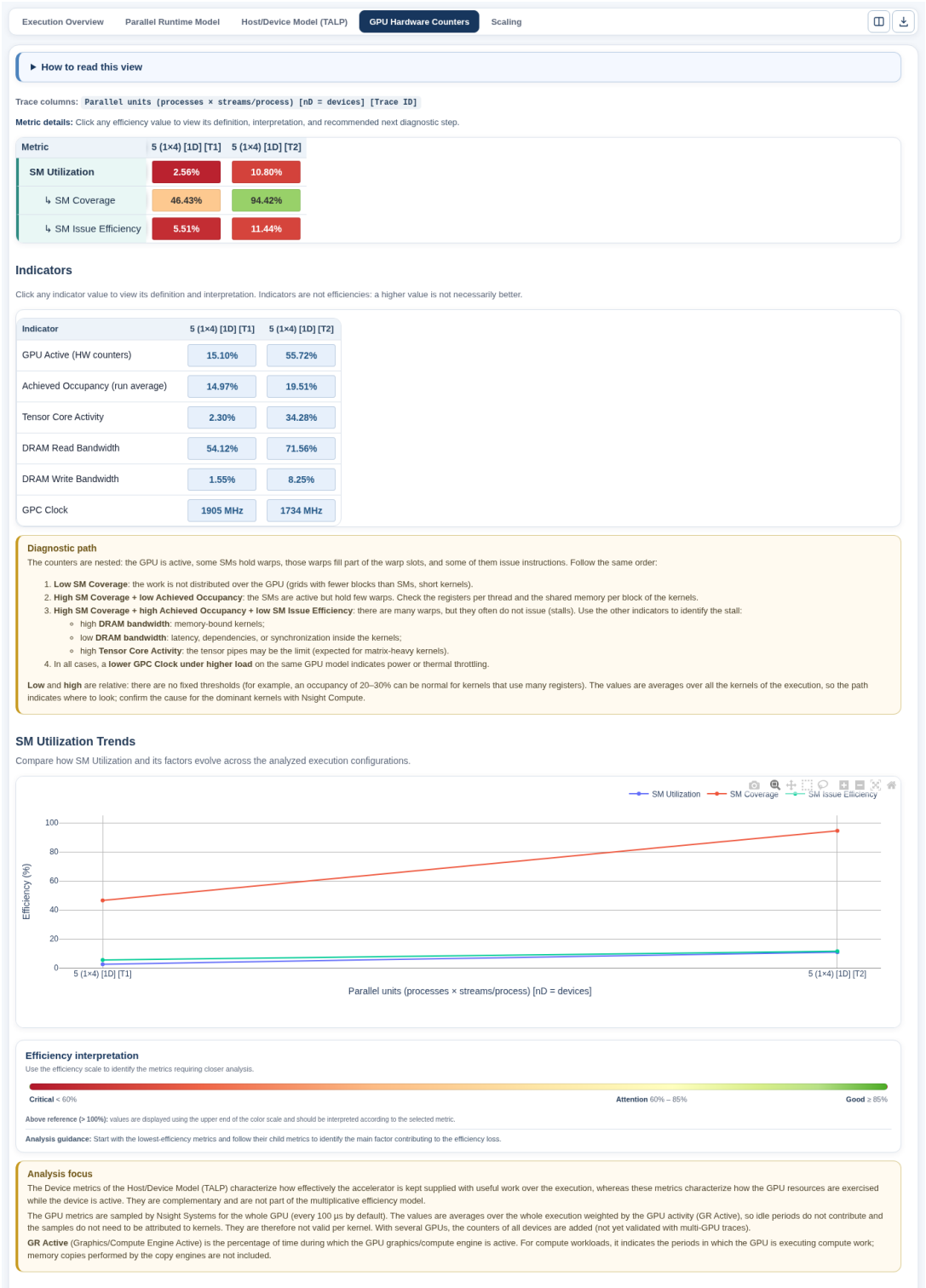


Fig. 7.6: GPU Hardware Counters.

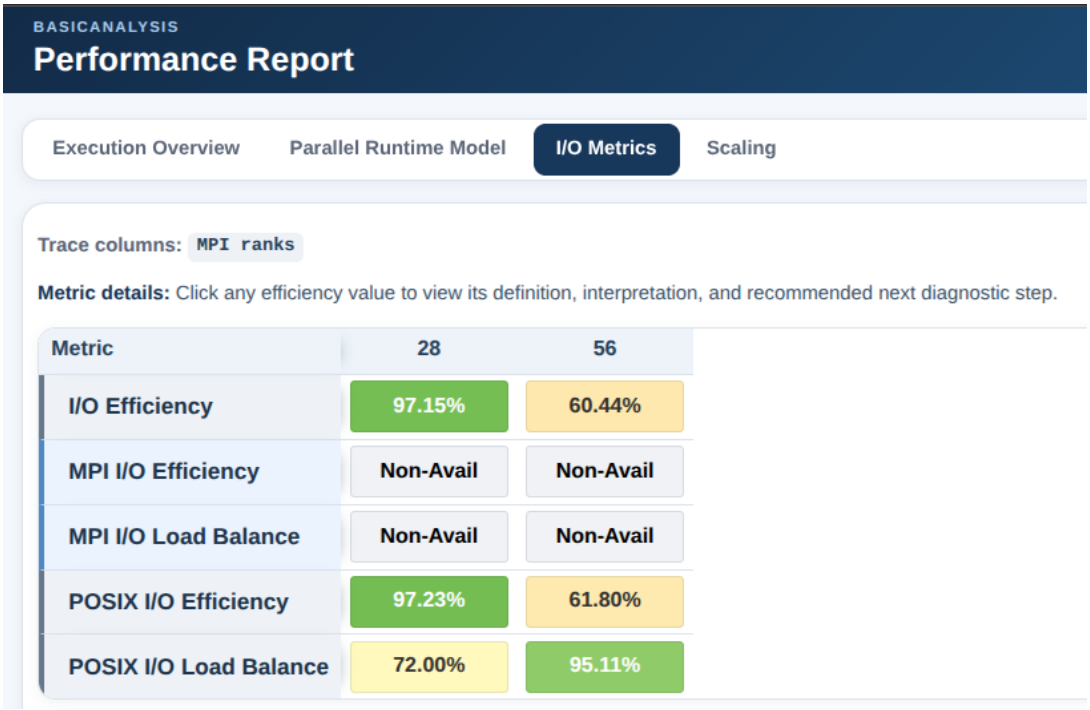


Fig. 7.7: I/O Analysis.

The lowest factor identifies the aspect of the I/O to investigate. The I/O efficiencies use the same thresholds as the other efficiencies, unless others are set for them in `metrics/definitions.yaml` (see *I/O Efficiency*); the efficiency scale of the view shows the thresholds in use.

With several executions, the **I/O Efficiency Trends** plot follows the efficiency table. Below them, the **I/O diagnostics** are shown in four cards, in reading order. Each card starts with a short note on how to read it and shows the diagnostics as plots, one bar per execution; hovering over a bar shows its value, and the **Values** link under each plot opens the table of the plotted values:

- **Where is the I/O time spent?** The share of the I/O time of each call (read and write, the Data Access calls, and open, close, seek, and control, the File Management calls) and, when an execution uses MPI-IO, of each API, as one stacked bar per execution. Then the average and maximum I/O time fractions, the participation, and the load balance of Data Access, read, write, File Management, and open.
- **Which calls take the time?** The time, number of operations, and mean duration (logarithmic scale) of each kind of call, and the share of redundant seeks (seeks to the position the descriptor already has).
- **What requests are made?** The volume and the mean request size (logarithmic scale) of the read and write calls; the request-size coverage is in the values. The *request-size histograms* below show, for the calls of every API (POSIX, ANSI C, and MPI-IO), the share of the requests (top panel) and of the time (bottom panel) in each power-of-two size interval. Compare the two panels: an interval with few requests and much of the time holds the expensive requests. Hovering over a bar shows the number of requests, the time, and the bytes. See *requests*. Above the *access pattern* plots, a legend explains each class with the color of its bars. The plots show, for the reads and the writes of the POSIX and ANSI C calls to regular files, the share of the requests, of the bytes, and of the time that are consecutive, strided, forward, backward, first, or with an unknown offset, as one stacked bar per execution and quantity; the values add the volume in consecutive requests, the volume re-accessed, and the offset coverage. See *access pattern*.
- **Which files are accessed?** A table with the files accessed, maximum processes per file, volume moved by the main process of each file, and file-name coverage. Then, for each execution, the I/O time of the ten files with the most I/O time (POSIX and ANSI C calls), split into read, write, and the other calls, with the share of the bytes moved by the main process of the file at the end of each bar. The last bar collects the calls on descriptors without an open in the trace. Hovering over a bar shows the full path, the processes, the bytes read and written, and the share of the bytes read and written in consecutive requests (the access pattern of the file; also in the values); all the files are in `io_files.csv`. See *files* and *access pattern*.

The printable report shows the plots without the tables of values. These values are not efficiencies, so they have no color scale; rows of calls or APIs that no execution uses are omitted. The same diagnostics are printed in the standard output and written to `io_metrics.csv`; see *I/O diagnostics*.

The I/O metrics are reported independently from the hierarchical performance-efficiency model and must not be interpreted as additional multiplicative components of Global Efficiency, Parallel Efficiency, or Computation Scalability. When no File I/O is detected, they are reported as `Non-Avail`.

Use the I/O Analysis view when File I/O represents a relevant fraction of the execution or when the distribution of I/O activity requires further investigation. Detailed trace analysis or specialized I/O tools may be required to determine the underlying cause of an observed I/O limitation.

7.8 Scaling

The **Scaling** view analyzes how performance and efficiency factors evolve across the analyzed execution configurations.

Fig. 7.8 shows the first part of the Scaling view, which reports the detected and selected scaling model and compares the measured Speedup and Efficiency with their ideal behavior.

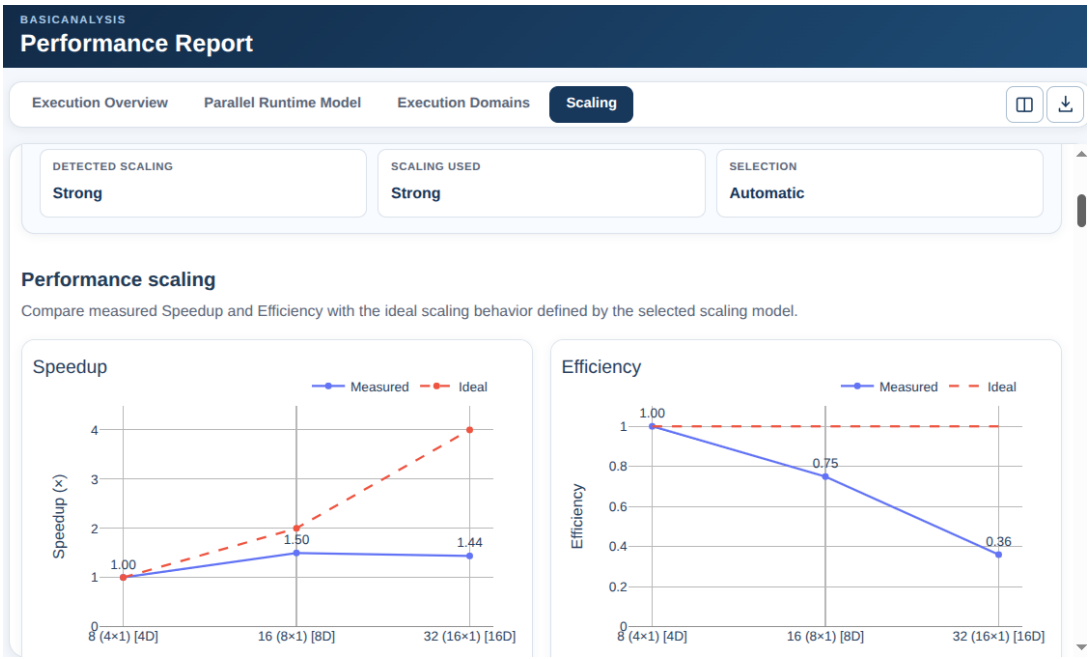


Fig. 7.8: Scaling model and performance trends.

The remaining plots show how the efficiency hierarchy and complementary runtime/domain metrics evolve across configurations. Fig. 7.9 shows an example of one of these metric trends.

Depending on the programming model and the available performance data, these can include:

- Global Efficiency, Parallel Efficiency, and Computation Scalability.
- Load Balance and Communication Efficiency.
- Computation Scalability and its available IPC, Instruction, and Frequency components.
- Runtime-specific Parallel Efficiency contributions.
- MPI Load Balance, Communication, Serialization, and Transfer Efficiency.
- Host execution-domain metrics.
- Device execution-domain metrics.
- I/O-related metrics when available.

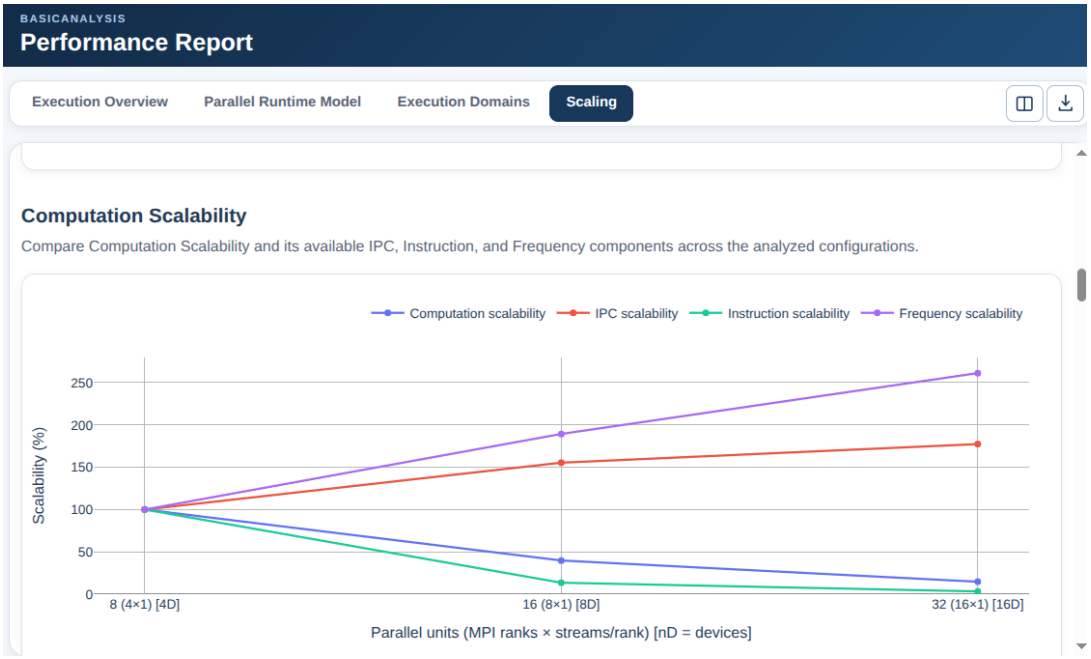


Fig. 7.9: Efficiency-factor scaling trends.

For GPU traces, the *Device Computation Scalability* plot is followed by the card **Which kernels change the device time?**. It lists the ten kernels with the largest share in the change of the device time from the ideal of the scaling model, for one execution compared with the reference (see *Kernel contributions*). A selector chooses the execution (the largest one by default), and the printable version shows every execution. Hover over a kernel for its full name.

Computation Scalability and its associated factors are evaluated relative to a reference execution. A value of 100% indicates behavior equivalent to the reference under the selected scaling model, values below 100% indicate degradation, and values above 100% indicate improvement.

Other efficiency metrics shown in the Scaling view retain their own metric definitions. Their trends should be interpreted by examining how the metric value changes across configurations rather than assuming that every plotted metric is normalized to the reference execution.

Use the trends to identify which factors degrade as the application scales. A degrading metric identifies where the scalability loss becomes visible, but it does not by itself establish the root cause. Further trace analysis may be required to explain the observed behavior.

7.9 Metric Details

The interactive report provides additional information for each performance metric through the **Metric Details** view. This information helps users understand the meaning of a metric value and identify the next steps for further performance analysis.

To open the Metric Details view, click a metric value in any of the interactive efficiency tables. A dialog is displayed for the selected metric and execution, as shown Fig. 7.10.

The dialog identifies the metric, its analysis context, the selected execution, and the corresponding metric value. It then provides:

- **Definition** – explains what the metric measures from a performance-analysis perspective.
- **Interpretation** – explains how to interpret the observed value. When applicable, the interpretation also accounts for the efficiency range associated with the value (for example, critical or low efficiency).
- **Next diagnostic step** – recommends the subsequent analysis to investigate the performance factor represented by the metric, including related metrics or execution domains that should be examined.

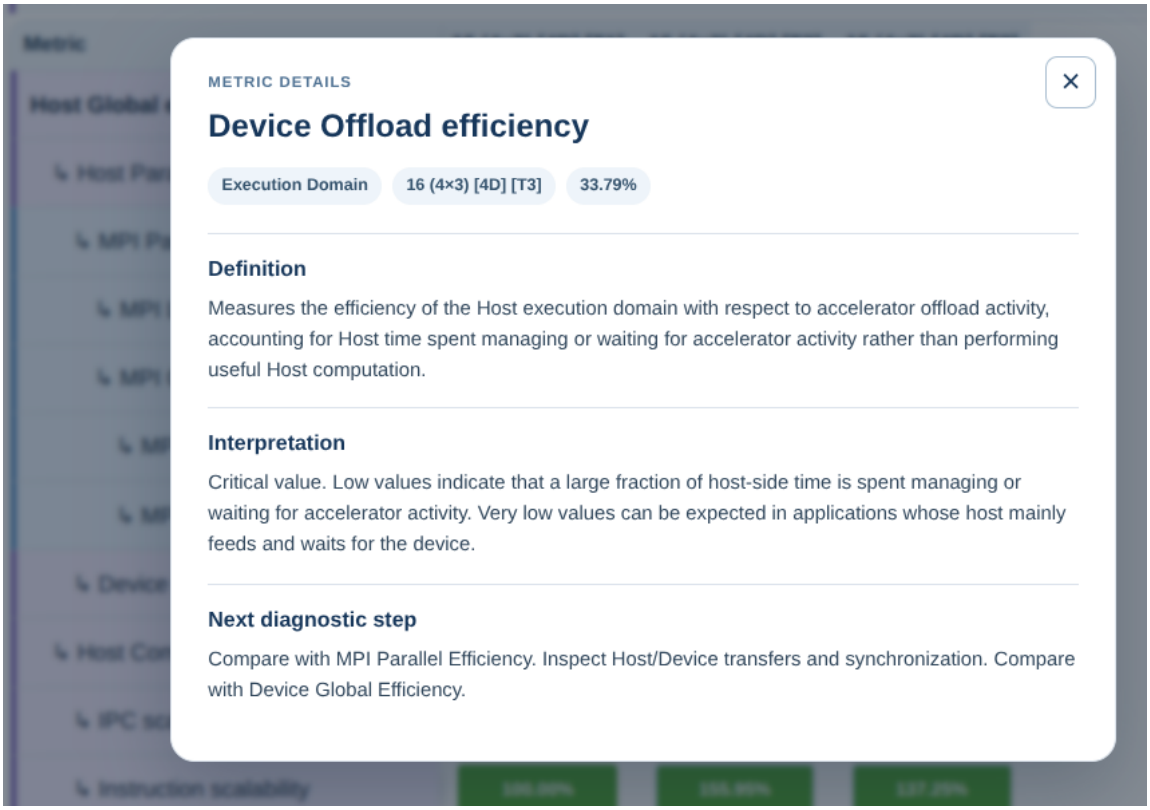


Fig. 7.10: Metric Details view for Device Offload Efficiency. The dialog provides the metric definition, interpretation of the selected value, and recommended next diagnostic steps.

The information presented in **Metric Details** is adapted to the selected metric and its analysis context. This is particularly relevant for hybrid applications, where the report distinguishes between application-level metrics, contributions from the different parallel runtimes, and Host/Device execution-domain metrics.

7.10 Comparing Complementary Views

Fig. 7.11 shows the split-view option, which allows two analysis views to be displayed simultaneously.

This is useful when evidence from two analytical perspectives should be compared directly, for example:

- Parallel Runtime Model with Scaling;
- Parallel Runtime Model with Host/Device Model;
- Parallel Runtime Model with I/O Analysis;
- Host/Device Model with Scaling; or
- Execution Overview with a detailed analytical view.

The two views remain analytically independent. The split layout is intended to facilitate comparison, not to imply that the metrics shown in both panels belong to the same multiplicative model.

7.11 Exporting Report Content

The export control provides several ways to save analysis results.

Depending on the active analysis, efficiency tables can be exported as PNG images. Multiple tables can also be exported together, and all available efficiency tables can be saved in a ZIP archive. In the GPU Hardware Counters view, the exported table includes the indicators below SM Utilization and its factors. The exported tables keep the trace identifiers of the columns (`[T1]`, `[T2]`, ...), so that they can be referenced in papers and slides.

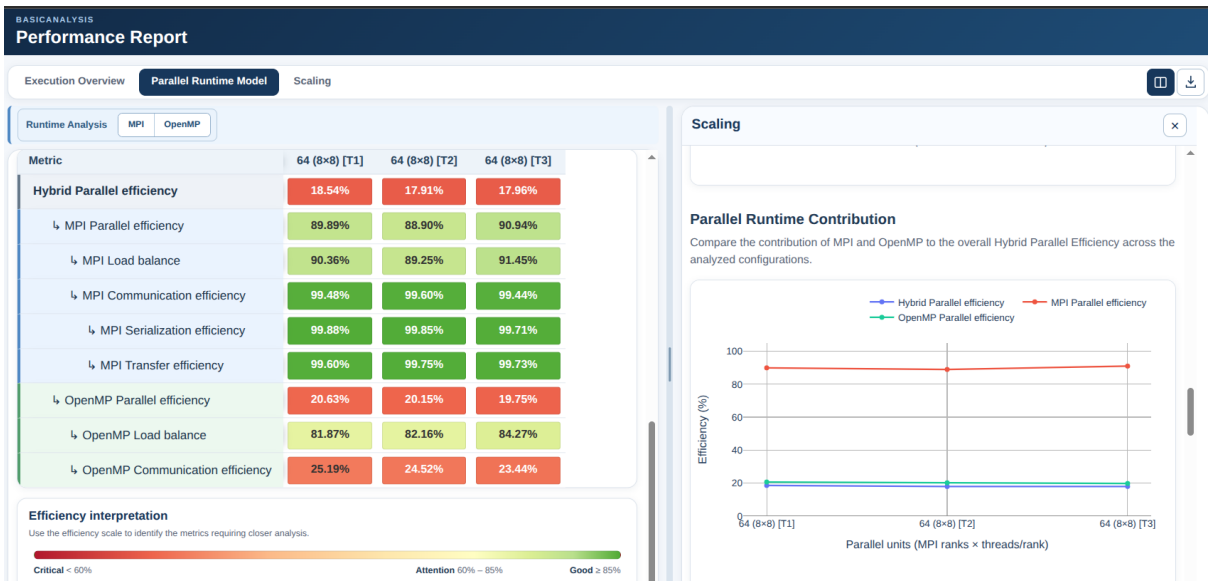


Fig. 7.11: Complementary split-view analysis.

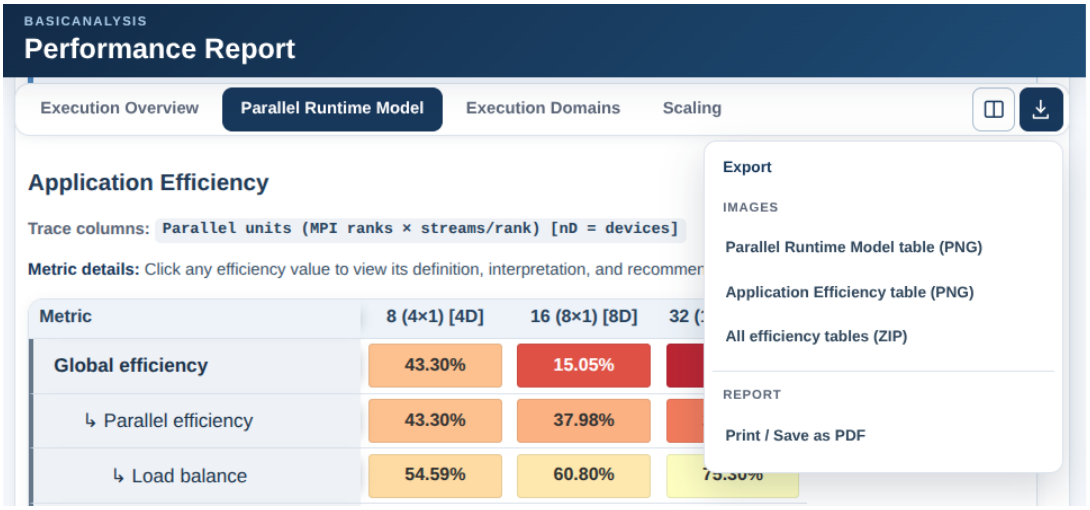


Fig. 7.12: Report export menu.

Fig. 7.12 shows the export menu for saving an efficiency table from the Parallel Runtime Model view as a PNG image.

The complete report can be exported using **Print / Save as PDF**. This generates the printable report from the current analysis data and opens the browser print dialog, where the report can be saved as PDF.

PDF generation is therefore initiated explicitly from the interactive report; BasicAnalysis does not automatically generate a PDF during normal execution.

7.12 Programming Model Comparison

When the analyzed traces use different programming models (see *Comparing programming models*), the report is organized into tabs instead of the views described above:

- **Programming Model Comparison**, which compares all the analyzed traces.
- One tab per programming model, for example **MPI**, **MPI+OpenMP**, or **MPI+CUDA**, with the complete report of the traces of that model.

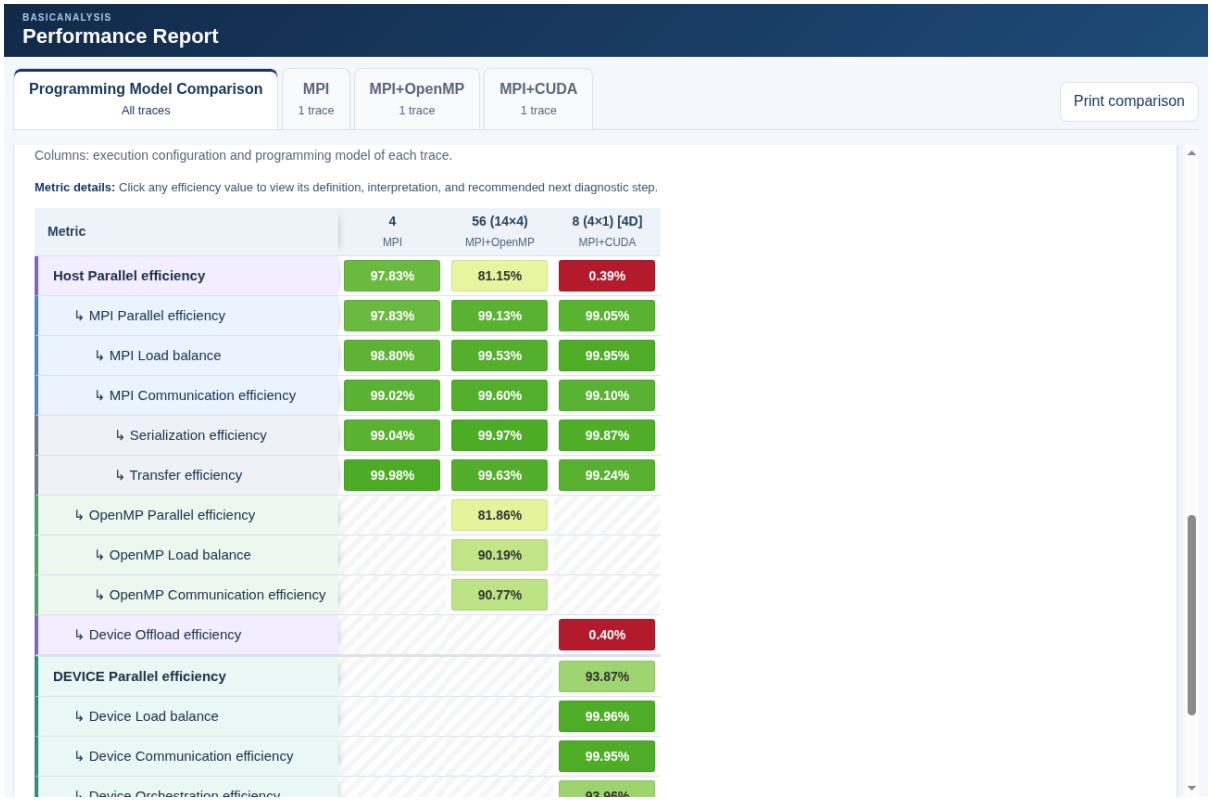


Fig. 7.13: Programming Model Comparison of MPI, MPI+OpenMP, and MPI+CUDA executions.

The **Programming Model Comparison** tab contains:

- the programming models compared and the reference execution used for Speedup;
- the trace configuration and execution mapping of all the traces;
- general metrics: elapsed time, Speedup, average IPC, and average frequency;
- the efficiency metrics of each trace, organized as described in *Comparison of programming models*; and
- I/O metrics, when File I/O activity is detected.

In the efficiency table, shown in Fig. 7.13, each column shows the metrics of the programming model of the corresponding trace. Hatched cells correspond to metrics that do not apply to that programming model, such as the OpenMP metrics of an MPI+CUDA execution. Device metrics are grouped after the Host metrics.

Metric values can be clicked to open **Metric Details**. Each value opens the description of the metric of its own programming model. For example, in the MPI Load Balance row, the value of an MPI execution describes Load Balance, while the value of an MPI+CUDA execution describes the MPI Load Balance of the Host.

Each programming-model tab contains the complete report of the traces of that model, analyzed as an independent set: Execution Overview, Parallel Runtime Model, Host/Device Model, I/O Analysis, and, when the tab contains several traces, Scaling. The report of each tab is the same report that BasicAnalysis generates when those traces are analyzed alone, and it provides its own navigation, Metric Details, and export functions.

The **Print comparison** button prints the Programming Model Comparison tab or saves it as PDF. The report of each programming model is printed from its own export menu.

A typical comparison starts in the Programming Model Comparison tab to identify which implementation shows the largest efficiency losses and in which factor, and continues in the tab of that programming model to analyze the losses in detail.

7.13 Following the Analysis Workflow

The report views are designed to be used together rather than as independent analyses. A typical analysis follows this progression:

1. **Verify the executions** in the Execution Overview.
2. **Identify the dominant efficiency loss** using Global Efficiency and its hierarchy.
3. **Determine the responsible parallel runtime** using the Parallel Runtime Model.
4. **Explain the runtime contribution** using Runtime-Specific Analysis.
5. For accelerator applications, **identify where the inefficiency manifests** using the Host and Device execution domains.
6. When File I/O activity is relevant, **evaluate its weight and distribution** using I/O Analysis.
7. **Examine how the relevant factors evolve** across configurations in the Scaling view.
8. Use detailed trace analysis or specialized performance tools when additional evidence is required to determine the underlying cause.

This workflow is not strictly linear. The different views provide complementary evidence and may use different analytical scopes.

The Execution Overview establishes the context of the experiment. The hierarchical performance model identifies whether a significant performance loss exists and which broad factor dominates. The Parallel Runtime Model and Runtime-Specific Analysis attribute and explain parallel-runtime losses. The Host/Device Model localizes accelerator-related inefficiencies to the Device or Host side. I/O Analysis independently characterizes the contribution and distribution of File I/O activity. Finally, the Scaling view shows how these relevant performance characteristics evolve as the execution configuration changes.

Together, these views progressively narrow the analysis from identifying a performance problem to determining which factors, runtimes, execution domains, or complementary activities should be investigated in greater detail.

CHAPTER

EIGHT

OUTPUT

BasicAnalysis generates several complementary outputs from the analyzed Paraver traces. These include numerical results, static visualizations, an interactive performance report, and intermediate data produced during the analysis.

The exact files generated depend on the programming model, the available performance data, the number of analyzed traces, and the selected analysis options.

8.1 Performance Report

The main output of BasicAnalysis is the interactive performance report:

```
basicanalysis_interactive_report.html
```

The report provides a guided view of the performance analysis through the Execution Overview, Parallel Runtime Model, Runtime-Specific Analysis, Host/Device Model, I/O Analysis, and Scaling views, as applicable to the analyzed execution.

The available views depend on the programming model and performance data. For example, the Host/Device Model view is available for accelerator applications, while I/O Analysis reports I/O Efficiency and its Data Access and File Management factors when POSIX, ANSI C, or MPI-IO calls are detected in the traces. When multiple execution configurations are analyzed, the report also provides scaling information and performance trends.

Metric values in the report are interactive. Selecting a metric opens its definition, interpretation, and recommended next diagnostic steps.

Efficiency tables can be exported from the report as PNG images. The report also provides export options for multiple tables and can be printed or saved as a PDF using the browser.

When the analyzed traces use different programming models, the report contains a Programming Model Comparison tab and one tab per programming model with the complete report of its traces.

See *Performance Report* for a detailed description of the report, its analytical views, and its export capabilities.

8.2 Numerical Output

BasicAnalysis writes the numerical results of the analysis to CSV files in the output directory.

The main files include:

`rawdata.csv`
: Contains the aggregated performance measurements extracted or constructed from the input traces and used to compute the performance metrics. These include timing and hardware-counter measurements and, when applicable, runtime-specific, I/O, and accelerator-related quantities.

`other_metrics.csv`
: Contains complementary performance quantities such as elapsed time, Speedup, Efficiency, IPC, processor frequency, tracer flushing information, and I/O Efficiency with its Data Access and File Management factors.

`io_metrics.csv`
: Written when File I/O is detected. Contains I/O Efficiency, its factors, and the I/O diagnostics: time fractions, participation, contributions, load balance, the time, count, and mean duration of each group of I/O calls, the read and write volume, mean request size, and request-size coverage, the files accessed, maximum processes per file, volume moved by the main process of each file, and file-name coverage, and the volume in consecutive requests, volume re-accessed, and offset coverage of the reads and writes, and the share of redundant seeks.

`io_request_sizes.csv`
: Written when the traces have read or write requests. One row per execution, operation, API (POSIX/ANSI C or MPI-IO), and power-of-two request-size interval, with the number of requests, time, bytes, and their shares of the operation.

`io_files.csv`
: Written when the traces have POSIX or ANSI C calls. One row per execution and file accessed, in decreasing I/O time, with the processes, opens, I/O time, the requests, bytes, time, and processes of the reads and writes, the bytes of the main process and their share, the shares of the I/O time and of the bytes, and the access pattern of the reads and writes of the file (share of the requests and of the bytes of each class). The row *(descriptors without a traced open)* collects the calls on descriptors without an open in the trace.

`io_access_pattern.csv`
: Written when the traces have POSIX or ANSI C reads or writes to regular files. One row per execution, operation, and access-pattern class (first, consecutive, strided, forward, backward, unknown offset), with the number of requests, bytes, time, and their shares of the operation.

`modelfactors.csv`
: Contains the application-level hierarchical efficiency and scalability factors computed by BasicAnalysis.

`efficiency_table*.csv`
: Contains efficiency metrics arranged in the format used to generate the corresponding static efficiency tables. The exact filename depends on the performance model being analyzed.

Additional model-specific CSV files may also be generated. For example, OpenMP analyses can generate `omp_talp_metrics.csv`, while MPI+GPU analyses using the Host/Device execution-domain model can generate `talp_metrics.csv`. For *nsys2prv* traces with GPU metrics, BasicAnalysis also writes `device_hwc_metrics.csv` with the GPU hardware counter metrics (percentages, and the GPC clock in MHz) and adds the integrals of the counters to `rawdata.csv`.

For GPU traces, `talp_metrics.csv` also contains the Device Work and Throughput Scalability (and, for *nsys2prv* traces with GPU metrics, the Device Issue and Frequency Scalability) when two or more traces are analyzed (see *Device Work and Throughput Scalability*). `rawdata.csv` adds the number of kernels executed on the devices and the GPU threads launched, and, for *nsys2prv* traces with GPU metrics, the integral of SM Issue multiplied by the GPC clock. With two or more GPU traces, BasicAnalysis also writes:

`device_kernels.csv`
: One row per execution other than the reference and kernel (or kernel family), in decreasing change of the kernel time, with the work ratio of the scaling model, the share of the kernel in the change of the device time from the ideal, and the calls, time (us), and GPU threads launched of the kernel in the reference and in the execution (see *Kernel contributions*). The kernel column has the full name recorded in the trace.

When the analyzed traces use different programming models, BasicAnalysis also writes:

`comparison_metrics.csv`
: Contains the comparison of the efficiency metrics of the traces, with the same rows as the comparison table printed on the terminal. Cells of metrics that do not apply to a programming model are empty. In this case, the reference-based metrics in `modelfactors.csv` and `other_metrics.csv`, such as scalability metrics and Efficiency, are reported as `N/A`.

The availability of individual files and metrics depends on the programming model, the information contained in the traces, the number of analyzed execution configurations, and the selected analysis options.

8.3 Static Visualizations

BasicAnalysis generates static visualizations of the performance metrics when the required Python plotting dependencies are available.

Efficiency tables are generated as both PNG and PDF files. Depending on the performance model, the output includes files such as:

```
efficiency_table-matplot.png
efficiency_table-matplot.pdf
```

or the corresponding global, hybrid, runtime-specific, or Host/Device variants. For the Host/Device model, `efficiency_table_talp_matplot` shows the Host and Device metrics together, and `efficiency_table_talp_host_matplot` and `efficiency_table_talp_device_matplot` show each execution domain on its own, for example for presentations that focus on the host or on the device. For *nsys2prv* traces with GPU metrics, `efficiency_table_device_hwc_matplot` shows SM Utilization and its factors, and `table_device_hwc_indicators_matplot` shows the indicators (GPU Active, Achieved Occupancy, Tensor Core Activity, DRAM Read and Write Bandwidth and GPC Clock) with a single neutral colour, since they are not efficiencies.

Each column of the static plots is labelled with its configuration and the trace identifier used in the interactive report (`[T1]`, `[T2]`, ...), so that the same trace can be referenced in a paper or in slides, even when several traces have the same configuration. Metrics without a value for any trace (for example IPC, Instruction and Frequency scalability when the traces have no hardware counters) are not shown in the static efficiency tables; the CSV files keep all the metrics.

When multiple execution configurations are analyzed, BasicAnalysis also generates plots showing the evolution of performance metrics across the configurations. These include Speedup and Efficiency plots and, depending on the programming model and available data, plots for:

- Global Efficiency and Parallel Efficiency;
- Load Balance and Communication Efficiency;
- Computation Scalability and its available factors;
- runtime-specific efficiency factors; and
- Host and Device execution-domain metrics.

Some multi-configuration analyses also generate Gnuplot `.gp` files containing plotting instructions and the corresponding metric data.

I/O metric plots are provided within the interactive report and are not generated as separate static visualization files.

When the analyzed traces use different programming models, the static efficiency tables and the scaling plots are not generated, because they assume a single programming model. The comparison is available in the terminal output, in `comparison_metrics.csv`, and in the interactive report.

8.4 Intermediate Analysis Files

BasicAnalysis stores the intermediate data produced while analyzing each trace under:

```
scratch_out_basicanalysis/
```

A separate subdirectory is created for each analyzed trace. These directories contain the Paraver/paramedir statistics used to derive timing, hardware-counter, I/O, runtime-specific, and accelerator measurements.

For accelerator traces, the intermediate data can also include Host, GPU-stream, and memory-transfer statistics. GPU stream activity is subsequently mapped and flattened to physical devices when constructing the Device measurements used by the performance metrics.

OpenMP analyses include additional timing data used to characterize serial execution, parallel-region load balance, and scheduling and fork/join overhead.

When Dimemas simulation is applicable, Dimemas is available, and simulation has not been disabled, the corresponding trace directory also contains the simulation input, configuration, simulated trace, and paramedir statistics obtained from the simulated execution.

These files represent intermediate analysis data. The summarized measurements and computed metrics intended for direct inspection are provided separately by the CSV files in the main output directory.

CHAPTER

NINE

LIMITATIONS

This section summarizes the main limitations of the current BasicAnalysis methodology and implementation. Metric-specific availability conditions are described in *Performance Metrics*.

9.1 Simulation-Derived Metrics

Serialization Efficiency and Transfer Efficiency rely on an ideal execution simulated with *Dimemas*. These metrics are therefore available only for programming models and runtime events supported by the current BasicAnalysis and *Dimemas* simulation workflow.

BasicAnalysis currently uses *Dimemas* simulation for MPI, MPI+OpenMP, and MPI+CUDA applications.

For MPI+HIP applications, MPI Communication Efficiency can be computed from the measured execution, but Serialization Efficiency and Transfer Efficiency are reported as `Non-Avail`. The current BasicAnalysis-*Dimemas* workflow has not been validated for HIP accelerator events, and BasicAnalysis therefore does not use simulation-derived communication submetrics for these executions.

Simulation-derived metrics are also unavailable when *Dimemas* is not installed, when the required simulation cannot be completed, or when simulation is explicitly disabled with the `--skip-simulation` option.

9.2 Number of Parallel Runtimes

The Parallel Runtime Model has been validated for applications with up to two parallel runtime levels, such as MPI+OpenMP and MPI+GPU.

Applications containing more than two parallel runtimes may be detected in the trace, but their decomposition into runtime-specific efficiency contributions has not been validated.

For example, an application combining MPI, a CPU threading runtime, and a GPU runtime introduces three parallel runtime levels. The current Parallel Runtime Model does not provide an independently validated efficiency decomposition for all three runtime contributions.

The application-level metrics may still provide useful performance information, but the runtime-specific decomposition should not be interpreted as a complete attribution of the efficiency loss across all active runtimes.

9.3 Hierarchical Execution Model

The Parallel Runtime Model assumes a hierarchical organization of the parallel runtimes. For hybrid applications, MPI is considered the outer parallel level and the second runtime, such as OpenMP, CUDA, or HIP, the inner parallel level.

The multiplicative runtime decomposition therefore assumes that the execution of the inner parallel runtime is compatible with this hierarchical organization.

For example, in an MPI+OpenMP application, when the master thread executes MPI calls outside an OpenMP parallel region, the remaining OpenMP worker threads are considered inactive. The corresponding unused thread capacity can then be attributed to the OpenMP serial component.

Applications whose runtime behavior substantially violates the assumed hierarchical organization, for example through concurrent useful activity across runtime levels that the model treats as mutually exclusive, may not be fully represented by the runtime decomposition.

9.4 GPU Execution-Domain Analysis

The Host/Device execution-domain analysis for MPI+CUDA and MPI+HIP applications depends on the accelerator activity represented in the Paraver trace.

GPU traces can contain multiple execution streams associated with the same physical device. BasicAnalysis maps these streams to physical devices and flattens overlapping useful-computation and memory-transfer intervals before constructing the Device metrics.

Consequently, the correctness of the Device metrics depends on BasicAnalysis being able to identify the GPU streams and map them to their corresponding physical devices from the trace information.

Device activity that is not represented in the trace, or accelerator configurations for which the stream-to-device mapping cannot be established correctly, cannot be fully characterized by the Device execution-domain metrics.

Device Work and Throughput Scalability (see *Device Work and Throughput Scalability*) depend on the work measure:

- *GPU threads launched.* Kernels launched with a fixed grid (grid-stride loops, reductions) launch the same threads whatever the data size, and libraries (cuBLAS, cuDNN, attention kernels) change the tile size and the work per thread with the problem size; in these kernels the threads launched do not follow the work. In a vLLM inference on one GPU (batch size 1 and 256), the threads launched grow 14.8 times when the issued instructions grow 26.9 times.
- *Issued instructions.* They are estimated from SM Issue and the GPC clock sampled by Nsight Systems (every 100 µs by default); kernels shorter than the sampling period are averaged with their neighbours.
- Both measures compare executions on the same GPU model.
- Kernels that overlap on different streams of a device count once in the useful device time and fully in the work, so the overlap appears as throughput.
- Kernels still running at the end of a cut trace are not counted in the kernel contributions and in the GPU threads launched.

9.5 GPU Trace Sources: Extrae and nsys2prv

BasicAnalysis computes the Host and Device metrics for GPU traces generated with Extrae and with *nsys2prv*. Both sources provide the metrics, but they record different information, so some quantities are obtained under different assumptions. The definitions are described in *Performance Metrics*.

Quantity	Extrae	nsys2prv
Useful Device time	*Running* state of the GPU streams (kernel execution)	Kernel events (`63000006`) of the GPU streams
Device communication	*Memory transfer* state of the GPU streams	Memory copies and `memset` (`63000001`) and NCCL kernels (`63000007`)
`memset` on the device	Not recorded on the GPU streams: not counted as useful time or communication; it appears as idle device time	Counted as a memory operation (communication)
`memset` on the host	Time in the call: Device Offload loss	Time in the call: Device Offload loss
Useful Host time	*Running* state of the host threads	Elapsed time minus the time in CUDA calls
Host hardware counters (IPC, instructions, frequency)	Available when recorded	Not available
GPU hardware counters	Not available	Available when the report includes GPU metrics (`Metrics GPU<n>` threads)
Dimemas simulation	Available for MPI+CUDA	Not available

Assumptions for nsys2prv traces

- **Useful Host time.** Without state records, the useful Host time is the elapsed time minus the time inside CUDA runtime and driver calls. It includes any Host time outside those calls, such as interpreter, waiting, or I/O time in Python applications. Device Offload Efficiency is therefore an upper bound of the efficiency that would be obtained with state records.
- **Processes without GPU streams.** Only the processes with GPU streams are analyzed as Host. Processes that do not offload work to the accelerator are reported in the standard output and are not included in the metrics.
- **Memory operations.** Following the TALP device model, `memset` and all memory copies count as communication. *nsys2prv* does not distinguish Device-to-Device copies within a GPU from copies between GPUs, so both are counted as communication.
- **Region of interest.** The metrics are computed over the whole trace, which BasicAnalysis assumes is the region of interest, as for Extrae traces. When the trace includes initialization (for example, loading model weights), that phase is part of the metrics, and it can dominate them: in a vLLM benchmark, the inference phases started at 38.6 s of a 53.7 s trace. The trace should therefore be cut to the region of interest with the Paraver Cutter before the analysis; the NVTX ranges recorded by *nsys2prv* (event `9600`) show where the application phases start and end. BasicAnalysis reports one set of metrics for the analyzed trace and does not break them down per NVTX range: comparing phases is part of the detailed analysis in Paraver.
- **Programming models and validation.** Only CUDA is supported; the configurations use the CUDA event types of *nsys2prv*. The metrics have been validated with single-GPU traces and with a multi-node trace of 32 GPUs (one GPU per process, NCCL collectives); several GPUs per process and shared GPUs are covered by the implementation but have not yet been validated with real traces.
- **Nodes.** *nsys2prv* does not record the node of each process: traces merged from several nodes place every process on the same node, and the device numbers repeat on each node. When the trace has more `Metrics GPU<n>` threads than GPUs identified by node and device number, BasicAnalysis assumes that each process uses its own GPUs (processes do not share a GPU). Without GPU metrics threads this cannot be checked: processes with the same device number are then analyzed as sharing the GPU, and BasicAnalysis prints a warning.
- **Trace metadata.** The `.row` file is required to identify the GPU streams, the GPU metrics threads, and the host threads.
- **GPU hardware counters.** The GPU metrics are sampled for the whole GPU (every 100 µs by default), so they cannot be attributed reliably to individual kernels, which are often shorter than the sampling period or run concurrently on several streams. BasicAnalysis reports them as averages over the whole execution

weighted by the GPU activity (GR Active), which do not need that attribution; they are not valid per kernel. Additional assumptions:

- The sampled values are integers, so low percentages (for example, SM Issue) lose precision.
- An idle GPU reports GR Active = 1%, never 0%, with SMs Active and SM Issue = 0. *nsys2prv* copies the values of the Nsight Systems report unchanged, so this floor comes from Nsight Systems; no official Nsight documentation of it has been found. These samples are included in the GR Active integral, so GPU Active is slightly overestimated and the metrics divided by GR Active (SM Coverage, SM Utilization, Tensor Core Activity, DRAM bandwidth, and GPC Clock) are slightly underestimated. The effect grows with the idle time: in the vLLM traces analyzed, the idle samples are 5.1% of the GR Active integral for a mostly idle GPU (SM Coverage 46.43% instead of 48.92%) and 0.8% for a busy GPU (94.42% instead of 95.18%). SM Issue Efficiency and Achieved Occupancy are not affected.
- The GPC clock is recorded in Hz although its label says MHz; BasicAnalysis converts averages above 10^5 to MHz.
- The control window of the configurations assumes that the GPC clock is sampled during the whole execution.
- The integrals cover the whole trace as loaded by Paraver, including the samples written after the end declared in the header, and the last sample of each GPU is extended until the end of the trace. GPU Active is computed over that same period. When the sampling of the GPUs ends at different times, GPU Active is overestimated (in a 32-GPU trace, 97.06% instead of 93.36% for the measured samples only).
- With several GPUs, the counters of all `Metrics GPU<n>` threads are added.
- The *Avg* variants of the warp counters are not used: values that underflow (for example, 4294967293) have been observed.

Limitations for Extrae traces

- The device-side execution of `cudaMemset` and `cudaMemsetAsync` is not recorded on the GPU streams; only the host-side call is recorded. Its device time therefore lowers Device Orchestration Efficiency instead of Device Communication Efficiency.
- Extrae does not record the direction of memory copies, so Device-to-Device copies within a GPU are counted as communication, as in *nsys2prv* traces.

The differences between the two sources are small in the traces analyzed so far, but they should be considered when comparing Extrae and *nsys2prv* traces of the same application.

9.6 I/O Analysis

The I/O metrics characterize the time of the File I/O captured in the trace: the POSIX, ANSI C, and MPI-IO calls recorded by Extrae, classified into Data Access and File Management.

The following assumptions and limitations apply:

- **Nested calls.** Extrae does not trace, by default, the POSIX calls performed inside MPI-IO calls. BasicAnalysis assumes this default; if the internal POSIX calls were traced, their time would be counted twice.
- **I/O library computation.** The CPU work of I/O libraries (for example HDF5 compression or metadata handling) is not distinguished from the application computation and counts as useful time. A library that spends more CPU time therefore increases I/O Efficiency.
- **Descriptor types.** Calls on non-file descriptors (standard output, sockets, pipes) are counted as File I/O; only the access pattern excludes them.
- **I/O not visible in the trace.** Memory-mapped I/O, asynchronous I/O, and I/O performed by threads that are not instrumented are not measured.
- **Whole-execution aggregation.** The metrics describe the whole execution; applications with distinct I/O phases (for example, a write phase and a read phase) should be inspected per phase in Paraver.

- **Critical path.** I/O Load Balance and I/O Critical-Path Efficiency assume that the units synchronize after each kind of I/O call, so that the others wait for the slowest unit. When the I/O of a unit overlaps the computation of the others, they underestimate the efficiency.
- **Request sizes.** The volume and the request-size distribution include the read and write calls whose request size is recorded in the trace (see the request-size coverage). The MPI-IO size is the requested size (count times the datatype size), which may be more than the bytes read at the end of a file. The nonblocking MPI-IO calls (`MPI_File_iread`, `MPI_File_iwrite`) are not instrumented by Extrae. Zero-byte requests are not counted. When a call starts before the previous call of the same thread has ended (unclosed or nested calls, rare in Extrae traces), only the later call is counted.
- **Files.** Only the POSIX and ANSI C calls are attributed to files: MPI-IO records neither the file name nor the file handle. A call is attributed through its descriptor to the last `open` or `fopen` of the process that returned it; calls on descriptors opened before the tracing starts or with calls that Extrae does not instrument (`openat`, `creat`, `dup`), and after an `open` or `fopen` without a file name in the trace, are not attributed to a file (see the file-name coverage). Files are identified by the name passed to `open`: the same file opened with different paths (relative and absolute, or through links) appears as several files. A failed `open` counts as an open of the file.
- **Access pattern.** The offset of the reads and writes is rebuilt from the `open` calls, the seeks, and the request sizes. Extrae records the seek offset without its origin (`whence`): every seek is taken as absolute (`SEEK_SET`), so a relative seek (`SEEK_CUR`, `SEEK_END`) misplaces the following requests of the descriptor. The seek offset is recorded only by recent Extrae versions; with older versions, the position is unknown after the first seek. The size is the requested size, so a short read advances the position too far. The open flags are not recorded: the writes to a file opened with `O_APPEND` go to its end, whatever the position. The `pread`/`pwrite` family, MPI-IO, and descriptors without a traced `open` (until their first seek) have no known offset (see the offset coverage). `fread`/`fwrite` show the pattern presented to the C library, before its buffering. The pattern is given per process: how the regions of several processes fit together in a shared file is not analyzed.

These metrics do not characterize achieved I/O bandwidth, storage-system utilization, or filesystem contention. They identify whether File I/O has a significant execution-time contribution, which component (Data Access or File Management) is responsible, and how it is distributed, but they do not by themselves determine the underlying storage-system cause.

When I/O activity represents a significant performance factor, detailed trace analysis or specialized I/O performance-analysis tools may be required for further diagnosis.

9.7 Comparison of Programming Models

The comparison of executions that use different programming models is supported for Detailed traces of MPI-based programming models, such as MPI, MPI+OpenMP, MPI+CUDA, and MPI+HIP. Burst traces and traces without MPI cannot be included in such a comparison and must be analyzed separately.

Scalability metrics, Global Efficiency, Efficiency, and the detection of the scaling model are not computed across programming models, because they would compare computation performed on different kinds of resources. Speedup is computed relative to the first trace and should be interpreted as a comparison of elapsed times. When the compared executions solve different problems or problem sizes, Speedup does not represent a performance improvement of the same computation.

The comparison relates metrics that represent the same performance factor in each programming model, but these metrics may be computed from different quantities. For example, the MPI metrics of an MPI execution are based on useful computation, while those of a hybrid execution are based on the time spent outside MPI. Small differences between executions should therefore be interpreted with care.

The interactive report embeds the complete report of each programming model. When many programming models are compared, the report file is therefore larger than the report of a single analysis.